《数字逻辑》

（总复习）

厦门大学信息学院软件工程系 曾文华

2023年12月25日

课程内容

- 全书共9章：

第1章　基本知识

第2章　逻辑代数基础

第3章　集成门电路与触发器

第4章　组合逻辑电路

第5章　同步时序逻辑电路

第6章　异步时序逻辑电路

第7章　中规模通用集成电路及其应用

第8章　可编程逻辑器件（没有讲）

第9章　综合应用举例（没有讲）



第1章　基本知识

- 1.1　概述
- 1.2　数制及其转换
- 1.3　带符号二进制数的代码表示
- 1.4　几种常用的编码

第1章主要内容

- 数字信号（数字量）、模拟信号（模拟量）
- A/D转换、D/A转换
- 数字逻辑电路的特点
- 数字系统的层次结构（5级）
- 数字逻辑电路的类型：
  ① 组合逻辑电路

  ② 时序逻辑电路（同步时序逻辑电路、异步时序逻辑电路）
- PLD、CAD、EDA

- 二进制数、八进制数、十六进制数
- 十进制数、二进制数、八进制数、十六进制数之间的转换
- 原码、反码、补码
- BCD码（8421码、2421码、余3码）
- 格雷码（特点：任意两个相邻的数，其格雷码仅有1位不同）
- 奇偶校验码（只能发现1位错，但是不能纠正错误）
- ASCII码、扩展的ASCII码

第2章　逻辑代数基础

- 2.1　逻辑代数的基本概念
- 2.2　逻辑代数的基本定理和规则
- 2.3　逻辑函数表达式的形式与变换
- 2.4　逻辑函数化简

第2章主要内容

- 逻辑代数也称布尔代数、开关代数。
- 逻辑代数的5个公理（交换律、结合律、分配律、0-1律、互补律）。
- 3种基本的逻辑运算：或、与、非。
- 逻辑函数的3种表示方法：
  ① 逻辑表达式：例如，F(A,B) = /A·B + A·/B
  ② 真值表
  ③ 卡诺图

表 2.4 函数 $F=A\bar{B}+\bar{A}C$ 的真值表

A	B	C	F
0	0	0	0
0	0	1	1
0	1	0	0
0	1	1	1
1	0	0	1
1	0	1	1
1	1	0	0
1	1	1	0

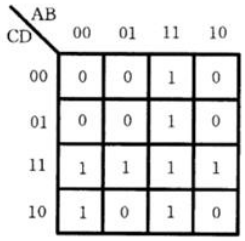


图 2.8 $F(A,B,C,D)=AB+CD+\bar{A}\bar{B}C$ 的卡诺图

- 逻辑代数的8个定理（0-1律、重叠律、吸收律、消除律、对合律、互补律、并项律、包含律）。

- 逻辑代数的重要规则：
  - 1、代入规则：任何一个含有变量A的逻辑等式，如果将所有出现A的位置都代之以同一个逻辑函数F，则等式仍然成立。
    - 例如，给定逻辑等式：A·(B+C) = A·B+A·C，若等式中的C都用(C+D)代替，则该等式仍然成立，即：A·[B+(C+D)] = A·B + A·(C+D)
  - 2、反演规则：如果将逻辑函数F表达式中所有的“·”变成“+”，“+”变成“·”，“0”变成“1”，“1”变成“0”，原变量（A）变成反变量（/A），反变量（/A）变成原变量（A），并保持原函数中的运算顺序不变，则所得到的新的函数为原函数F的反函数/F。
    - 例如，F = /A·B+C·/D，则：/F = (A+/B)·(/C+D)
  - 3、对偶规则：如果将逻辑函数F表达式中所有的“·”变成“+”，“+”变成“·”，“0”变成“1”，“1”变成“0”，并保持原函数中的运算顺序不变，则所得到的新的逻辑表达式称为函数F的对偶式，记作F’。
    - 例如：F=/A·B+/B·(C+0)，则：F’=(/A+B)·(/B+C·1)

- 复合逻辑：
  - 1、与非门：F = /(A·B)
  - 2、或非门：F = /(A+B)
  - 3、与或非门：F = /(A·B + C·D)
  - 4、异或门：F = A⊕B = /A·B + A·/B
  - 5、同或门（异或非门）：F = A⊙B = /A·/B + A·B = /(A⊕B)
- 逻辑函数表达式的基本形式：
  - 1、“与-或”表达式：F(A,B,C) = /A·B + A·/B·C + /C
  - 2、“或-与”表达式：F(A,B,C,D) = (/A + B) · (B + /C) · (A + /B + C) · /D
- 最小项（m_i）：/A·/B·/C， /A·/B·C， /A·B·/C， /A·B·C， A·/B·/C， A·/B·C， A·B·/C， A·B·C
  - 最小项的4个性质。
- 最大项（M_i）：A+B+C，A+B+/C，A+/B+C，A+/B+/C，/A+B+C，/A+B+/C，/A+/B+C，/A+/B+/C
  - 最大项的4个性质。
- 最小项与最大项的关系：$/m_i = M_i$，或者，$m_i = /M_i$。
- 逻辑函数表达式的标准形式：
  - 1、标准“与-或”表达式：F(A,B,C) = /A·/B·C + /A·B·/C + A·/B·/C + A·B·C = m1+m2+m4+m7 =∑m(1,2,4,7)
  - 2、标准“或-与”表达式：F(A,B,C) = (A+B+C) · (/A+B+/C) · (/A+/B+/C) = M0·M5·M7 =∏M(0,5,7)
- 将一个任意逻辑函数表达式转换成标准表达式有两种常用方法：
  - 1、代数转换法
  - 2、真值表转换法

- 逻辑函数化简：
  - 1、代数化简法
    - 最简“与-或”表达式应满足两个条件：
      ① 表达式中的与项个数最少；
      ② 在满足条件①的前提下，每个与项中的变量个数最少。
    - 最简“或-与”表达式应满足两个条件：
      ① 表达式中的或项个数最少；
      ② 在满足条件①的前提下，每个或项中的变量个数最少。
  - 2、卡诺图化简法
    - 第一步：画出函数的卡诺图。
    - 第二步：在卡诺图上圈出函数的全部质蕴涵项。
    - 第三步：从全部质蕴涵项中找出所有必要质蕴涵项。
    - 第四步：若函数的所有必要质蕴涵项，尚不能覆盖卡诺图上的所有1方格，则从剩余质蕴涵项中找出最简的所需质蕴涵项，使它和必要质蕴涵项一起构成函数的最小覆盖（即最简的质蕴涵项集）。
  - 3、列表化简法
    - 第一步：将函数表示成“最小项之和”（标准“与-或”表达式）的形式，并用二进制码表示每一个最小项。
    - 第二步：找出函数的全部质蕴涵项。
    - 第三步：找出函数的必要质蕴涵项。
    - 第四步：找出函数的最小覆盖。



第2章例题

- 例2.1：将逻辑函数表达式F(A,B,C) = /((A·/B+B·/C)·/(A·B))，转换为标准与-或表达式。

- 解：
  - 第一步：将函数表达式变换成一般“与-或”表达式：
    - F(A,B,C) = /((A·/B+B·/C)·/(A·B)) = /A·/B + /A·C + B·C + A·B
  - 第二步：反复使用公式X=X·(/Y+Y)，将表达式中所有非最小项的与项扩展成最小项：
    - F(A,B,C) = /A·/B + /A·C + B·C + A·B = /A·/B·(/C+C) + /A·C·(/B+B) + B·C·(/A+A) + A·B·(/C+C) = /A·/B·/C + /A·/B·C + /A·B·C + A·B·/C + A·B·C = $m_0+m_1+m_3+m_6+m_7$ = $\sum m(0,1,3,6,7)$

- 例2.2：将逻辑函数表达式F(A,B,C) = /(A·B+/A·C) + /B·C，转换为标准或-与表达式。

- 解：
  - 第一步：将函数表达式变换成一般“或-与”表达式。
    - F(A,B,C) = /(A·B+/A·C) + /B·C = (/A+/B) · (A+/B+/C) · (/A+/B+C)
  - 第二步：反复使用公式X=(X+/Y)·(X+Y)，将表达式中所有非最大项的或项扩展成最大项：
    - F(A,B,C) = (/A+/B) · (A+/B+/C) · (/A+/B+C) = (/A+/B +/C) · (/A+/B +C) · (A+/B+/C) · (/A+/B+C) = (A+/B+/C) · (/A+/B +C) · (/A+/B +/C) = $M_3 \cdot M_6 \cdot M_7 = \prod M(3,6,7)$

- 例2.3：将函数表达式F(A,B,C)=A·/B+B·/C表示成最小项表达式。

- 解：
  - 首先列出F的真值表（表2.6），然后根据真值表直接写出F的最小项表达式 F(A,B,C)=∑m(2,4,5,6)

表 2.6　函数 $F(A,B,C)=A\bar{B}+B\bar{C}$ 的真值表

A	B	C	F
0	0	0	0
0	0	1	0
0	1	0	1
0	1	1	0
1	0	0	1
1	0	1	1
1	1	0	1
1	1	1	0

$F(A,B,C)=\sum m(2,4,5,6)$

- 例2.4：将函数表达式F(A,B,C)=/A·C+A·/B·/C表示成最大项表达式。

- 解：
  - 首先列出F的真值表（表2.7），然后根据真值表直接写出F的最大项表达式F(A,B,C)=∏M(0,2,5,6,7)

表 2.7　函数 $F(A,B,C)=\bar{A}C+A\bar{B}\,\bar{C}$ 的真值表

A	B	C	F
0	0	0	0
0	0	1	1
0	1	0	0
0	1	1	1
1	0	0	1
1	0	1	0
1	1	0	0
1	1	1	0

$F(A,B,C)=\prod M(0,2,5,6,7)$

- 例2.5：用代数法化简逻辑公式：F=A·D+A·/D+A·B+/A·C+B·D+/B·E+D·E。

- 解：
  - F=A·D+A·/D+A·B+/A·C+B·D+/B·E+D·E　　A·D+A·/D=A
    =A+A·B+/A·C+B·D+/B·E+D·E　　A+A·B=A
    =A+/A·C+B·D+/B·E+D·E　　A+/A·C=A+C
    =A+C+B·D+/B·E+D·E

教材上写成：F=A+C+B·D+/B·E，有误

- 例2.6：用代数法化简逻辑公式：F=/A·/C· (/B+B·D)+A·/C·D。

- 解：
  - F=/A·/C· (/B+B·D)+A·/C·D　　/B+B·D=/B+D
    =/A·/C· (/B+D)+A·/C·D
    =/A·/B·/C+/A·/C·D+A·/C·D　　/A·/C·D+A·/C·D=/C·D
    =/A·/B·/C+/C·D

- 例2.7： 用代数法化简逻辑公式：F=A·B+A·/C+/B·C+B·/C+/B·D+B·/D+A·D·E。

- 解：
  - F=A·B+A·/C+/B·C+B·/C+/B·D+B·/D+A·D·E　　B+/C=/(/B·C)
    =A·/(/B·C)+/B·C+B·/C+/B·D+B·/D+A·D·E　　A/(/B·C)+/B·C=A+/B·C
    =A+/B·C+B·/C+/B·D+B·/D+A·D·E　　A+A·D·E=A
    =A+/B·C+B·/C+/B·D+B·/D
    =A+/B·C·(/D+D)+B·/C+/B·D+B·/D·(C+/C)
    =A+/B·C·/D+/B·C·D+B·/C+/B·D+B·C·/D+B·/C·/D
    =A+(/B·C·/D+B·C·/D)+(/B·D·C+/B·D)+(B·/C+B·/C·/D)
    =A+C·/D+/B·D+B·/C

- 例2.8： 用代数法化简逻辑公式：F=(A+B)·(A+/B)·(B+C)·(B+C+D)。

- 解：
  - F=(A+B)·(A+/B)·(B+C)·(B+C+D)
    =(A+B)·(A+/B)·(B+C)
    =A·(B+C)

(B+C)·(B+C+D)=B+C
(A+B)·(A+/B)=A

X·(X+Y)=X·X+X·Y=X+X·Y=X·(1+Y)=X

- 例2.9：用代数法（两次对偶）化简逻辑公式：F=(A+/B)·(/A+B)·(B+C)·(/A+C)。

- 解：
  - 先求F的对偶式F’，并进行化简（“与-或”表达式的化简）：
    - F’=A·/B+/A·B+B·C+/A·C
      =A·/B+/A·B+(B+/A)·C　　B+/A=/(A·/B)
      =A·/B+/A·B+/(A·/B)·C　　A·/B+/(A·/B)·C=A·/B+C
      =A·/B+/A·B+C
  - 再对F’求对偶，得到F：
    - F = (F’)’ = (A+/B)·(/A+B)·C

(X+Y)·(X+/Y)=X·X+X·/Y+Y·X+Y·/Y=X+X·/Y+X·Y+0=X· (1+/Y)+X·Y=X+X·Y=X· (1+Y)=X

- 例2.10：用卡诺图法化简逻辑公式：F(A,B,C,D) = ∑ m(0,3,5,6,7,10,11,13,15）。

- 解：
  - 第一步：画出函数F的卡诺图，如图2.13(a)。
  - 第二步：在卡诺图上圈出函数的全部质蕴涵项（卡诺圈），如图2.13(b)；先画2个大的卡诺圈（包含4个小方格），再画2个小卡诺圈（包含2个小方格），剩余1个左上角的最小卡诺圈（包含1个小方格）；共有5个卡诺圈（5个质蕴涵项）。
  - 第三步：从全部质蕴涵项中找出所有必要质蕴涵项，如图2.13(c)；图中标有“*”的最小项（小方格）称为必要最小项；包含必要最小项的质蕴涵项称为必要质蕴涵项；共有5个必要质蕴涵项。
  - 因为5个必要质蕴涵项覆盖了卡诺图上的所有1方格，因此： F(A,B,C,D) = /A·/B·/C·/D + /A·B·C + A·/B·C + B·D + C·D。

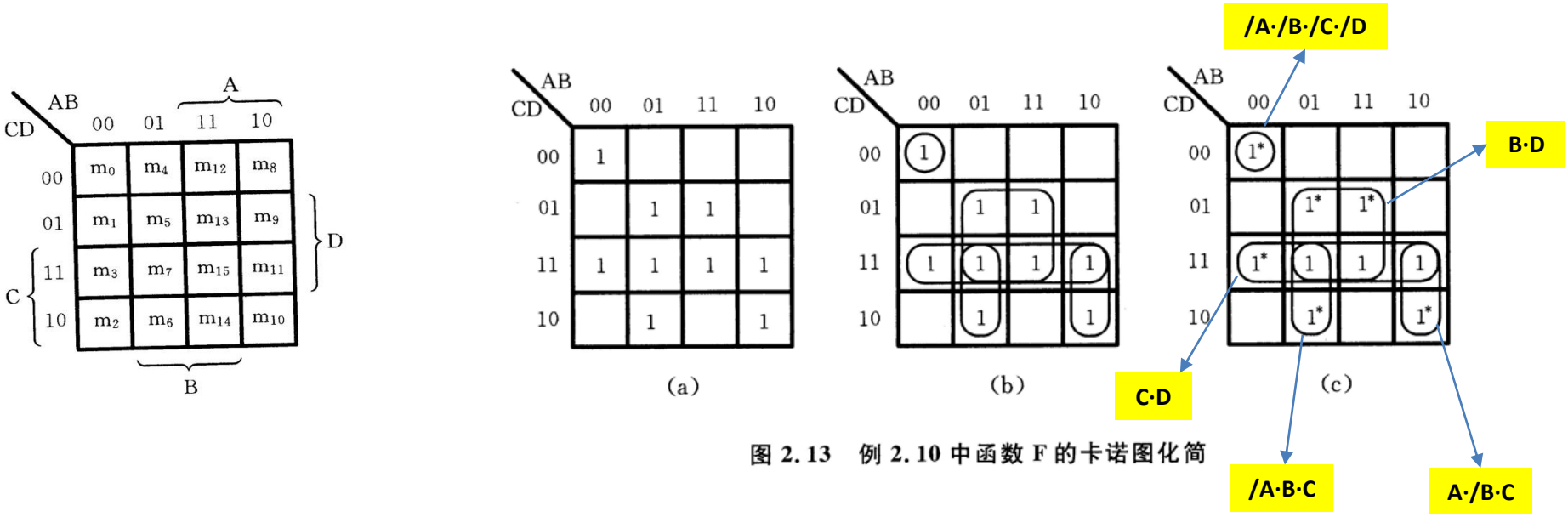


图 2.13　例 2.10 中函数 F 的卡诺图化简

- **例2.11**：用卡诺图法化简逻辑公式：**F(A,B,C,D) = /A·C·D + /B·C·/D + /A·B·C·/D + A·/B·/D + A·B·/C·/D。**

- 解：
  - 第一步：画出函数F的卡诺图，如图2.14(a)。
  - 第二步：在卡诺图上圈出函数的全部质蕴涵项（卡诺圈），如图2.14(b)；先画1个大的卡诺圈（包含4个小方格），再画3个小卡诺圈（包含2个小方格）；共有4个卡诺圈（4个质蕴涵项）。
  - 第三步：找出所有必要质蕴涵项，如图2.14(c)；图中标有“*”的最小项（小方格）称为必要最小项；包含必要最小项的质蕴涵项称为必要质蕴涵项；共有2个必要质蕴涵项（红框），这2个必要质蕴涵项不能覆盖卡诺图上的所有1方格。
  - 第四步：确定除必要质蕴涵项外的最简质蕴涵项（1个，有2种形式：A·/B·/D或/B·C·/D）；求出最简蕴涵项集（最小覆盖：2个必要质蕴涵项+1个最简质蕴涵项）。
  - 因此：F(A,B,C,D) = /A·C+A·/C·/D+A·/B·/D；或者：F(A,B,C,D) = /A·C+A·/C·/D+/B·C·/D；可见函数的最简“与-或”表达式不是唯一的。

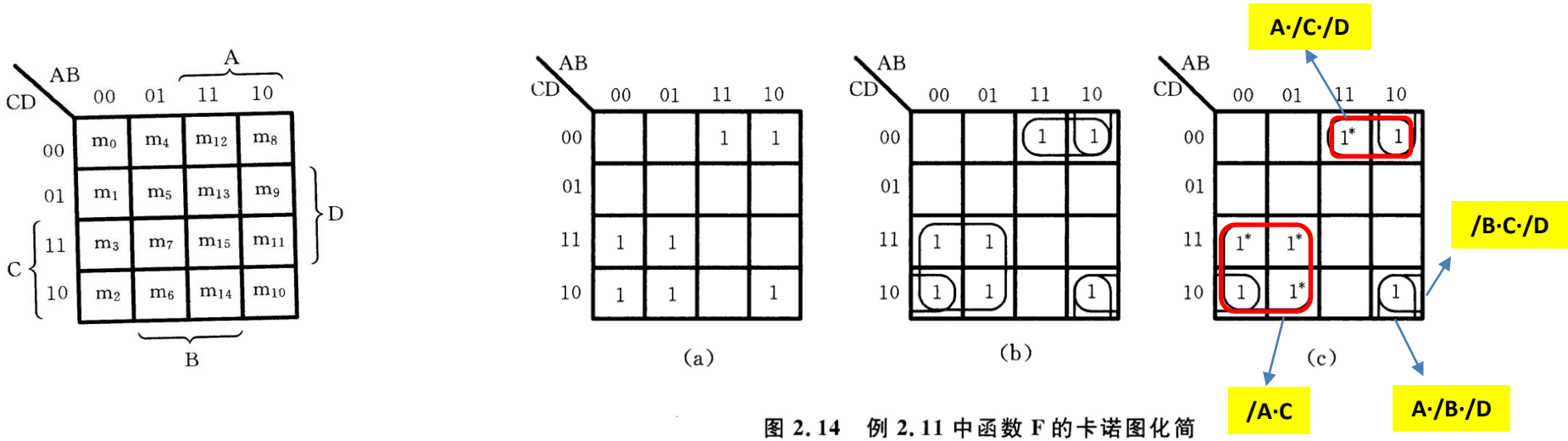
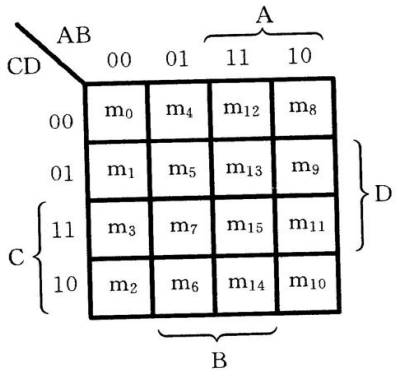


图 2.14　例 2.11 中函数 F 的卡诺图化简

- **例2.12**：用卡诺图法化简逻辑函数F(A,B,C,D) = A·/C + A·D + /B·/C + /B·D的最简“或-与”表达式。

- 解：
  - 第一步：画出卡诺图，如图2.15；先根据逻辑函数F的表达式标出1小方格，再标出0小方格。
  - 第二步：合并卡诺图中的0小方格（卡诺圈），如图2.15。
  - 第三步：根据卡诺圈，得到反函数/F的最简“与-或”表达式：/F(A,B,C,D) = /A·B + C·/D。
  - 第四步：对/F取反，即可得到F的最简“或-与”表达式：F(A,B,C,D) = (A+/B)·(/C+D)。



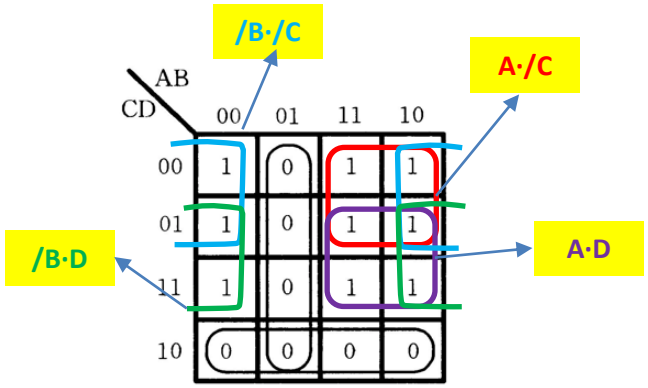


图 2.15　例 2.12 中函数 F 的卡诺图

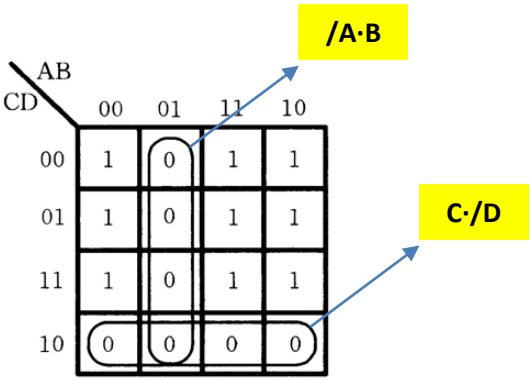
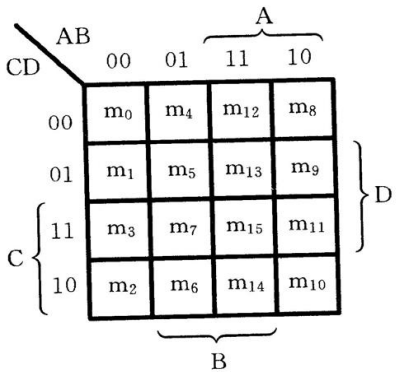


图 2.15　例 2.12 中函数 F 的卡诺图

- 例2.13：用卡诺图法化简逻辑函数F(A,B,C,D) = (/A+D)·(B+/D)·(A+B)的最简“或-与”表达式。
- 解：
  - 第一步：根据反演规则，求F的反函数：/F(A,B,C,D) = A·/D + /B·D + /A·/B。
  - 第二步：画出/F的卡诺图，如图2.16。
  - 第三步：合并卡诺图中的1小方格（卡诺圈），如图2.16。
  - 第四步：根据卡诺圈，得到反函数/F的最简“与-或”表达式：/F(A,B,C,D) = /B + A·/D。
  - 第五步：对/F取反，即可得到F的最简“或-与”表达式：F(A,B,C,D) = B·(/A+D)。



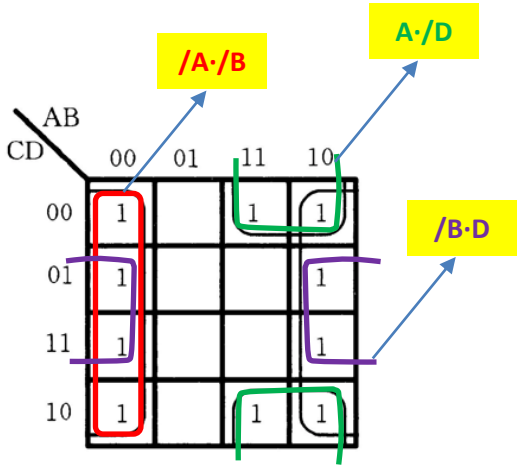


图 2.16　例 2.13 中反函数 $\bar{F}$ 的卡诺图

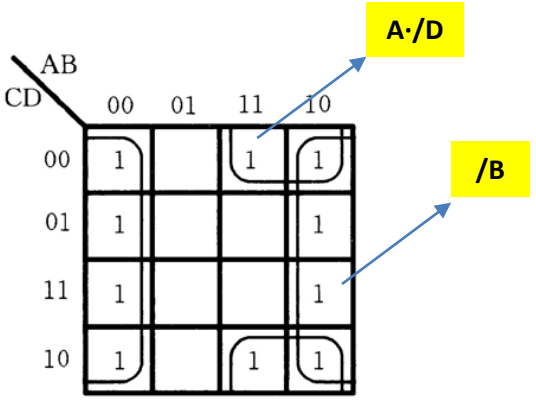


图 2.16　例 2.13 中反函数 $\bar{F}$ 的卡诺图

- **例2.14**：用卡诺图法化简逻辑函数F(A,B,C,D) = ∏M(3,4,6,7,11,12,13,14,15)的最简“与-或”表达式和最简“或-与”表达式。

- 解：
  - 第一步：画出F的卡诺图，如图2.17(a)；F为标准“或-与”表达式，对应的最大项位置填0（M_3、M_4、M_6、M_7、M_{11}、M_{12}、M_{13}、M_{14}、M_{15}），其他位置填1。
  - 第二步：合并卡诺图中的1小方格（卡诺圈），如图2.17(b)。
  - 第三步：根据卡诺圈，得函数F的最简“与-或”表达式：F(A,B,C,D) = /B·/D+/B·/C+/A·/C·D。
  - 第四步：合并卡诺图中的0小方格（卡诺圈），如图2.17(c)。
  - 第五步：根据卡诺圈，得到函数/F的最简“与-或”表达式：/F(A,B,C,D) = A·B+C·D+B·/D。
  - 第六步：对/F取反，即可得到函数F的最简“或-与”表达式：F(A,B,C,D) = (/A+/B)·(/C+/D)·(/B+D)。

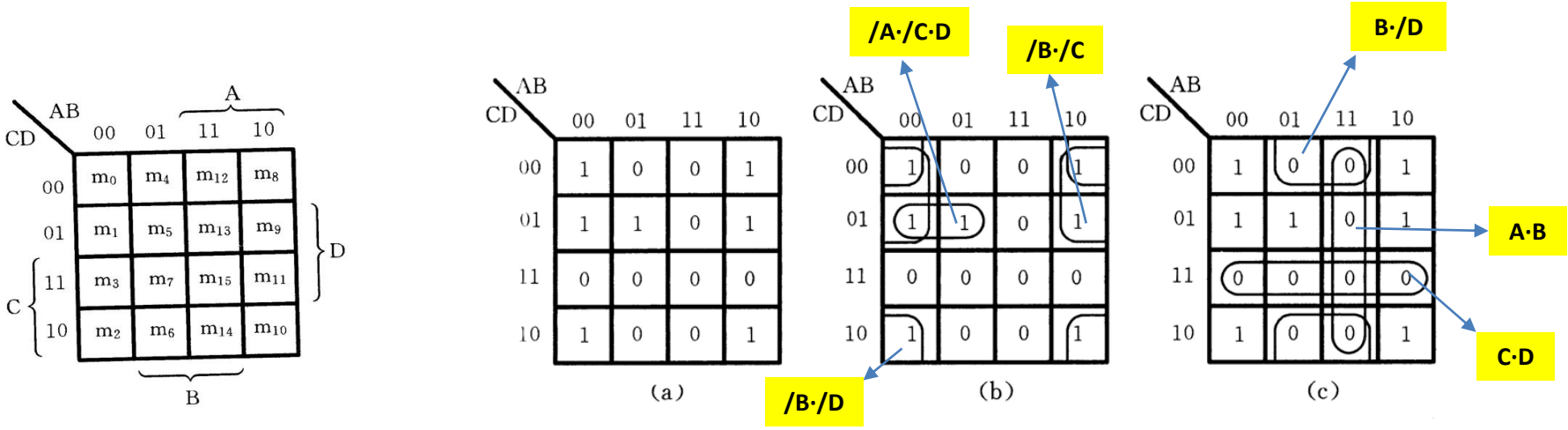


图 2.17　例 2.14 中函数 F 的卡诺图

- 例2.15：用列表法化简逻辑函数F(A,B,C,D) = ∑m(0,5,7,8,9,10,11,14,15) 。

- 解：
  - 第一步：用二进制码表示函数F的每一个最小项，如表2.8。

例 2.15　F 的部分真值表

项号	A	B	C	D	F	项号	A	B	C	D	F
0	0	0	0	0	1	10	1	0	1	0	1
5	0	1	0	1	1	11	1	0	1	1	1
7	0	1	1	1	1	14	1	1	1	0	1
8	1	0	0	0	1	15	1	1	1	1	1
9	1	0	0	1	1						

  - 第二步：找出函数F的全部质蕴涵项。
    - （1）将表2.8的9个最小项重新排列：4个变量全部为0的编为第0组，4个变量只有1个为1的编为第1组，4个变量有2个为1的编为第2组，4个变量有3个为1的编为第3组，4个变量全部为1的编为第4组；见表2.9的“(I)最小项”。

表 2.9　质蕴涵项产生表

(Ⅰ)最小项				(Ⅱ)(n−1)个变量的与项				(Ⅲ)(n−2)个变量的与项			
组号	m_i	ABCD	p_i	组号	$\sum m_i$	ABCD	p_i	组号	$\sum m_i$	ABCD	p_i
0	0	0000	✓	0	0,8	−000	p_5	1	8,9,10,11	10−−	p_2
1	8	1000	✓	1	8,9	100−	✓	2	10,11,14,15	1−1−	p_1
2	5	0101	✓		8,10	10−0	✓				
	9	1001	✓	2	5,7	01−1	p_4				
	10	1010	✓		9,11	10−1	✓				
3	7	0111	✓		10,11	101−	✓				
	11	1011	✓		10,14	1−10	✓				
	14	1110	✓	3	7,15	−111	p_3				
4	15	1111	✓		11,15	1−11	✓				
					14,15	111−	✓				

- （2）比较“(I)最小项”中相邻组号中的两个最小项，如果只有1位是不同的，则进行合并，得到3个变量的与项（n-1=3）：

  - 组0和组1是相邻组号，两个最小项m_0和m_8只有1位是不同的，将其合并为“—0000”。

  - 组1和组2是相邻组号，两个最小项m_1和m_5有3位是不同的，不合并；两个最小项m_8和m_9只有1位是不同的，将其合并为“100—”；两个最小项m_8和m_{10}只有1位是不同的，将其合并为“10—0”。

  - 组2和组3是相邻组号，m_5和m_7、m_9和m_{11}、m_{10}和m_{14}都只有1位是不同的，将其分别合并为“01—1”、“10—1”、“101—”、“1—10”；m_5和m_{14}、m_9和m_7、m_9和m_{14}、m_{10}和m_7都有多位是不同的，不合并。

  - 组3和组4是相邻组号，m_7和m_{15}、m_{11}和m_{15}都只有1位是不同的，将其分别合并为“—111”、“1—11”、“111—”。

  - 将合并后的10个3变量的与项进行编号：3个都是0的编为第0组，2个0、1个1的编为第1组，1个0、2个1的编为第2组，3个都是1的编为第3组；见表2.9的“(II)(n-1)个变量的与项”；同时，对9个最小项进行标注，因为9个最小项都参与了合并，因此，表2.9中的第4列p_i全部打“√”。

表 2.9　质蕴涵项产生表

(Ⅰ)最小项				(Ⅱ)(n−1)个变量的与项				(Ⅲ)(n−2)个变量的与项			
组号	m_i	ABCD	p_i	组号	$\sum m_i$	ABCD	p_i	组号	$\sum m_i$	ABCD	p_i
0	0	0000	√	0	0,8	−000	p_5	1	8,9,10,11	10−−	p_2
1	8	1000	√	1	8,9	100−	√	2	10,11,14,15	1−1−	p_1
2	5	0101	√		8,10	10−0	√				
	9	1001	√	2	5,7	01−1	p_4				
	10	1010	√		9,11	10−1	√				
3	7	0111	√		10,11	101−	√				
	11	1011	√		10,14	1−10	√				
	14	1110	√	3	7,15	−111	p_3				
4	15	1111	√		11,15	1−11	√				
					14,15	111−	√				

- （3）比较“(II)(n-1)个变量的与项”中相邻组号中的两个与项，如果只有1位是不同的，则进行合并，得到2个变量的与项（n-2=2）：

  - 组0和组1是相邻组号，“—000”和“100—”、“—000”和“10—0”都有多位是不同的，不合并。

  - 组1和组2是相邻组号，“100—”和“101—”、“[illegible]0”和“10—1”都只有1位是不同的，将其合并为“10——”；其余，“100—”和“01—1”、“[illegible]”和“10[illegible]”和“1—10”、“10—0”和“01—1”、“10—0”和“101—”、“10—0”和“1—10”[illegible]位是[illegible]的，不合并。

  - 组2和组3是相邻组号，“101—”和“[illegible]”和“1—10”和“1—11”都只有1位是不同的，将其合并为“1—1—”；其余，“01—1”和“—111”、“01—1[illegible]1”、“01—1”和“111—”、“10—1”和“—111”、“10—1”和“1—11”、“10—1”和“111—”、[illegible]“—111”、“101—”和“1—11”、“1—10”和“—111”、“1—10”和“111—”都有多位是不同的[illegible]合并。

  - 将合并后的2个2变量的与项进行[illegible]有1个1、[illegible]编为第1组，有2个1的编为第2组；见表2.9的“(III)(n-2)个变量的与项”；同时，对10个3变量的[illegible]项进行标注，因为[illegible]3变量的与项参与了合并，因此，表2.9中第8列的p_i中7个打“√”。

表 2.9　质蕴涵项产生表

(Ⅰ)最小项				(Ⅱ)(n−1)个变量的与项				(Ⅲ)(n−2)个变量的与项			
组号	m_i	ABCD	p_i	组号	$\sum m_i$	ABCD	p_i	组号	$\sum m_i$	ABCD	p_i
0	0	0000	√	0	0,8	−000	p_5	1	8,9,10,11	10−−	p_2
1	8	1000	√	1	8,9	100−	√	2	10,11,14,15	1−1−	p_1
2	5	0101	√		8,10	10−0	√				
	9	1001	√	2	5,7	01−1	p_4				
	10	1010	√		9,11	10−1	√				
3	7	0111	√		10,11	101−	√				
	11	1011	√		10,14	1−10	√				
	14	1110	√	3	7,15	−111	p_3				
4	15	1111	√		11,15	1−11	√				
					14,15	111−	√				

- （4）比较“(III)(n-2)个变量的与项”中相邻组号中的两个与项，“10——”和“1—1—”有多位是不同的，无法合并。

- （5）表2.9中没有参与合并的“最小项”、“(n-1)个变量的与项”、“(n-2)个变量的与项”，即为全部质蕴涵项（即没有打“√”的），分别是：
  - $p_1 = \sum m(10,11,14,15)$ = “1—1—” =
  - $p_2 = \sum m(8,9,10,11)$ = “10——” = A·/L
  - $p_3 = \sum m(7,15)$ = “—111” = B·C·D
  - $p_4 = \sum m(5,7)$ = “01—1” = /A·B·D
  - $p_5 = \sum m(0,8)$ = “—000” = /B·/C·/D

表 2.9　质蕴涵项产生表

（Ⅰ）最小项				（Ⅱ）$(n-1)$个变量的与项				（Ⅲ）$(n-2)$个变量的与项			
组号	m_i	ABCD	p_i	组号	$\sum m_i$	ABCD	p_i	组号	$\sum m_i$	ABCD	p_i
0	0	0000	✓	0	0,8	—000	p_5	1	8,9,10,11	10— —	p_2
1	8	1000	✓	1	8,9	100—	✓	2	10,11,14,15	1—1—	p_1
2	5	0101	✓		8,10	10—0	✓				
	9	1001	✓	2	5,7	01—1	p_4				
	10	1010	✓		9,11	10—1	✓				
3	7	0111	✓		10,11	101—	✓				
	11	1011	✓		10,14	1—10	✓				
	14	1110	✓	3	7,15	—111	p_3				
4	15	1111	✓		11,15	1—11	✓				
					14,15	111—	✓				

– 第三步：找出函数F的全部必要质蕴涵项。
  » 建立必要质蕴涵项产生表，如表2.10。

  » 表2.10中，第2行为F的9个最小项，F(A,B,C,D) = ∑m(0,5,7,8,9,10,11,14,15)

  » 第3～7行中的“x”对应p_1～p_5包含最小项情况：
    - p_1=∑m(10,11,14,15)
    - p_2=∑m(8,9,10,11)
    - p_3=∑m(7,15)
    - p_4=∑m(5,7)
    - p_5=∑m(0,8)

  » 依次检查表2.10中的第2～10列，“x”，如果该列只有1个“x”，则标注为“x外加圆圈”，共有4个，其对应的行（p_1*、p_2*、p_4*、p_5*）即为必要质蕴涵项，如表2.10。

  » 检查p_1*=∑m(10,11,14,15)、p_2*=∑m(8,9,10,11)、p_4*=∑m(5,7)、p_5*=∑m(0,8)覆盖最小项的情况，将覆盖的情况在表2.10的最后一行进行标注（打√），全部覆盖。

表 2.10　必要质蕴涵项产生表

p_i	m_i								
	0	5	7	8	9	10	11	14	15
p_1*						×	×	⊗	×
p_2*				×	⊗	×	×		
p_3			×						×
p_4*		⊗	×						
p_5*	⊗			×					
覆盖情况	√	√	√	√	√	√	√	√	√

– 第四步：找出函数F的最小覆盖。
  » 由表2.10可知，必要质蕴涵项（p_1*、p_2*、p_4*、p_5*）覆盖了全部最小项；因此，函数F最终简化为：
    - F = p_1* + p_2* + p_4* + p_5* = A·C + A·/B + /A·B·D + /B·/C·/D

- 例2.16：已知函数F(A,B,C,D) = ∑m(0,3,4,5,6,7,8,10,11)的必要质蕴涵项产生表如表2.11所示，求出该函数的最小覆盖 。

- 解：
  - 根据表2.11可知，必要质蕴涵[illegible]*，但是[illegible]覆盖了最小项m_4、m_5、m_6、m_7，没有覆盖全部的最小项。
  - 从表2.11中去掉p_1*行，以及其对[illegible]、m_6、m_7列，得到表2.12（所需质蕴涵项产生表）。
  - 分析表2.12，p_4行完全包含在p_3行[illegible]全包含在p_6行中，根据行消去规则，可以去掉p_4和p_7行，得到表2.13（消去多余行后的所需质蕴[illegible]产生[illegible]。注：表2.12中没有可以消去的列。



行消去规则：对于所需质蕴涵项产生表中的任意质蕴涵项p_i和p_j，若p_i行中的“x”号完成包含在pj行中，即$p_i \in p_j$，则可消去p_i行。这是因为选取了质蕴涵项p_j后，不仅可以覆盖质蕴涵项p_i所能覆盖的最小项，而且还可以覆盖其他最小项。

表 2.11　必要质蕴涵项产生表

p_i	m_i								
	0	3	4	5	6	7	8	10	11
p_1*			×	⊗	⊗	×			
p_2								×	×
p_3		×							×
p_4		×				×			
p_5							×	×	
p_6	×						×		
p_7	×		×						
覆盖情况			✓	✓	✓	✓			

表 2.12　所需质蕴涵项产生表

p_i	m_i				
	0	3	8	10	11
p_2				×	×
p_3		×			×
p_4	—	×	—	—	—
p_5			×	×	
p_6	×		×		
p_7	-×-	—	—	—	—

表 2.13　消去多余行后的所需质蕴涵项产生表

p_i	m_i				
	0	3	8	10	11
p_2			|	×	×
p_3		×	|		×
p_5			×	×	|
p_6	×		×		|

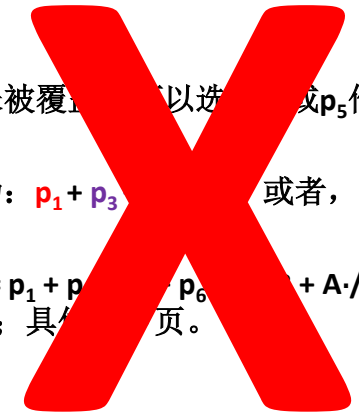
- 进一步分析表2.13，m_0列完全包含在m_8列中，m_3列完全包含在m_{11}列中，根据列消去规则，可以去掉m_8列和m_{11}列，得到表2.14（消去多余行和多余列后的所需质蕴涵项产生表）。

- 由表2.14可见， m_0和m_3所对应的列，均只有一个“x”号，因此，p_3^{**}和p_6^{**}是必须选取的质蕴涵项，也称为二次质蕴涵项。

- 此时，还剩下最小项m_{10}未被覆盖，可以选取p_5作为所需质蕴涵项（注：p_2和p_5的复杂度是一样的）。

- 因此，函数F的最小覆盖为：$p_1 + p_3$ 或者， $p_1 + p_3 + p_6 + p_5$

- 即，函数F可以化简为：F= $p_1 + p_3 + p_6$ + A·/B·C + /B·C·D + /B·/C·/D；或者，F= $p_1 + p_3 + p_5 + p_6$ = /A·B + /B·C·D + A·/B·/D + /B·/C·/D；具体见 页。



列消去规则：对于所需质蕴涵项产生表中的任意最小项m_i和m_j，若m_i列中的“x”号完成包含在m_j列中，即$m_i \in m_j$，则可消去m_j列。这是因为选取了覆盖m_i的质蕴涵项后，一定能覆盖m_j，反之则不一定。

表 2.13　消去多余行后的所需质蕴涵项产生表

p_i	m_i				
	0	3	8	10	11
p_2			|	×	×
p_3		×	|		×
p_5			×	×	|
p_6	×		×		|

表 2.14　消去多余行和多余列后的所需质蕴涵项产生表

p_i	m_i		
	0	3	10
p_2			×
p_3**		⊗	
p_5			×
p_6**	⊗		

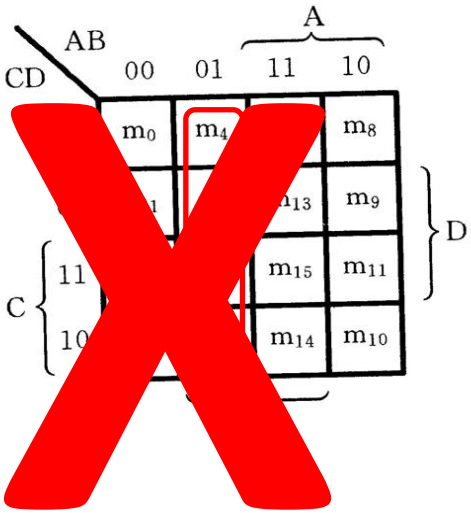
表 2.11　必要质蕴涵项产生表

p_i	m_i								
	0	3	4	5	6	7	8	10	11
p_1*			×	⊗	⊗	×			
p_2								×	×
p_3		×							×
p_4		×				×			
p_5							×	×	
p_6	×						×		
p_7	×		×						
覆盖情况			✓	✓	✓	✓			

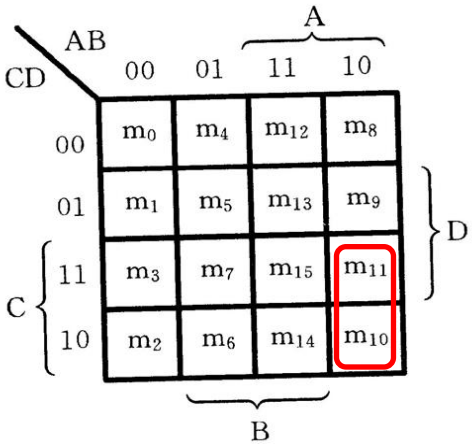
表 2.11　必要质蕴涵项产生表

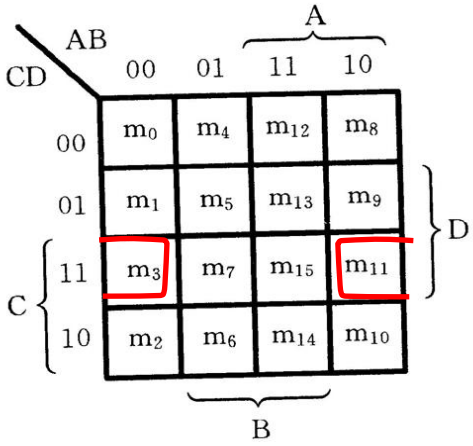
p_i	m_i								
	0	3	4	5	6	7	8	10	11
p_1 *			×	⊗	⊗	×			
p_2								×	×
p_3		×							×
p_4		×				×			
p_5							×	×	
p_6	×						×		
p_7	×		×						
覆盖情况			✓	✓	✓	✓			

$p_1 = /A \cdot B$

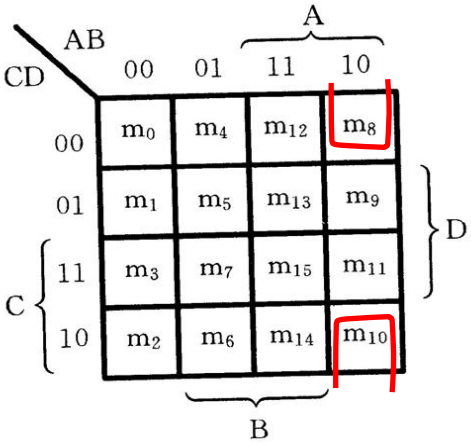


$p_2 = A \cdot /B \cdot C$

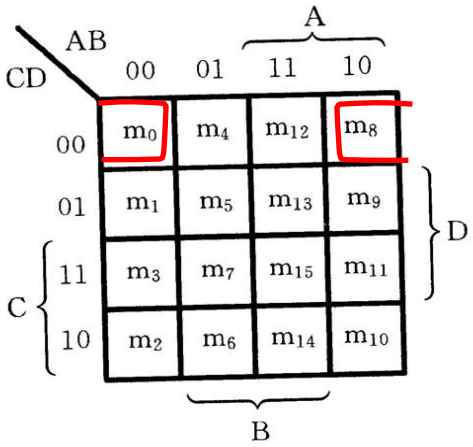




$p_3 = /B \cdot C \cdot D$



$p_5 = A \cdot /B \cdot /D$



$p_6 = /B \cdot /C \cdot /D$

第3章　集成门电路与触发器

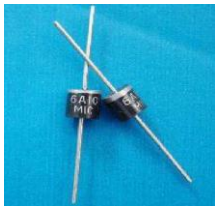
- 3.1　数字集成电路的分类
- 3.2　半导体器件的开关特性
- 3.3　逻辑门电路
- 3.4　触发器

第3章主要内容

- 集成电路的分类：
  - 根据采用的半导体器件分类：双极型集成电路（TTL、ECL、I2L）；单极型集成电路（PMOS、NMOS、CMOS）。
  - 根据集成规模的大小分类：SSI、MSI、LSI、VLSI。
  - 根据设计方法和功能定义分类：非用户定制电路（标准集成电路）；全用户定制电路（ASIC）；半用户定制电路（PLD）。

- 晶体二极管：由1个PN结构成。

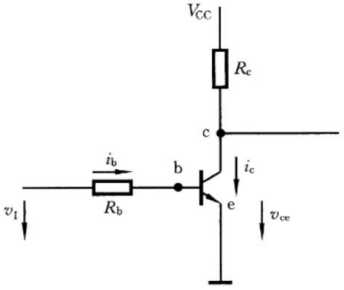




- 晶体二极管的开关特性：
  - 静态开关特性（导通状态、截止状态）。
  - 动态开关特性（反向恢复时间、开通时间）。

- 晶体三极管：由2个PN结构成，共用的1个电极称为基极（b），其他的2个电极分别是集电极（c）和发射极（e）。

- 晶体三极管的开关特性：
  - 静态开关特性（截止状态、放大状态、饱和状态）。
  - 动态开关特性（开通时间、关闭时间）。





- 简单逻辑门电路：

  ① 与门：采用2个二极管和1个电阻实现。

  ② 或门：采用2个二极管和1个电阻实现。

  ③ 非门：采用1个三极管和3个电阻实现。

- TTL集成逻辑门电路：

  ① TTL与非门：3输入TTL与非门电路由4个二极管（D1、D2、D3、D4）、4个晶体管（T1、T2、T3、T4）、4个电阻（R1、R2、R3、R4）组成。TTL与非门集成电路芯片：7400、7410、7420。

  ② TTL非门：由1个二极管、4个三极管、4个电阻构成。

  ③ TTL或非门：2输入TTL或非门由1个二极管、6个三极管、4个电阻构成。

  ④ TTL与或非门：2-2输入TTL与或非门由1个二极管、6个晶体管、5个电阻构成。

  ⑤ TTL异或门：7486芯片（4个2输入异或门）。

  ⑥ TTL与门：7411芯片（3个3输入与门）。

  ⑦ TTL或门：7432芯片（4个2输入或门）。

- 两种特殊的门电路：

  ① 集电极开路与非门（OC门）：可以实现“线与”逻辑，还可以实现电平转换。

  ② 三态输出门（三态门，TS门）：有高电平、低电平、高阻状态等三种状态。用三态门可以构成单向数据传输总线、双向数据传输总线。

- MOS晶体管有3个电极：栅极（G）、漏极（D）、源极（S），由栅极电压控制漏源电流。

- MOS晶体管有3个工作状态：截止状态、非饱和状态、饱和状态。

- CMOS集成逻辑门电路：
  ① CMOS反相器：由一个P沟道增强型MOS管TP和一个N沟道增强型MOS管TN串联组成。

  ② CMOS与非门：2输入CMOS与非门由两个串联的NMOS管和两个并联的PMOS管构成。

  ③ CMOS或非门：2输入CMOS或非门由两个并联的NMOS管和两个串联的PMOS管构成。

  ④ CMOS三态门：CMOS三态非门是在CMOS反相器的基础上，增加了控制电路。

  ⑤ CMOS传输门：由一个NMOS管和一个PMOS管并接构成。

- 正逻辑和负逻辑：
  – 正逻辑：高电平表示逻辑1，低电平表示逻辑0。

  – 负逻辑：高电平表示逻辑0，低电平表示逻辑1。

  – 即若将一个逻辑门的输出和所有输入都反向，则正逻辑变为负逻辑。

- 触发器是一种具有记忆功能的电子器件，触发器是存储一位二进制信息的理想器件。

- 触发器的3个特点：
  ① 有两个互补的输出端Q和/Q。
  ② 有两个稳定状态：Q=1、/Q=0时，称为“1”状态；Q=0、/Q=1时，称为“0”状态。当输入信号不发生变化时，触发器状态稳定不变。
  ③ 在一定输入信号作用下，触发器可以从一个稳定状态转移到另一个稳定状态；输入信号撤销后，保持新的状态不变。

- 触发器可以分为4种类型： R-S触发器、D触发器、J-K触发器、T触发器。

- 基本R-S触发器（Reset， Set）： 可以用2个与非门构成，也可以用2个或非门构成。

- 钟控触发器： 具有时钟脉冲（CP，Clock Pulse）控制端的触发器称为时钟控制触发器，简称钟控触发器，也称定时触发器。

- 简单结构的钟控触发器：
  ① 钟控R-S触发器。
  ② 钟控D触发器（只有1个输入端）。
  ③ 钟控J-K触发器（具有翻转功能）。
  ④ 钟控T触发器（只有1个输入端，具有翻转功能）。

- 电位触发方式：简单结构钟控触发器的状态转移是被控制在一个约定的时间间隔内，而不是控制在某一时刻进行；触发器的这种控制方式称为电位触发方式。

- “空翻”现象： 电位触发方式的钟控触发器可能会出现“空翻”现象（在同一个时钟脉冲作用期间，触发器的状态会发生两次或两次以上的变化。原因是：同一个时钟脉冲作用期间，CP一直是1，只要输入发生变化，输出就可能发生变化）。



- 主从钟控触发器：

① 主从R-S触发器：主从R-S触发器的状态变化发生在CP由1变为0的时刻（下降沿触发），其他时刻触发器状态保持不变，从而克服了“空翻”现象。

② 主从J-K触发器：是对主从R-S触发器稍加修改后得到的，也克服了“空翻”现象。主从J-K触发器存在“一次翻转”现象：所谓“一次翻转”是指在时钟脉冲作用（CP=1）期间，主触发器的状态只能根据输入信号的变化改变一次。

X“一次翻转”现象

- 维持-阻塞钟控触发器：

– 维持-阻塞钟控触发器是一种广泛使用的边沿触发器（边沿触发器：仅仅在时钟脉冲CP的上升沿或下降沿时刻响应输入信号，从而大大提高了触发器的抗干扰能力）。

– 维持-阻塞D触发器：维持-阻塞D触发器不仅克服了“空翻”现象，而且由于是边沿触发，抗干扰能力强，因而应用十分广泛。

第4章　组合逻辑电路

- 4.1　组合逻辑电路分析
- 4.2　组合逻辑电路设计
- 4.3　组合逻辑电路的险象

第4章主要内容

- 组合逻辑电路是指电路在任何时刻产生的稳定输出值，仅仅取决于该时刻各输入值的组合，而与过去的输入值无关。

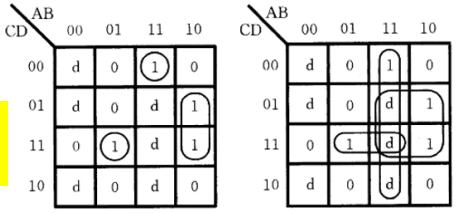
- 组合逻辑电路的特点：
  ① 由逻辑门电路组成，不包含任何记忆元件；
  ② 信号是单向传输的，不存在任何反馈回路。

- 组合逻辑电路分析的一般步骤：
  ① 根据逻辑电路图写出输出函数表达式；
  ② 化简输出函数表达式（见第2章的“2.4 逻辑函数化简”）；
  ③ 列出输出函数真值表；
  ④ 功能评述。

- 组合逻辑电路设计（逻辑综合）的一般步骤：
  ① 建立给定问题的逻辑描述（建立逻辑表达式的方法有两种：真值表法、分析法）；
  ② 求出逻辑函数的最简表达式；
  ③ 选择逻辑门类型并进行逻辑函数变换；
  ④ 画出逻辑电路图。

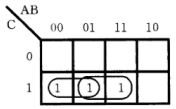
- 设计中几个实际问题的处理：
  - 1、包含无关条件的组合逻辑电路设计：实际问题中，有一些输入变量的取值组合是不会出现的；或者有一些输入变量的取值组合，其对应的输出值并不关心。通常将这类问题称为包含无关条件的逻辑问题，与这些输入取值组合对应的最小项称为无关最小项（无关项、任意项，d），描述这类问题的逻辑函数称为包含无关条件的逻辑函数。

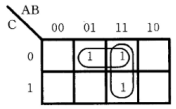
没有利用无关项：F = A·/B·D + A·B·/C·/D + /A·B·C·D

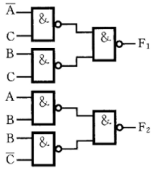


利用无关项：在卡诺图法化简时，将无关项画到卡诺圈中，这样可以是逻辑公式更简单；F = A·B + A·D + B·C·D

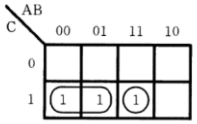
  - 2、多输出函数的组合逻辑电路设计：设计多输出函数（多个输出）的组合逻辑电路时，因为各输出函数之间往往存在相互联系，因此应该将它们作为一个整体考虑，而不应该将其截然分开。关键是在函数化简时，要找出各输出函数的共用项，以便在逻辑电路中实现对逻辑门的共享，从而使电路整体结构最简。

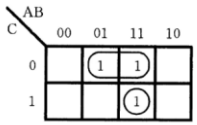


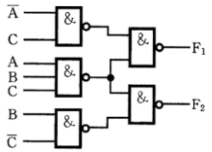




分开设计



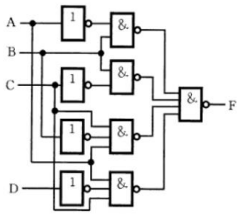


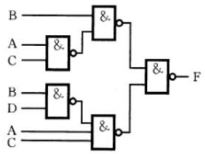


整体设计：找出共用项，这样使电路更简单

  - 3、无反变量提供的组合逻辑电路设计：有一些输出函数的逻辑表达式中含有输入变量的反变量，如果采用非门来实现反变量，则会增加电路的复杂性。可以通过逻辑公式变换的方式，在不需要非门的情况下，实现这种逻辑电路。

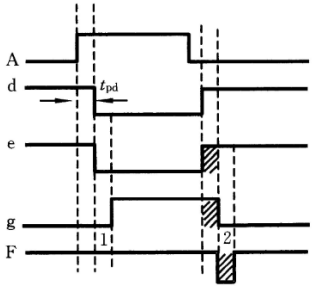
有非门的电路； F= /A·B + B·/C + A·/B·C + A·C·/D



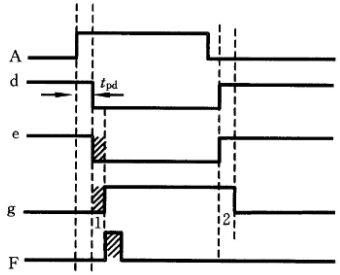


通过逻辑公式变换，不需要非门的电路； F= /(/(B·/(A·C)) + /(A·C·/(B·D)))

- 组合逻辑电路的险象：
  - 在实际逻辑电路中，信号经过同一电路中的不同路径所产生的时延通常是不一样的，即输入信号经过不同路径到达输出端的时间有先后，这种现象称为竞争现象。
  - 由于电路中存在竞争现象，使得输入信号的变化可能引起输出信号出现非预期的错误输出，这个现象称为险象。险象包括“0”型险象和“1”型险象。
  - 通常将不产生错误输出的竞争称为非临界竞争，而导致错误输出的竞争称为临界竞争。



“0”型险象



“1”型险象

- 险象的判断方法：
  - 1、代数法：如果逻辑函数表达式中有某个变量的原变量（X）和反变量（/X），则通过消去逻辑函数表达式中的其他变量，再看逻辑函数表达式是否会变为“X+/X”或者“X·/X”的形式，若是，则说明该逻辑函数电路可能会产生险象。
  - 2、卡诺图法：当逻辑函数采用“与-或”表达式时，如果卡诺图中的两个卡诺圈存在“相切”关系，则该电路可能产生险象。

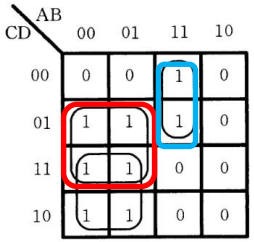


图 4.22　卡诺图

- 消除险象的方法有3种：
  - 1、用增加冗余项的方法消除险象：在逻辑函数表达式中，“或”上多余的与项，或者“与”上多余的或项，使逻辑函数不可能在某种条件下变成“X+/X”或“X·/X”的形式，从而消除可能产生的险象。

存在险象的逻辑函数：F=/A·C+B·/C·D+A·/B·/C

冗余项：/A·B·D、A·/C·D

增加冗余项后的逻辑函数： F=/A·C+B·/C·D+A·/B·/C+/A·B·D+A·/C·D

  - 2、增加惯性延时环节：通常采用RC电路作为惯性延时环节，RC电路的时间常数（τ=RC）要求大于毛刺（尖脉冲）的宽度。当然τ也不能太大，否则会使正常的输出信号产生不允许的畸变。

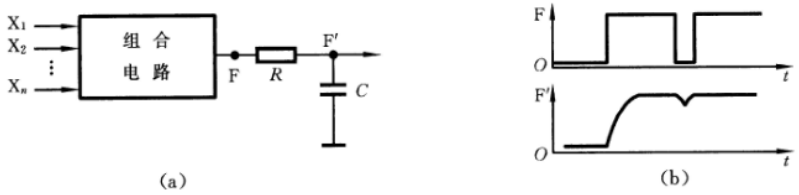


图 4.25 惯性延时环节

  - 3、选通法：利用选通脉冲的作用，从时间上加以控制，以避开险象脉冲。选通法不需要增加任何器件。

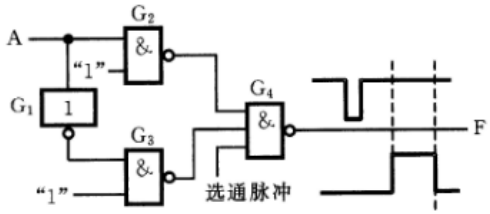


图 4.26 用选通法避开险象原理图

- 例4.1　分析图4.2(a)所示组合逻辑电路。

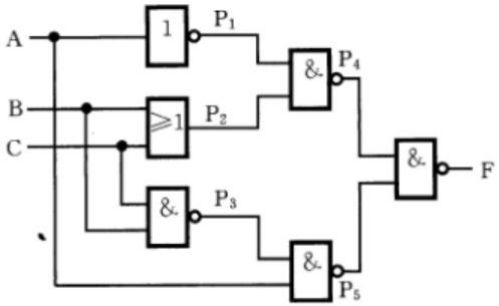


图4.2(a)　逻辑电路

- 解：
  - 1、根据逻辑电路图写出输出函数表达式：
    - P_1=/A
    - P_2=B+C
    - P_3=/(B·C)
    - P_4=/(P_1·P_2)=/(/A·(B+C))
    - P_5=/(A·P_3)=/(A·/(B·C))
    - F=/(P_4·P_5) = /(/(/A·(B+C)) · /(A·/(B·C)))
  - 2、化简输出函数表达式（采用代数化简法）：
    - F = /(/(/A·(B+C)) · /(A·/(B·C))) = (A⊕B) + (A⊕C)
  - 3、列出输出函数真值表（见表4.1）。

表 4.1　真值表

A	B	C	F
0	0	0	0
0	0	1	1
0	1	0	1
0	1	1	1
1	0	0	1
1	0	1	1
1	1	0	1
1	1	1	0

  - 4、功能评述
    - 根据真值表，可知该电路仅当A、B、C同为0，或者同为1时，输出为0；其他情况，输出均为1。该电路通常称为“不一致电路”。
    - 此外，根据化简后的函数表达式，可以用图4.2(b)实现该电路，并且图4.2(b)要比图4.2(a)简单。

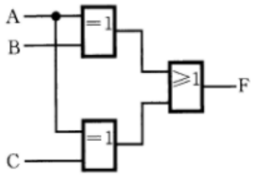


图4.2(b)　逻辑电路

- 例4.2　分析图4.3(a)所示组合逻辑电路。
- 解：
  - 1、根据逻辑电路图写出输出函数表达式：
    - S=/(/((A·B)·A) · /((A·B)·A))
    - C=//(A·B)
  - 2、化简输出函数表达式（采用代数化简法）：
    - S=/(/((A·B)·A) · /((A·B)·A)) = A·/B + /A·B = A⊕B
    - C=//(A·B) = A·B
  - 3、列出输出函数真值表（见表4.2）。
  - 4、功能评述
    - 根据真值表，可知该电路为1位半加器HA（Half Adder），其中A、B为输入，S为“和”，C为“进位”。半加器的逻辑符号如图4.3(b)所示。

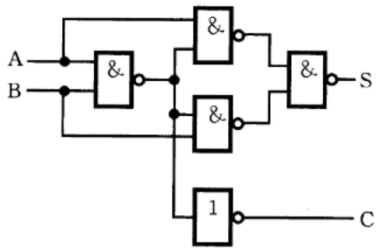


图4.3(a)　逻辑电路

表 4.2　真值表

A	B	S	C
0	0	0	0
0	1	1	0
1	0	1	0
1	1	0	1

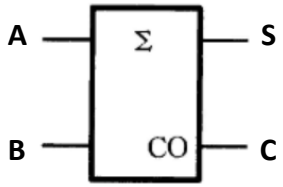


图4.3(b)　半加器的逻辑符号

- 例4.3　分析图4.4所示组合逻辑电路。已知A、B、C、D为8421码，说明该电路的功能。
- 解：
  - 1、根据逻辑电路图写出输出函数表达式：
    - W=A⊕B·(C+D)
    - X=B⊕(C+D)
    - Y=C⊕/D
    - Z=/D
  - 2、化简输出函数表达式：
    - 因为W、X、Y、Z已经是最简逻辑式，因此不需要化简。
  - 3、列出输出函数真值表（见表4.3）：
    - 因为A、B、C、D为8421码，因此输入只允许为0000～1001。
  - 4、功能评述
    - 根据真值表，可知该电路的输出为1位十进制数的余3码（见第1章的表1.3），即该电路是将8421码转换为余3码的代码转换电路。

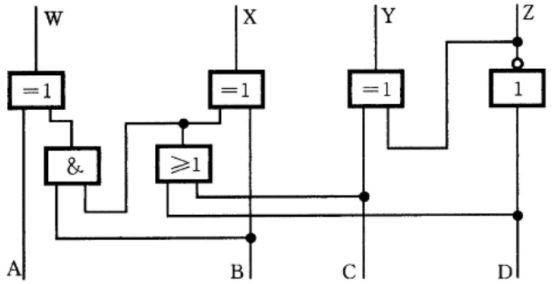


图 4.4　逻辑电路

表 4.3　真值表

ABCD	WXYZ	ABCD	WXYZ
0000	0011	0101	1000
0001	0100	0110	1001
0010	0101	0111	1010
0011	0110	1000	1011
0100	0111	1001	1100

表 1.3　常用的 3 种 BCD 码

十进制字符	8421 码	2421 码	余 3 码
0	0000	0000	0011
1	0001	0001	0100
2	0010	0010	0101
3	0011	0011	0110
4	0100	0100	0111
5	0101	1011	1000
6	0110	1100	1001
7	0111	1101	1010
8	1000	1110	1011
9	1001	1111	1100

- 例4.4　设计一个3变量“多数表决电路”。

- 解：
  - 1、建立给定问题的逻辑描述（采用真值表法）
    - 输入为A、B、C，输出为F。
    - 多数表决电路：当A、B、C中有2个或3个1时，F的值为1；其他情况，F的值为0。
    - 因此，$F(A,B,C)=\sum m(3,5,6,7)$。

序号	ABC	几个1
m_0	000	0
m_1	001	1
m_2	010	1
m_3	011	2
m_4	100	1
m_5	101	2
m_6	110	2
m_7	111	3

  - 2、求出逻辑函数的最简表达式
    - 函数F的真值表见表4.4，卡诺图见图4.5(a)。
    - 采用卡诺图化简法，得到： $F(A,B,C)=A\cdot B+A\cdot C+B\cdot C$。

表 4.4　真值表

A	B	C	F
0	0	0	0
0	0	1	0
0	1	0	0
0	1	1	1
1	0	0	0
1	0	1	1
1	1	0	1
1	1	1	1

  - 3、选择逻辑门类型并进行逻辑函数变换
    - 假设采用“与非门”来实现该电路，需要将F转换为“与非-与非”的表达形式：
    - $F(A,B,C)=/(/(A\cdot B)\cdot /(A\cdot C)\cdot /(B\cdot C))$。

  - 4、画出逻辑电路图
    - 见图4.5(b)。

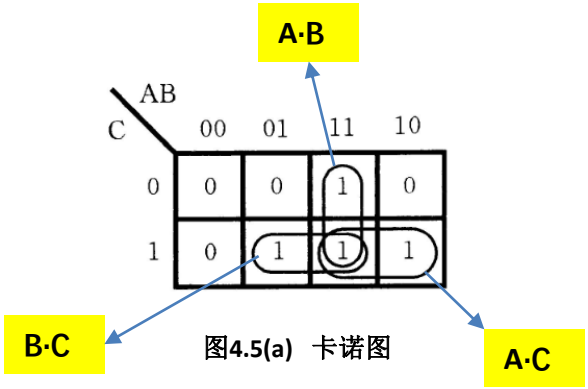


图4.5(a)　卡诺图

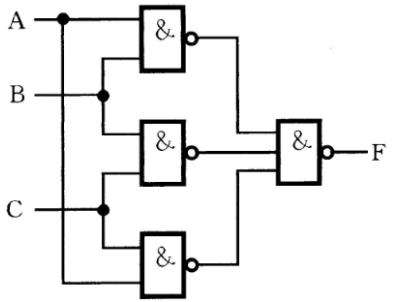


图4.5(b)　逻辑电路

- 例4.5　设计一个比较两个3位二进制数是否相等的数值比较器。

- 解：
  - 1、建立给定问题的逻辑描述（采用分析法）
    - 输入为$A=a_3a_2a_1$、$B=b_3b_2b_1$，共6位；输出为F，1位二进制数。
    - A与B相等，必须是：$a_3=b_3$，$a_2=b_2$，$a_1=b_1$。
    - 因此，$F = (a_3 \cdot b_3 + /a_3 \cdot /b_3) \cdot (a_2 \cdot b_2 + /a_2 \cdot /b_2) \cdot (a_1 \cdot b_1 + /a_1 \cdot /b_1)$。
  - 2、求出逻辑函数的最简表达式
    - 函数F已经是最简表达式，不需要再简化。
  - 3、选择逻辑门类型并进行逻辑函数变换
    - 假设采用“异或门”和“或非门”来实现该电路，将F转换为如下的形式：
    - $F = /(a_3 \oplus b_3) \cdot /(a_2 \oplus b_2) \cdot /(a_1 \oplus b_1) = /((a_3 \oplus b_3) + (a_2 \oplus b_2) + (a_1 \oplus b_1))$。
  - 4、画出逻辑电路图
    - 见图4.6。

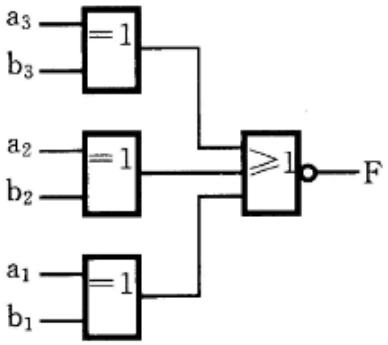


图 4.6　逻辑电路

- 例4.6　设计一个乘法器，用于产生两个2位二进制数相乘的积。

- 解：
  - 方法一：采用真值表法
    - （1）建立给定问题的逻辑描述
      - 2位二进制数的最大值为3，相乘后，最大值为9。
      - 因此，输入为$A=A_1A_0$、$B=B_1B_0$，共4位；输出为$M=M_3M_2M_1M_0$，共4位。
      - 列出输出与输入的真值表，见表4.5。
      - 根据真值表，可以得到：
        - » $M_3=\sum m(15)$
        - » $M_2=\sum m(10,11,14)$
        - » $M_1=\sum m(6,7,9,11,13,14)$
        - » $M_0=\sum m(5,7,13,15)$

表 4.5　真值表

A_1	A_0	B_1	B_0	M_3	M_2	M_1	M_0	A_1	A_0	B_1	B_0	M_3	M_2	M_1	M_0
0	0	0	0	0	0	0	0	1	0	0	0	0	0	0	0
0	0	0	1	0	0	0	0	1	0	0	1	0	0	1	0
0	0	1	0	0	0	0	0	1	0	1	0	0	1	0	0
0	0	1	1	0	0	0	0	1	0	1	1	0	1	1	0
0	1	0	0	0	0	0	0	1	1	0	0	0	0	0	0
0	1	0	1	0	0	0	1	1	1	0	1	0	0	1	1
0	1	1	0	0	0	1	0	1	1	1	0	0	1	1	0
0	1	1	1	0	0	1	1	1	1	1	1	1	0	0	1

    - （2）求出逻辑函数的最简表达式
      - 采用卡诺图化简法，得到输出函数的最简“与-或”表达式：
        - » $M_3=A_1\cdot A_0\cdot B_1\cdot B_0$
        - » $M_2=A_1\cdot\overline{A_0}\cdot B_1+A_1\cdot B_1\cdot\overline{B_0}$
        - » $M_1=A_1\cdot\overline{B_1}\cdot B_0+A_1\cdot\overline{A_0}\cdot B_0+\overline{A_1}\cdot A_0\cdot B_1+A_0\cdot B_1\cdot\overline{B_0}$
        - » $M_0=A_0\cdot B_0$
    - （3）选择逻辑门类型并画出逻辑电路图
      - 假设采用“与门”和“或门”来实现该电路，具体见图4.7。

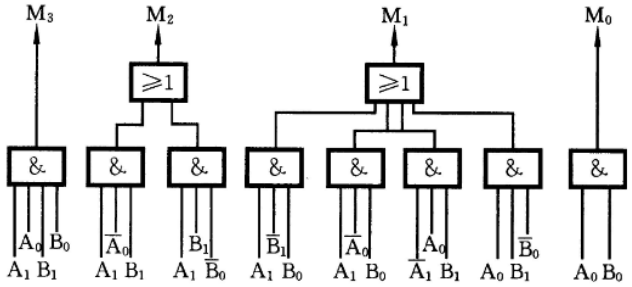


图 4.7　方法 I 的逻辑电路

– 方法二：采用分析法

- （1）建立给定问题的逻辑描述
  - 2位二进制数乘法运算方法如右所示，其中：
    - » $C_1 = A_1 \cdot B_0 \cdot A_0 \cdot B_1$
    - » $C_2 = C_1 \cdot A_1 \cdot B_1 = A_1 \cdot B_0 \cdot A_0 \cdot B_1 \cdot A_1 \cdot B_1 = A_1 \cdot B_0 \cdot A_0 \cdot B_1$
    - » $M_0 = A_0 \cdot B_0$
    - » $M_1 = A_1 \cdot B_0 \oplus A_0 \cdot B_1$
    - » $M_2 = C_1 \oplus A_1 \cdot B_1 = A_1 \cdot B_0 \cdot A_0 \cdot B_1 \oplus A_1 \cdot B_1$
    - » $M_3 = C_2 = A_1 \cdot B_0 \cdot A_0 \cdot B_1$

			A_1	A_0
×（乘）			B_1	B_0
	C_2	C_1	$A_1 \times B_0$	$A_0 \times B_0$
＋（加）		$A_1 \times B_1$	$A_0 \times B_1$	
	M_3	M_2	M_1	M_0

- （2）选择逻辑门类型并画出逻辑电路图
  - 假设采用“异或门”和“与门”来实现该电路，具体见图4.8。

– 比较图4.7和图4.8，图4.7使用了10个门电路，图4.8只使用了7个门电路，因此图4.8的电路更简单。

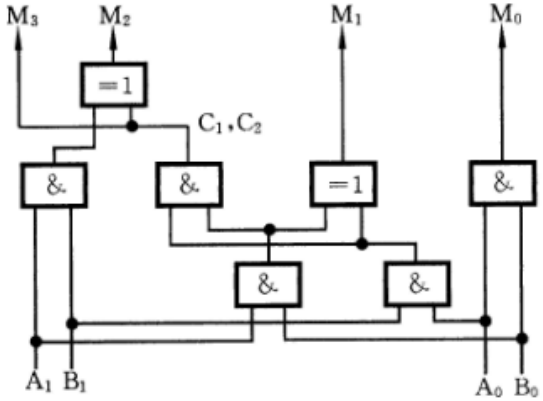


图 4.8　方法Ⅱ的逻辑电路

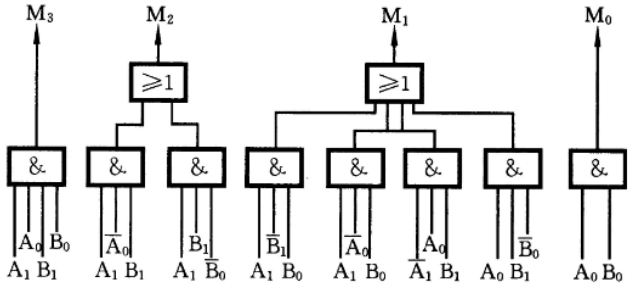


图 4.7　方法Ⅰ的逻辑电路

- 例4.7　设计一个组合逻辑电路，用于判别以余3码表示的1位十进制数是否为合数（0～9中，4、6、8、9为合数；与合数对应的是质数：2、3、5、7）。

- 解：
  - 输入为A、B、C、D，4位二进制数；输出为F，1位二进制数。输入与输出的关系见表4.6（真值表）：
    ① 当输入对应的余3码其对应的十进制数为4、6、8、9时，输出F=1。
    ② 当输入对应的余3码其对应的十进制数为0、1、2、3、5、7时，输出F=0。
    ③ 当输入为不允许出现的值时（0000、0001、0010、1101、1110、1111，即表1.3最右列中没有出现的6种情况），输出F=d（d表示可以是0，也可以是1）。
  - 因此有：F=∑m(7,9,11,12) + ∑d(0,1,2,13,14,15) 。函数F对应的卡诺图如图4.9(a)所示。如果只对卡诺图中的“1小方块”画卡诺圈（图4.9(a)），则得到F的简化函数：F = A·/B·D + A·B·/C·/D + /A·B·C·D。
  - 如果将卡诺图中的“d小方块”也加入卡诺圈（图4.9(b)），则得到F的简化函数：F = A·B + A·D + B·C·D。显然，这个式子更简单。
  - 采用“与非门”实现F，需要先将F转换为“与非-与非”表达式： F = //(A·B + A·D + B·C·D) = /(/(A·B) · /(A·D) · /(B·C·D))。对应的逻辑电路如图4.10所示。

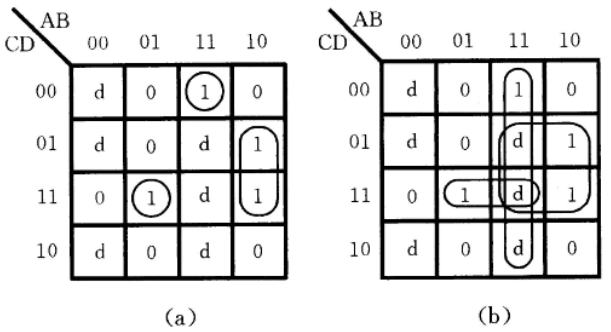


图 4.9　卡诺图

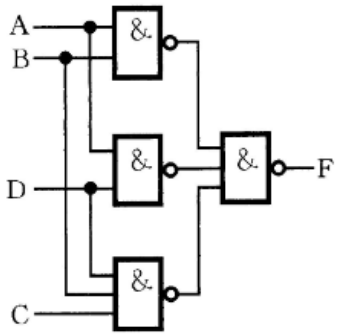


图 4.10　逻辑电路

表 4.6　真值表

A	B	C	D	F	A	B	C	D	F
0	0	0	0	d	1	0	0	0	0
0	0	0	1	d	1	0	0	1	1
0	0	1	0	d	1	0	1	0	0
0	0	1	1	0	1	0	1	1	1
0	1	0	0	0	1	1	0	0	1
0	1	0	1	0	1	1	0	1	d
0	1	1	0	0	1	1	1	0	d
0	1	1	1	1	1	1	1	1	d

表 1.3　常用的 3 种 BCD 码

十进制字符	8421 码	2421 码	余 3 码
0	0000	0000	0011
1	0001	0001	0100
2	0010	0010	0101
3	0011	0011	0110
4	0100	0100	0111
5	0101	1011	1000
6	0110	1100	1001
7	0111	1101	1010
8	1000	1110	1011
9	1001	1111	1100

- 例4.8　假定某组合逻辑电路结构框图如图4.11所示，试用最少的“与非门”实现该电路的功能（实现组合逻辑电路）。

- 解：
  - 首先画出F_1和F_2的卡诺图，如图4.12(a)和图4.12(b)所示。
  - 如果分别对F_1和F_2进行简化，并采用“与非门”实现，则有（图4.12(c)）：
    - F_1= /A·C + B·C = //(/A·C + B·C) = /(/(/A·C) · /(B·C))
    - F_2= A·B + B·/C = //(A·B + B·/C) = /(/(A·B) · /(B·/C))

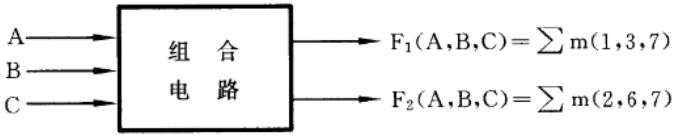


图 4.11　组合逻辑电路结构框图

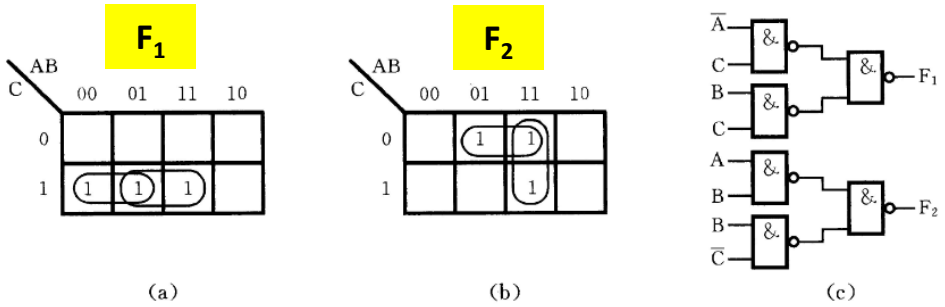
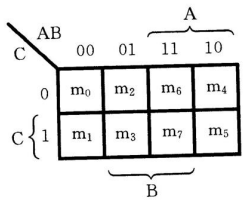


图 4.12　卡诺图及逻辑电路之一



– 考虑到两个输出函数应充分“共享”，则可以采用图4.13(a)和图4.13(b)的卡诺图化简方式，并采用“与非门”实现，则有（图4.13(c)）：

- F_1= /A·C + A·B·C = //(/A·C + A·B·C) = /(/(/A·C) · /(A·B·C))

- F_2= A·B·C + B·/C = //(A·B·C + B·/C) = /(/(A·B·C) · /(B·/C))

– 可见，图4.13(c)的电路（5个与非门）要比图4.12(c)的电路（6个与非门）简单一些。

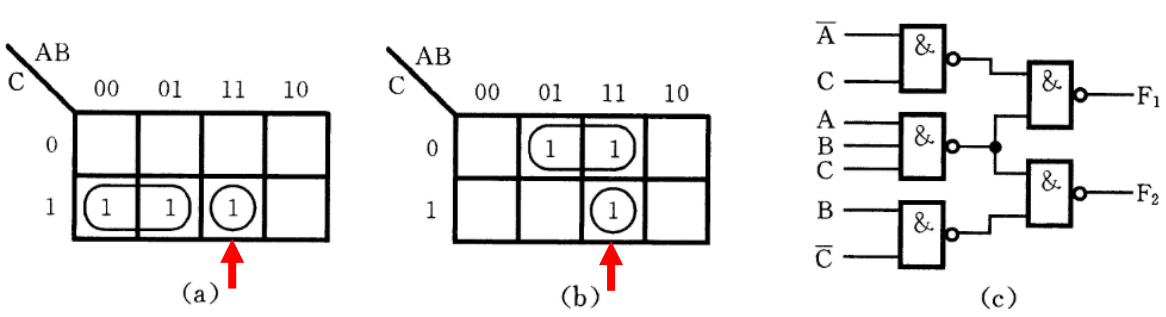


图 4.13　卡诺图及逻辑电路之二

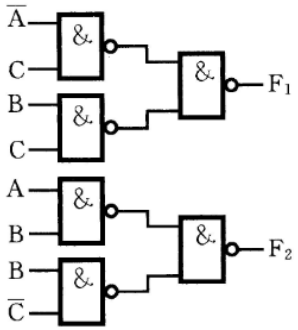


图4.12(c)　逻辑电路

- **例4.9　设计一个全加器。**

- **解：**
  - 全加器（FA：Full Adder）的输入有3个：A_i、B_i、C_{i-1}；输出有2个：S_i、C_i。其中，C_{i-1}为来自低位的进位，C_i为向高位的进位。全加器的真值表见表4.7。
  - 根据真值表，可以得到全加器的最小项表达式，以及卡诺图（图4.14）：
    - $S_i = \sum m(1,2,4,7)$
    - $C_i = \sum m(3,5,6,7)$
  - 根据卡诺图，可以得到简化后的逻辑表达式：
    - $S_i = /A_i \cdot /B_i \cdot C_{i-1} + /A_i \cdot B_i \cdot /C_{i-1} + A_i \cdot /B_i \cdot /C_{i-1} + A_i \cdot B_i \cdot C_{i-1}$
    - $C_i = A_i \cdot B_i + A_i \cdot C_{i-1} + B_i \cdot C_{i-1}$
  - 如果采用“异或门”和“与非门”实现，需要将逻辑表达式进行变换，得到图4.15(a)所示的逻辑电路：
    - $S_i = /A_i \cdot /B_i \cdot C_{i-1} + /A_i \cdot B_i \cdot /C_{i-1} + A_i \cdot /B_i \cdot /C_{i-1} + A_i \cdot B_i \cdot C_{i-1} = A_i \oplus B_i \oplus C_{i-1}$
    - $C_i = A_i \cdot B_i + A_i \cdot C_{i-1} + B_i \cdot C_{i-1} = /(/(A_i \cdot B_i) \cdot /(A_i \cdot C_{i-1}) \cdot /(B_i \cdot C_{i-1}))$

表 4.7　全加器真值表

A_i	B_i	C_{i-1}	S_i	C_i
0	0	0	0	0
0	0	1	1	0
0	1	0	1	0
0	1	1	0	1
1	0	0	1	0
1	0	1	0	1
1	1	0	0	1
1	1	1	1	1

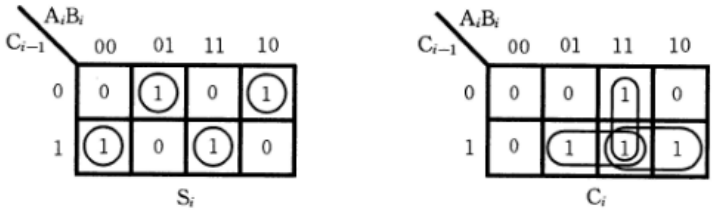


图 4.14　函数 S_i 和 C_i 的卡诺图

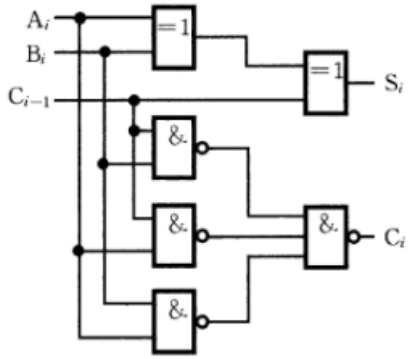


图4.15(a)　逻辑电路

– 图4.15(a)虽然对单个函数是最简的，但是在整体上不是最简的。

– 可以对C_i进行如下的变换，得到图4.15(b)所示的逻辑电路：
  - $C_i = /A_i \cdot /B_i \cdot C_{i-1} + /A_i \cdot B_i \cdot /C_{i-1} + A_i \cdot /B_i \cdot /C_{i-1} + A_i \cdot B_i \cdot C_{i-1} = /(/((A_i \oplus B_i) \cdot C_{i-1}) \cdot /(A_i \cdot B_i))$

– 显然，图4.15(b)（3个与非门+2个异或门）比图4.15(a)（4个与非门+2个异或门）要简单一些。

– 并且实现图4.15(a)需要三种芯片（2输入异或门、2输入与非门、3输入与非门），实现图4.15(b)只需要两种芯片（ 2输入异或门、2输入与非门）。

– 图4.16给出了采用7486（4个2输入异或门）和7400（4个2输入与非门）实现图4.15(b)全加器的引脚连接图。

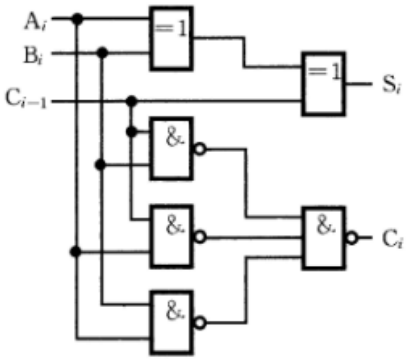


图4.15(a) 逻辑电路

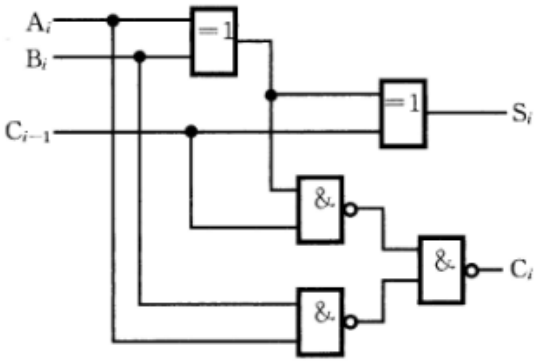


图4.15(b) 逻辑电路

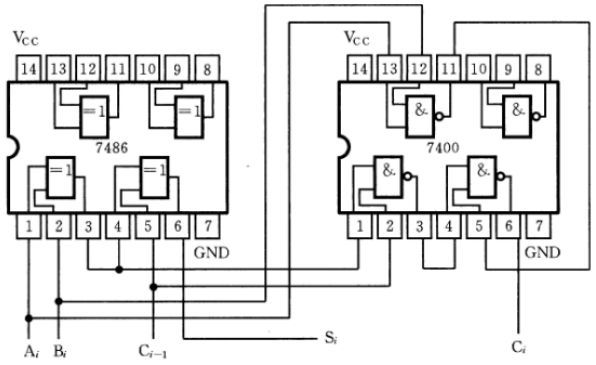


图 4.16 全加器的芯片引脚连接图

- 例4.10　输入变量中无反变量时，用“与非门”实现逻辑函数：F= /A·B + B·/C + A·/B·C + A·C·/D（实现组合逻辑电路）。

- 解：
  - 函数F已经是最简表达式，当采用“与非门”和“非门”实现时，直接按照函数F的表达式画出的电路如图4.17(a)所示。
  - 如果将函数F进行如下的变换，可以不需要“非门”，电路见图4.17(b)：
    - F= /A·B + B·/C + A·/B·C + A·C·/D = B·/(A·C) + A·C·/(B·D) = /(/(B·/(A·C)) + /(A·C·/(B·D)))
  - 可见，图4.17(b)的电路（5个与非门）要比图4.17(a)的电路（5个与非门+4个非门）简单很多。

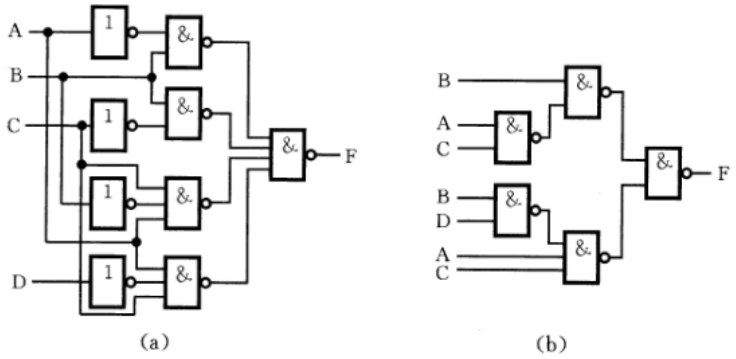


图 4.17　逻辑电路

- 例4.11　在输入无反变量提供时，用最少的逻辑门实现逻辑函数：F= /A·B + B·/C + A·/B·C（实现组合逻辑电路）。

- 解：
  - 函数F已经是最简表达式，当采用“与非门”和“非门”实现时，直接按照函数F的表达式画出的电路如图4.18(a)所示。

  - 如果将函数F进行如下的变换，可以不需要“非门”，并且使电路简化，见图4.18(b)：
    - F= /A·B + B·/C + A·/B·C = (A·C) ⊕ B

  - 可见，图4.18(b)的电路（1个与门+1个异或门）要比图4.18(a)的电路（4个与非门+3个非门）简单很多。

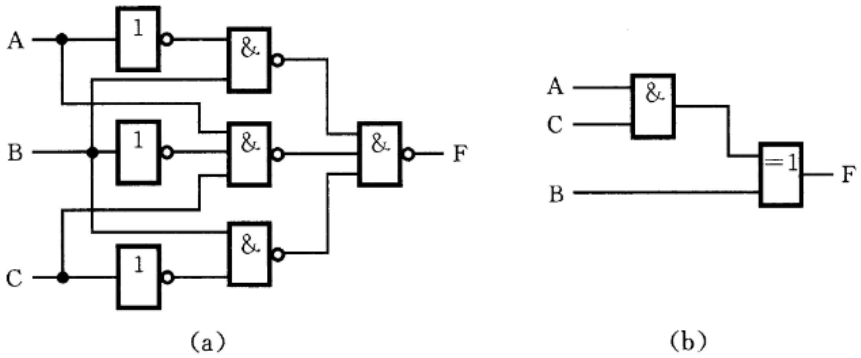


图 4.18　逻辑电路

- 例4.12　设计一个组合逻辑电路，用来判断献血者与受血者血型是否相容。血型相容规则如表4.8所示。

- 解：
  - 输入变量：献血者的血型（4种情况，用2位二进制数表示，WX）、受血者的血型（4种情况，用2位二进制数表示，YZ），血型编码如表4.9所示。输出变量：血型是否相容（1位二进制数，F）。

  - 根据表4.8和表4.9，可以得到F的逻辑公式：
    - F = /W·/X·(/Y·/Z+Y·/Z) + /W·X·(/Y·Z+Y·/Z) + W·/X·Y·/Z + W·X·(/Y·/Z+/Y·Z+Y·/Z+Y·Z)
      = /W·/X·/Z + Y·/Z + W·X + X·/Y·Z

  表 4.8　血型相容规则表

献血 \ 受血	A	B	AB	O
A	√		√	
B		√	√	
AB			√	
O	√	√	√	√

  - 如果直接用“与非门”和“非门”实现上述逻辑公式，见图4.19(a)。

  - 如果改变血型的编码，见表4.10，则可以得到下面的逻辑公式：
    - F = /W·/X·(/Y·/Z+/Y·Z+Y·/Z+Y·Z) + /W·X·(/Y·Z+Y·Z) + W·/X·(Y·/Z+ Y·Z) + W·X·Y·Z = (/W+Y)·(/X + Z)

  - 如果用“或非门”和“非门”实现上述逻辑公式，见图4.19(b)。

  - 可以，图4.19(b)的电路（3个或非门+2个非门）要比图4.19(a)的电路（5个与非门+4个非门）简单很多。

表 4.9　血型编码(1)

血　型	献 W　X	受 Y　Z
A	0　0	0　0
B	0　1	0　1
AB	1　0	1　0
O	1　1	1　1

表 4.10　血型编码(2)

血　型	献 W　X	受 Y　Z
O	0　0	0　0
A	0　1	0　1
B	1　0	1　0
AB	1　1	1　1

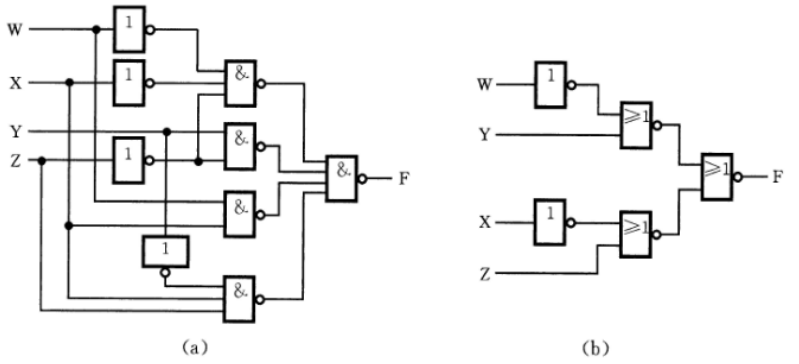


图 4.19　逻辑电路

- 例4.13　已知描述某组合逻辑电路的逻辑函数为F=/A·/C+/A·B+A·C，判断该逻辑电路是否可能产生险象。

- 解：
  - 因为逻辑函数中有A和/A、C和/C，因此，变量A、C均具备产生险象的条件。

  - 先考察A，可知，A的变化可能使电路产生险象：
    - BC=00　F=/A
    - BC=01　F=A
    - BC=10　F=/A
    - BC=11　F=/A+A

  - 再考察C，可知，C的变化不会导致电路产生险象：
    - AB=00　F=/C
    - AB=01　F=/C+1
    - AB=10　F=C
    - AB=11　F=C

- 例4.14　试判断逻辑函数F=(A+B)·(/A+C)·(/B+C)描述的电路是否可能产生险象。

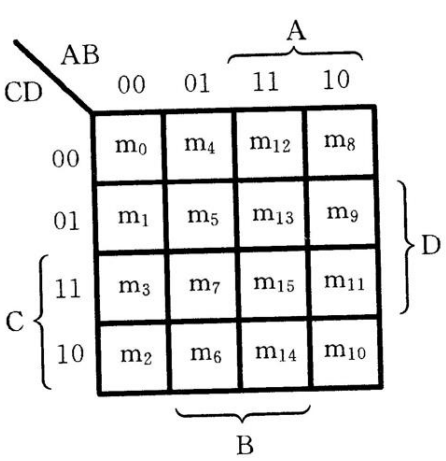
- 解：
  - 因为逻辑函数中有A和/A、B和/B，因此，变量A、B均具备产生险象的条件。

  - 先考察A，可知，A的变化可能使电路产生险象：
    - BC=00　F=A·/A
    - BC=01　F=A
    - BC=10　F=0
    - BC=11　F=1

  - 再考察B，可知，B的变化也可能使电路产生险象：
    - AC=00　F=B·/B
    - AC=01　F=B
    - AC=10　F=0
    - AC=11　F=1

- 例4.15　已知描述某组合逻辑电路的逻辑函数为F=/A·D+/A·C+A·B·/C，判断该逻辑电路是否可能产生险象。

- 解：
  - 首先画出函数F的卡诺图，并画出卡诺圈，如图4.22所示。
  - 图中红色的卡诺圈和蓝色的卡诺圈“相切”，因此，该电路可能产生险象。验证：当B=D=1、C=0时，F=A+/A。
  - 红色的卡诺圈和绿色的卡诺圈“相交”，蓝色的卡诺圈和绿色的卡诺圈既不“相交”也不“相切”，这两种情况都不会产生险象。



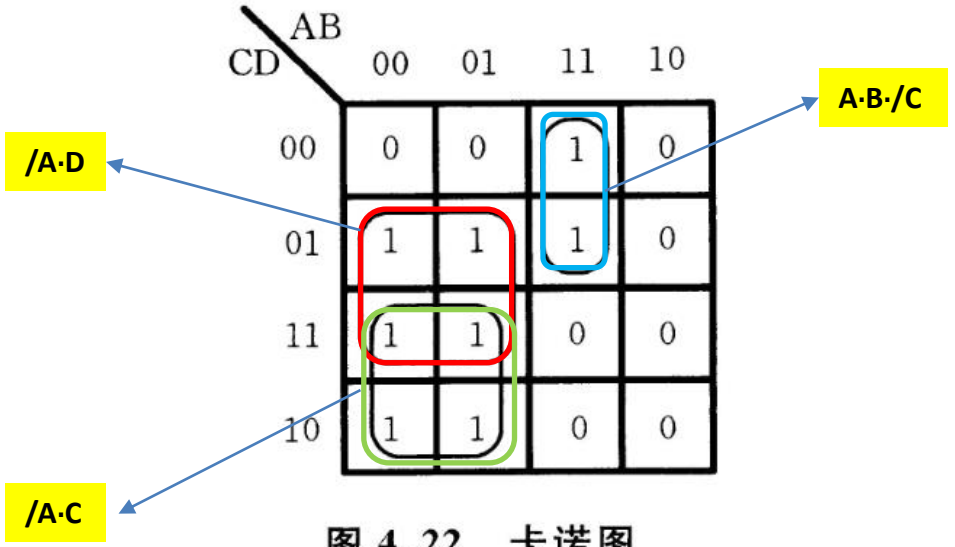


图 4.22　卡诺图

- 例4.16　用增加冗余项的方法消除图4.20(a)所示电路中可能产生的险象。

- 解：
  - 图4.20(a)对应的逻辑公式为： F = /(/(A·B) · /(/A·C)) = A·B+/A·C。当B=C=1时， F = A+/A = 1，产生了“0”型险象。

  - 因为，B=C=1时， F = A+/A = 1；可知B·C为逻辑公式F的冗余项。即： F = A·B+/A·C+B·C。增加冗余项后的电路如图4.23所示，该电路不再产生险象。

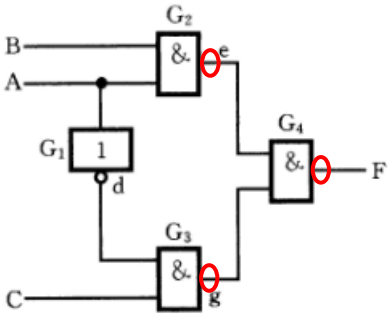


图4.20(a) 具有险象的逻辑电路

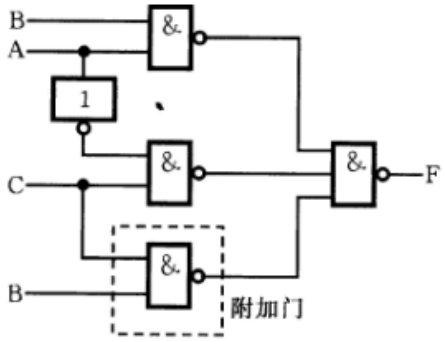


图 4.23　增加冗余项后的逻辑电路

- 例4.17　已知描述某组合逻辑电路的函数表达式为F=/A·C+B·/C·D+A·/B·/C，用增加冗余项的方法消除该电路中可能产生的险象。

- 解：
  - 首先画出函数的卡诺图和卡诺圈，如图4.24所示。可见，红色卡诺圈与蓝色卡诺圈“相切”，蓝色卡诺圈与紫色卡诺圈“相切”。因此，该电路存在险象。

  - 可以采用增加两个（绿色、黄色）冗余的卡诺圈（冗余项）来消除险象（消除卡诺圈“相切”的现象），即增加/A·B·D和A·/C·D冗余项。

  - 增加冗余项后的函数表达式为： F=/A·C+B·/C·D+A·/B·/C+/A·B·D+A·/C·D。

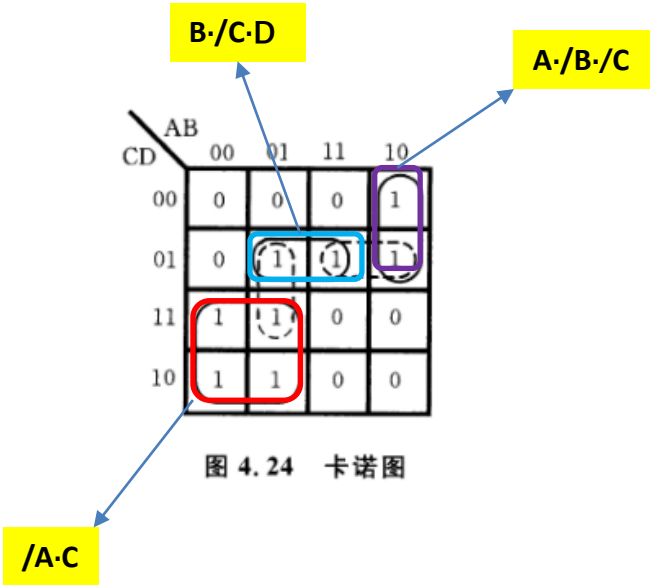


图 4.24　卡诺图

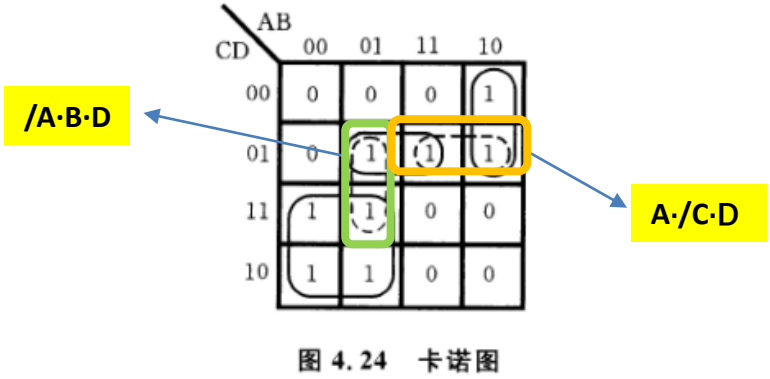


图 4.24　卡诺图

第5章　同步时序逻辑电路

- 5.1　时序逻辑电路概述
- 5.2　同步时序逻辑电路分析
- 5.3　同步时序逻辑电路设计

第5章主要内容

- 时序逻辑电路在任何时刻产生的稳定输出，不仅与该时刻电路的输入有关，而且与过去的输入值有关。组合逻辑电路在任何时刻产生的稳定输出，仅与该时刻电路的输入有关，而与过去的输入值无关。

- 时序逻辑电路的一般结构（包括：组合电路和存储电路）。

x：输入
Z：输出
y：现态；y^{n+1}：次态
Y：激励
CP：时钟脉冲信号

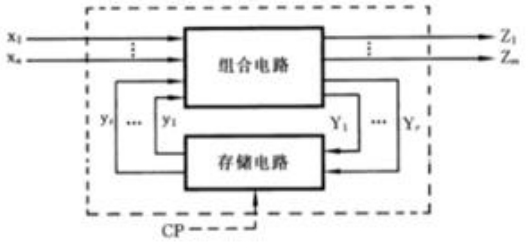


图 5.1 时序逻辑电路的一般结构

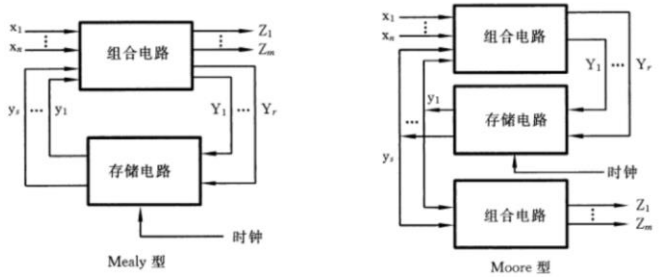


图 5.4 同步时序逻辑电路中两种模型的结构框图

- 时序逻辑电路的特征：
  ① 电路由组合电路和存储电路组成，具有对过去输入进行记忆的功能；
  ② 电路中包含反馈回路，通过反馈使电路功能与“时序”相关；
  ③ 电路的输出由电路当时的输入和状态（对过去输入记忆的结果）共同决定。

- 时序逻辑电路的分类：
  ① 同步时序逻辑电路（第5章）和异步时序逻辑电路（第6章）。
  ② Mealy型时序逻辑电路（电路的输出是电路输入和电路状态的函数）和Moore型时序逻辑电路（电路的输出仅仅是电路状态的函数）。
  ③ 脉冲型（输入信号是脉冲信号）和电平型（输入信号是电平信号）。

- 时序逻辑电路的描述方式：逻辑函数表达式（激励函数、输出函数、次态函数）、状态表、状态图、时间图（时序图）。组合逻辑电路的描述方式：逻辑函数表达式（输出函数）、真值表、卡诺图。

表 5.1　Mealy 型电路状态表格式

现　态	次态/输出		
		输入 x	
y		y^{n+1}/Z	

表 5.2　Moore 型电路状态表格式

现　态	次　态		输　出
		输入 x	
y		y^{n+1}	Z

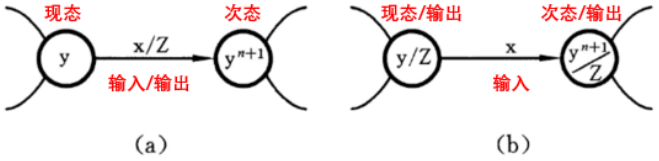


图 5.6　Mealy 型和 Moore 型电路的状态图形式

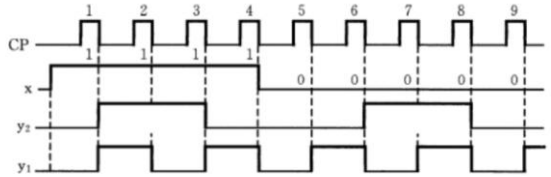


图 5.10　时间图

- 同步时序逻辑电路分析方法：表格法、代数法。

- 表格分析法的一般步骤：
  ① 根据给定的同步时序逻辑电路，写出输出函数和激励函数表达式。
  ② 列出电路次态真值表。根据输入和现态在各种取值下的激励函数值，以及触发器的功能表，确定电路的相应次态，它反映了电路在不同时刻和不同输入下的状态转移关系。
  ③ 根据次态真值表和输出函数表达式，作出给定电路的状态表和状态图。
  ④ 拟定一典型输入序列，画出时间图，并用文字描述电路的逻辑功能。

- 代数分析法的一般步骤：
  ① 根据给定的同步时序逻辑电路，写出输出函数和激励函数表达式。
  ② 把激励函数表达式代入触发器的次态方程，导出电路的次态方程组。
  ③ 根据次态方程组和输出函数表达式，作出给定电路的状态表和状态图。
  ④ 拟定一典型输入序列，画出时间图，并用文字描述电路的逻辑功能。

- 同步时序逻辑电路设计的一般步骤（6步）：
  - 1、形成原始状态图和原始状态表
    - 有一些状态图和状态表，对于所设立的每一个状态，在不同输入取值下，都有确定的次态和输出，这类状态图和状态表称为完全确定状态图和状态表，所描述的电路称为完全确定同步时序逻辑电路。
    - 还有一些状态图和状态表，存在某些状态，它们在某些输入取值下的次态或输出是随意的（不确定的），这类状态图和状态表称为不完全确定状态图和状态表，所描述的电路称为不完全确定同步时序逻辑电路。
  - 2、状态化简
    - 状态化简是从原始状态表中消去多余状态，得到既能正确描述给定的逻辑功能，又能使所包含的状态数目达到最少的状态表，这个状态表称为最简状态表或最小化状态表。
  - 3、状态编码
    - 状态编码是用二进制代码表示最简状态表中的状态，状态编码也称为状态分配，或者状态赋值。
  - 4、确定触发器数目和类型
    - 触发器数目由状态编码的位数（二进制代码的位数）决定，触发器类型如果设计要求中没有具体要求，可以根据设计要求灵活确定。
  - 5、确定激励函数和输出函数表达式
    - 根据状态编码后的二进制状态表，以及所选触发器的激励表，求出激励函数和输出函数的逻辑表达式，并进行化简。
  - 6、画出逻辑电路图
    - 首先，根据触发器数目和类型，实现存储电路部分（触发器）；然后，根据激励函数和输出函数的最简逻辑表达式，选择合适的逻辑门，实现组合逻辑部分。

- 完全确定同步时序逻辑电路设计：
  - 1、形成原始状态图和原始状态表
    - 包括：确定电路模型（Mealy型，Moore型）、设立初始状态、根据需要记忆的信息增加新的状态、确定各时刻电路的输出。
  - 2、状态化简
    - 状态化简的目的是：从[illegible]中消去多余的状态[illegible]个既能正确描述给定的逻辑功能，又能使所包含的状态数目达到最[illegible]表。
    - 通常是利用隐含表进行状态[illegible]图5.25）[illegible]小化状态表（最简状态表）。几个概念：等效状态、等效类、最大等效类

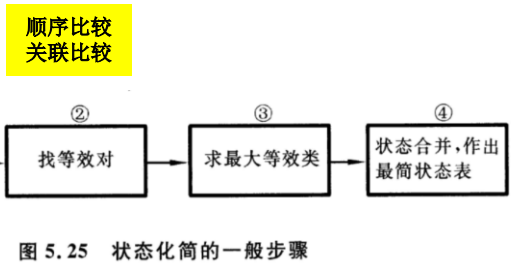


图 5.25　状态化简的一般步骤

  - 3、状态编码
    - 状态编码的目的是：在选择[illegible]，尽可[illegible]激励函数和输出函数的化简，并且将初始状态分配为“0”状态。
    - 通常是采用相邻编码法[illegible]则）进行状态编[illegible]进制状态表。
  - 4、确定激励函数和输出函数，并画出逻辑电路图
    - 方法一：根据二进制状态表和触发器激励表，列出激励函数和输出函数真值表；根据真值表，画出激励函数和输出函数卡诺图；根据卡诺图，化简得到激励函数和输出函数的逻辑表达式。
    - 方法二：根据二进制状态表，作出次态卡诺图和输出卡诺图；根据卡诺图，化简得到次态函数逻辑表达式和输出函数逻辑表达式；将次态函数逻辑表达式与触发器的次态方程相比较，得到激励函数的逻辑表达式。

- 不完全确定同步时序逻辑电路设计（不完全确定同步时序逻辑电路存在不确定的次态或输出，设计中应充分利用不确定次态和输出的随意性，使电路更简单）：
  - 1、形成原始状态图和原始状态表（也称不完全确定原始状态图和不完全确定原始状态表）
  - 2、不完全确定状态表[illegible]
    - 不完全确定状态表的化简[illegible]容状态基础[illegible]概念：相容状态、相容类、最大相容类。
    - 不完全确定状态表的化简过程[illegible].34）

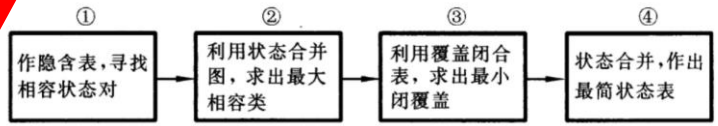


图 5.34　不完全确定状态表的化简过程

- 无效状态：
  - 例如采用3位二进制数进行编码[illegible]个状[illegible]使用了5个状态，存在3个无效状态（001、011、111）。
  - 带来的问题：
    ① 电路偶然进入无效状态[illegible]，能否在输入信号和时钟脉[illegible]，自动进入有效状态？如果能够进入有效状态，则称为具有自恢复功能；否则称为“挂起”。
    ② 电路进入无效状态时，会不会输出错误的结果。如果产生了错误的结果，这是不允许的。
  - 解决方法：
    ① 建立无效状态表，画出包含无效状态的状态图，检测无效状态不会产生“挂起”现象。
    ② 修改输出函数公式，确保在无效状态时，不出现错误输出现象。

表 3.23　钟控 J-K 触发器功能表

J	K	Q^{n+1}	功能说明
0	0	Q	不变
0	1	0	置 0
1	0	1	置 1
1	1	$\overline{Q}$	翻转

- 例5.1：用表格法分析如图5.8所示的同步时序逻辑电路。
- 解：
  - 由图5.8可知，该电路的存储电路是两个J-K触发器，组合电路是一个异或门。
  - 电路的输入为x，状态为y_2和y_1。该电路属于Moore型电路，电路的输出就是电路的状态。
  - （1）写出激励函数表达式。由于该电路的输出就是电路的状态，故只需写出激励函数表达式：
    - $J_2 = K_2 = x \oplus y_1$　　$J_1 = K_1 = 1$
  - （2）列出电路次态真值表（表5.3）。次态真值表的输入为电路的输入（x）和现态（y_2、y_1），输出为激励函数的输出（J_2、K_2、J_1、K_1）和次态（y_2^{n+1}、y_1^{n+1}）。前3列直接写出，中间4列根据激励函数得到，最后2列根据J-K触发器的功能表得到。

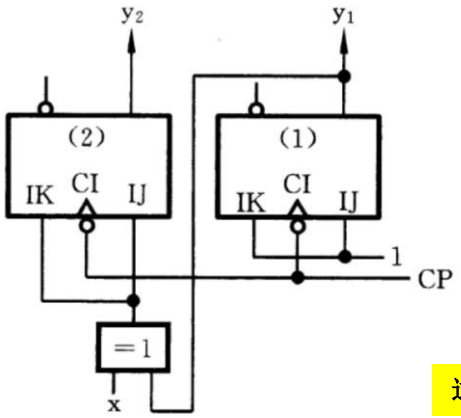


图 5.8　同步时序逻辑电路

首先列出这三列的值

由$J_2=x\oplus y_1$得到这一列的值

这一列的值与左边一列的值相同（$K_2=J_2$）

如果$J_2K_2=00$，y_2^{n+1}与y_2相同；如果$J_2K_2=11$，y_2^{n+1}与y_2相反（翻转）

这两列的值全部为1

因为$J_1K_1=11$，因此y_1^{n+1}与y_1相反

表 5.3　次态真值表

输入	现态		激励函数				次态	
x	y_2	y_1	J_2	K_2	J_1	K_1	y_2^{n+1}	y_1^{n+1}
0	0	0	0	0	1	1	0	1
0	0	1	1	1	1	1	1	0
0	1	0	0	0	1	1	1	1
0	1	1	1	1	1	1	0	0
1	0	0	1	1	1	1	1	1
1	0	1	0	0	1	1	0	0
1	1	0	1	1	1	1	0	1
1	1	1	0	0	1	1	1	0

– （3）作出状态表（表5.4）和状态图（图5.9）。状态表的输入为现态（y_2、y_1）和电路的输入（x），输出为次态（y_2^{n+1}、y_1^{n+1}）；状态图的起点为现态（4个圆圈，00、01、10、11），终点为次态（4个圆圈，00、01、10、11），连线上的信号为输入（x）。

表 5.4 状态表

现态 y_2	现态 y_1	次态 y_2^{n+1} x=0	次态 y_1^{n+1} x=0	次态 y_2^{n+1} x=1	次态 y_1^{n+1} x=1
0	0	0	1	1	1
0	1	1	0	0	0
1	0	1	1	0	1
1	1	0	0	1	0

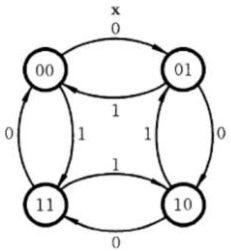


图 5.9 状态图

– （4）用时间图和文字描述电路的逻辑功能。由状态图（图5.9）可知，该电路是一个2位二进制数可逆计数器：
  - x=0：计数器加1，00 -> 01 -> 10 -> 11 -> 00 -> 01 ……
  - x=1：计数器减1，00 -> 11 -> 10 -> 01 -> 00 -> 11 ……

  - 假设电路的初始状态y_2y_1=00，输入x为电平信号，典型输入序列=111100000，则现态和次态如下：

CP	1	2	3	4	5	6	7	8	9
x	1	1	1	1	0	0	0	0	0
y_2	0	1	1	0	0	0	1	1	0
y_1	0	1	0	1	0	1	0	1	0
y_2^{n+1}	1	1	0	0	0	1	1	0	0
y_1^{n+1}	1	0	1	0	1	0	1	0	1

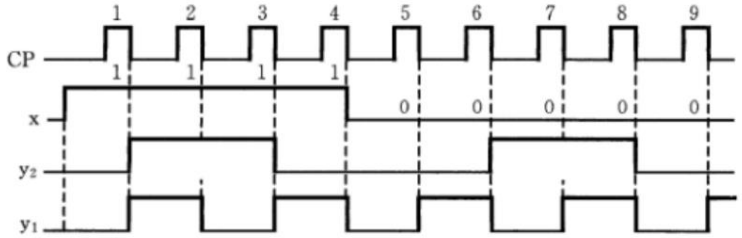


图 5.10 时间图

  - 这样就可以得到时间图（图5.10）。由于前一个时钟脉冲的次态即为后一个时钟脉冲的现态，因此图5.10中只给出了现态的波形。

- 例5.2：某同步时序逻辑电路的芯片连接图如图5.11所示，它由双D触发器芯片74LS74和或非门芯片74LS27构成，试（用表格法）分析该电路的功能（分析同步时序逻辑电路）。

- 解：
  - 在Logisim上画出图5.11的电路。

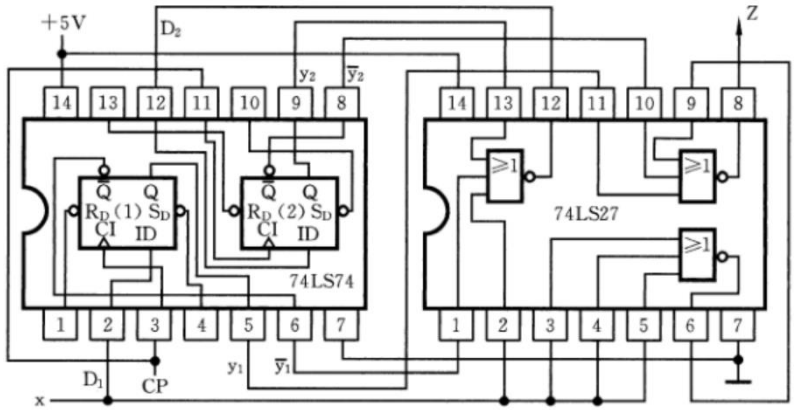
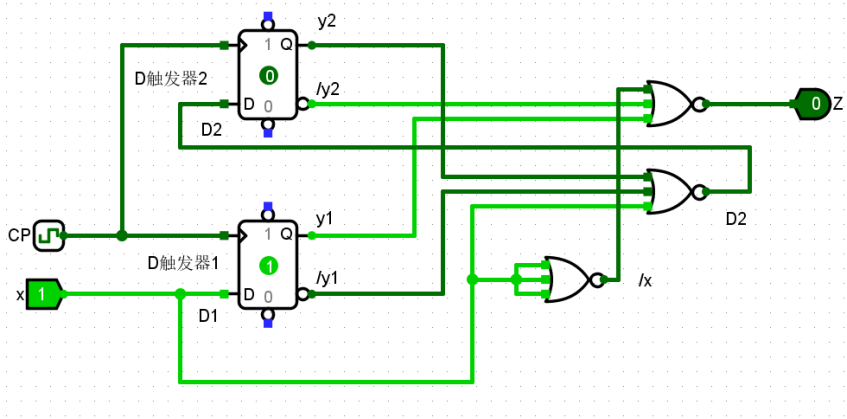


图 5.11　同步时序逻辑芯片连线图



  - （1）写出输出函数和激励函数表达式。
    - $Z = /(/x+/y_2+y_1) = x \cdot y_2 \cdot /y_1$　　$D_2 = /(x+y_2+/y_1) = /x \cdot /y_2 \cdot y_1$　　$D_1 = x$
    - 该电路属于Mealy型同步时序逻辑电路。

– （2）列出电路次态真值表（表5.5）。

- 前3列直接写出，中间2列根据激励函数得到，最后2列根据D触发器的功能表得到。

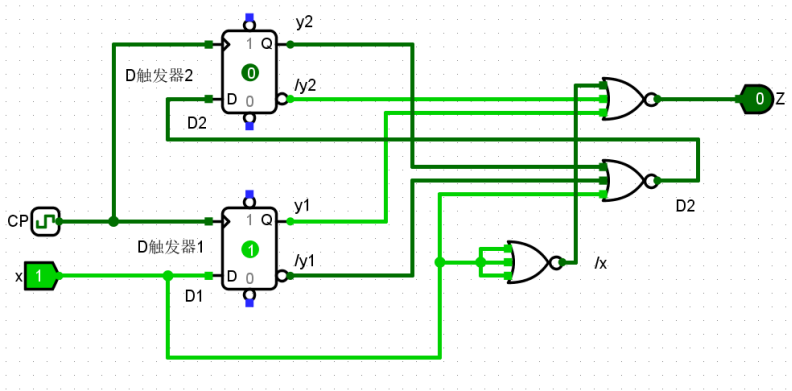
首先列出这三列的值

由激励函数 $D_2=/(x+y_2+/y_1)=/x\cdot/y_2\cdot y1$ 、$D_1=x$，得到这两列

因为是D触发器，因此，y_2^{n+1}与D_2相同，y_1^{n+1}与D_1相同

表 5.5　次态真值表

输入	现态		激励函数		次态	
x	y_2	y_1	D_2	D_1	y_2^{n+1}	y_1^{n+1}
0	0	0	0	0	0	0
0	0	1	1	0	1	0
0	1	0	0	0	0	0
0	1	1	0	0	0	0
1	0	0	0	1	0	1
1	0	1	0	1	0	1
1	1	0	0	1	0	1
1	1	1	0	1	0	1



– （3）作出状态表（表5.6）和状态图（图5.12）。

- 状态表的输入为现态（y_2、y_1）和电路的输入x，输出为次态（y_2^{n+1}、y_1^{n+1}）和电路的输出Z。状态图的起点为现态（4个圆圈，00、01、10、11），终点为次态（3个圆圈，00、01、10），连线上的信号为输入/输出（x/Z）。

表 5.6　状态表

现态		次态/输出($y_2^{n+1}y_1^{n+1}$/Z)	
y_2	y_1	x=0	x=1
0	0	00/0	01/0
0	1	10/0	01/0
1	1	00/0	01/0
1	0	00/0	01/1

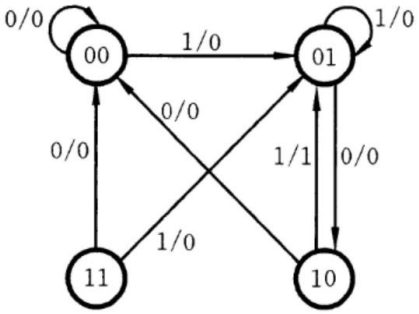


图 5.12　状态图

表 3.20　钟控 D 触发器的功能表

D	Q^{n+1}	功能说明
0	0	置 0
1	1	置 1

– （4）作出时间图，并说明电路的逻辑功能。

- 假设电路的初始状态y_2y_1=00，输入x为电平信号，典型输入序列=010110100，则现态、次态和输出如下：

CP	1	2	3	4	5	6	7	8	9
x	0	1	0	1	1	0	1	0	0
y_2	0	0	0	1	0	0	1	0	1
y_1	0	0	1	0	1	1	0	1	0
y_2^{n+1}	0	0	1	0	0	1	0	1	0
y_1^{n+1}	0	1	0	1	1	0	1	0	0
Z	0	0	0	1	0	0	1	0	0

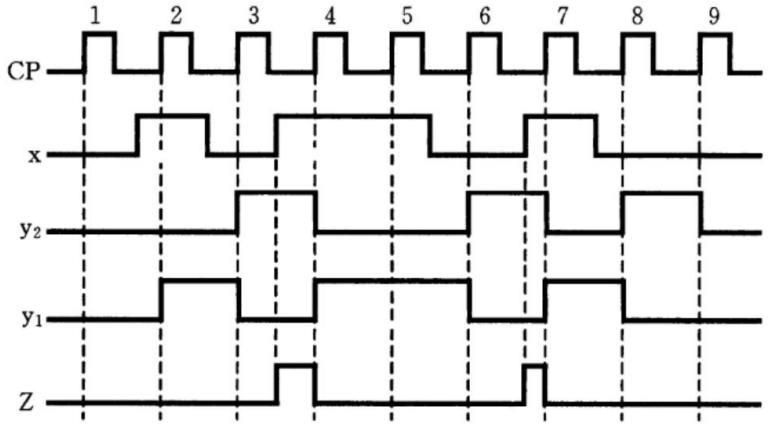


图 5.13　时间图

- 对应的时间图如图5.13所示。
- 由图5.13可知，一旦输入x出现信号“101”，输出Z=1；其他情况，输出Z=0。
- 因此，该电路是一个“101”序列检测器。

- 例5.3：试用代数分析法分析图5.14所示同步时序逻辑电路，设触发器初始状态为“0”，说明该电路的逻辑功能（分析同步时序逻辑电路）。

- 解：
  - 由图5.14可知，该电路的输出Z与输入（x_1、x_2）和状态（y）均有关系，因此该电路属于Mealy型电路。

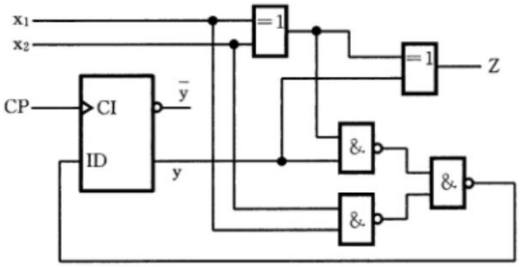


图 5.14　逻辑电路

  - （1）写出输出函数和激励函数表达式。
    - $Z = x_1 \oplus x_2 \oplus y$　　$D = /(\ /((x_1 \oplus x_2)\cdot y) \cdot /(x_1 \cdot x_2)\ ) = x_1 \cdot x_2 + x_1 \cdot y + x_2 \cdot y$
  - （2）根据D触发器的次态方程，写出电路的次态方程。
    - $y^{n+1} = D = x_1 \cdot x_2 + x_1 \cdot y + x_2 \cdot y$

D触发器的次态方程：$Q^{n+1}=D$

  - （3）根据次态方程和输出函数，作出状态表（表5.7）和状态图（图5.15）。

表 5.7　状态表

现态 y	次态/输出(y^{n+1}/Z)			
	$x_1x_2=00$	$x_1x_2=01$	$x_1x_2=11$	$x_1x_2=10$
0	0/0	0/1	1/0	0/1
1	0/1	1/0	1/1	1/0

状态表的输入为现态（y）和电路的输入（x_1、x_2），输出为次态（y^{n+1}）和电路的输出Z

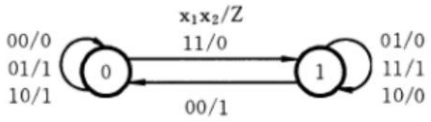


图 5.15　状态图

状态图的起点为现态（2个圆圈，0、1），终点为次态（2个圆圈，0、1），连线上的信号为输入/输出（x_1x_2/Z）

– （4）作出时间图，并说明电路的逻辑功能。

- 假设电路的初始状态y=0，输入（x_1、x_2）为电平信号，输入x_1的序列=00110110，输入x_2的序列=01011100，则现态和输出如下：

时钟节拍	1	2	3	4	5	6	7	8
输入 x_1	0	0	1	1	0	1	1	0
输入 x_2	0	1	0	1	1	1	0	0
状态 y	0	0	0	0	1	1	1	1
输出 Z	0	1	1	0	0	1	0	1

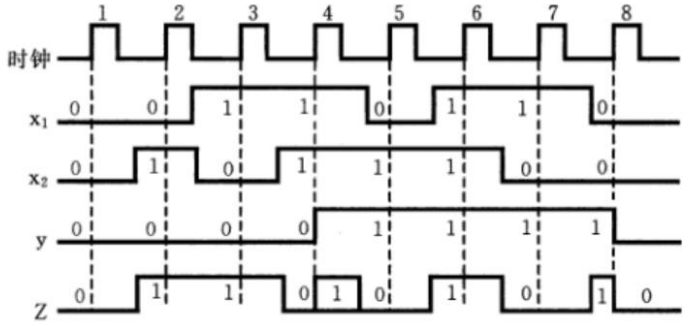


图 5.16　时间图

- 对应的时间图如图5.16所示。由图5.16可知，该电路是一个串行加法器。

- 图5.16给出了两个8位二进制数（x_1=01101100，x_2=00111010）相加得到和数（Z=10100110）的过程，y为进位信号（y=11110000）。

时钟节拍	8	7	6	5	4	3	2	1
输入 x_1	0	1	1	0	1	1	0	0
输入 x_2	0	0	1	1	1	0	1	0
状态 y	1	1	1	1	0	0	0	0
输出 Z	1	0	1	0	0	1	1	0

- 例5.4：（用代数法）分析图5.17所示同步时序逻辑电路。

- 解：
  - 由图5.17可知，该电路的输出Z与输入x没有直接关系，因此该电路属于Moore型电路。

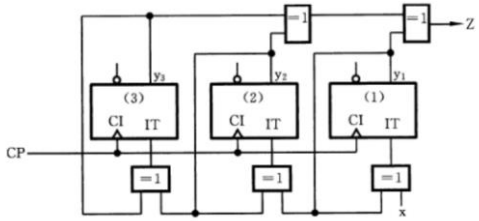


图 5.17 逻辑电路

  - （1）写出输出函数和激励函数表达式。
    - $Z = y_3 \oplus y_2 \oplus y_1$　　$T_3 = y_3 \oplus y_2$　　$T_2 = y_2 \oplus y_1$　　$T_1 = y_1 \oplus x$

T触发器的次态方程：$Q^{n+1}=Q \oplus T$

  - （2）根据T触发器的次态方程，写出电路的次态方程组。
    - $y_3^{n+1}=y_3 \oplus T_3=y_3 \oplus y_3 \oplus y_2=y_2$　　$y_2^{n+1}=y_2 \oplus T_2=y_2 \oplus y_2 \oplus y_1=y_1$　　$y_1^{n+1}=y_1 \oplus T_1=y_1 \oplus y_1 \oplus x=x$

  - （3）根据次态方程组和输出函数，作出状态表（表5.8）和状态图（图5.18）。

  - （4）由状态表和状态图可知该电路为一个3位串行输入移位寄存器。输入x与寄存器的低位相连，每来1个时钟脉冲CP，寄存器的内容从低位向高位左移1位，输入x送寄存器的低位。输出Z用来指示3位数据中1的个数是否为奇数个；如是，Z=1；否则，Z=0。

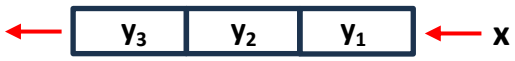


表 5.8 状态表

现态 $y_3\ y_2\ y_1$	次态 $y_3^{n+1}y_2^{n+1}y_1^{n+1}$ x=0	次态 x=1	输出 Z
0 0 0	0 0 0	0 0 1	0
0 0 1	0 1 0	0 1 1	1
0 1 0	1 0 0	1 0 1	1
0 1 1	1 1 0	1 1 1	0
1 0 0	0 0 0	0 0 1	1
1 0 1	0 1 0	0 1 1	0
1 1 0	1 0 0	1 0 1	0
1 1 1	1 1 0	1 1 1	1

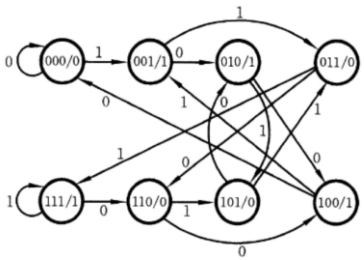


图 5.18 状态图

状态表的输入为现态（y_3、y_2、y_1）和电路的输入（x），输出为次态（y_3^{n+1}、y_2^{n+1}、y_1^{n+1}）和电路的输出Z

状态图的起点为现态（8个圆圈，圆圈中的值为现态和输出Z，$y_3y_2y_1/Z$），终点为次态（8个圆圈），连线上的信号为输入（x）

- 例5.5：设计一个模5可逆计数器，该电路有一个输入x和一个输出Z。输入x为加、减控制信号，当x=0时，计数器在时钟脉冲作用下进行加1计数，输出Z为进位输出信号；当x=1时，计数器在时钟脉冲作用下进行减1计数，输出Z为借位输出信号。试建立该计数器的Mealy型原始状态表和原始状态图。

- 解：
  - 假设计数器的5个状态分别用0、1、2、3、4表示（0为初始状态），则根据题目的要求，得到原始状态表（表5.9）。
  - 根据原始状态表（表5.9），得原始状态图（图5.20）。

表 5.9　原始状态表

现态	次态/输出 Z	
	x=0	x=1
0	1/0	4/1
1	2/0	0/0
2	3/0	1/0
3	4/0	2/0
4	0/1	3/0

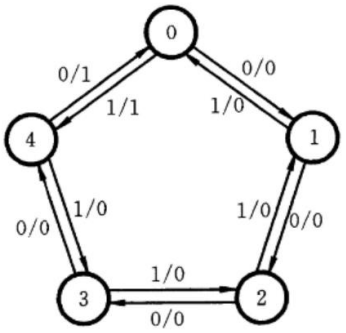


图 5.20　原始状态图

状态表的输入为现态和电路的输入（x），输出为次态和电路的输出Z

状态图的起点为现态（5个圆圈），终点为次态（5个圆圈），连线上的信号为输入/输出（x/Z）

- 例5.6：某序列检测器有一个输入端x和一个输出端Z。从输入端x输入一串随机的二进制代码，当输入序列中出现011时，输出Z=1；否则，输出Z=0。典型输入、输出序列如下，试作出该序列检测器的原始状态图和原始状态表。

输入 x	1	0	1	0	1	1	1	0	0	1	1	0
输出 Z	0	0	0	0	0	1	0	0	0	0	1	0

- 解：
  - 假设采用Mealy型电路实现该序列检测器，设电路的初始状态为A，则根据题目的要求，得到图5.21的原始状态图。

（1）初始状态A下，如果x=0，则Z=0，因为0是“011”中的第1个信号，因此，需要转移到一个新的状态B；如果x=1，则Z=0，因为1不是“011”中的第1个信号，因此，不需要转移到状态B（仍然停留在初始状态A）。

（2）状态B下，如果x=1，则Z=0，因为1是“011”中的第2个信号，因此，需要转移到一个新的状态C；如果x=0，则Z=0，因为0不是“011”中的第2个信号，因此，不需要转移到状态C（仍然停留在状态B）。

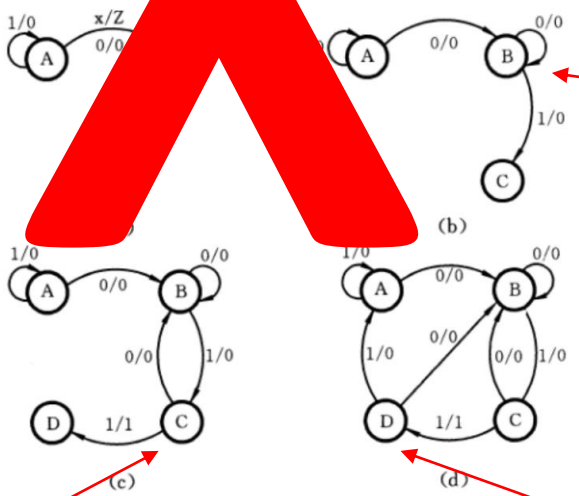


图 5.21　Mealy 型电路的原始状态图

（3）状态C下，如果x=1，则Z=1，因为1是“011”中的第3个信号，因此，需要转移到一个新的状态D，并且使Z=1；如果x=0，则Z=0，因为0不是“011”中的第3个信号，但是0可能是新的“011”序列中的第1个信号，因此，需要转移到状态B。

状态图的起点为现态（4个圆圈），终点为次态（4个圆圈），连线上的信号为输入/输出（x/Z）

（4）状态D下，如果x=1，则Z=0，因为1不是“011”中的第1个信号，因此，需要转移到状态A；如果x=0，则Z=0，因为0可能是新的“011”序列中的第1个信号，因此，需要转移到状态B。

– 根据原始状态图（图5.21），得到原始状态表（表5.10）。

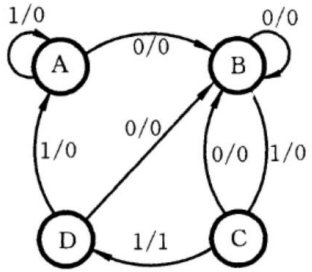


图 5.21　Mealy 型电路的原始状态图

表 5.10　Mealy 型电路的状态表

现　态	次态/输出	
	x=0	x=1
	B/0	A/0
	B/0	C/0
	B/0	D/1
	B/0	A/0

状态表的输入为现态（A、B、C、D）和电路的输入（x），输出为次态（A、B、C、D）和电路的输出Z

– 如果采用Moore型电路，[illegible]应的[illegible]状态图如图5.22所示。根据原始状态图（图5.22(b)），得到原始状[illegible]（表5.1[illegible]

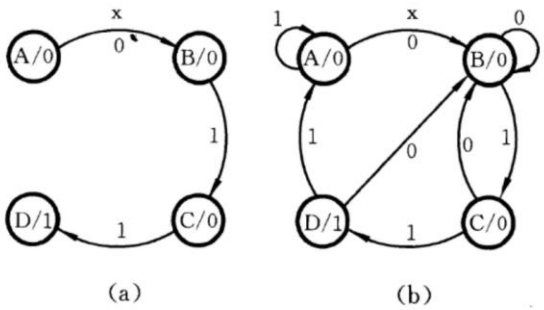


图 5.22　Moore 型电路的原始状态图

表 5.11　Moore 型电路的状态表

现　态	次　态		输　出
	x=0	x=1	Z
A	B	A	0
B	B	C	0
C	B	D	0
D	B	A	1

状态图的起点为现态（4个圆圈，圆圈中的值为状态和输出Z，状态/Z），终点为次态（4个圆圈），连线上的信号为输入（x）

状态表的输入为现态（A、B、C、D）和电路的输入（x），输出为次态（A、B、C、D）和电路的输出Z

- 例5.7：某同步时序逻辑电路用于检测串行输入的8421码，其输入的顺序是先低位后高位，当出现非法数字（1010、1011、1100、1101、1110、1111）时，电路输出1，否则输出0。试作出该时序电路的Mealy模型原始状态图和原始状态表。

- 解：
  - 假设采用Mealy型电路实现[illegible]检测[illegible]电路有1个输入x用来接收8421码，设电路的初始状态为A，则根据题目的要求[illegible]到原[illegible]态图（图5.23）。
  - 因为输入的顺序是先低位后高位[illegible]入为0101（A->B->E->J）、1101（A->C->G->N）、0011（A->B->D->I）、1011（[illegible]、0111（A->B->E->K）、1111（A->C->G->P）时，输出为1；其它情况，输出为0。
  - 根据原始状态图（图5.23[illegible]得到原始状[illegible]（表5.12）。

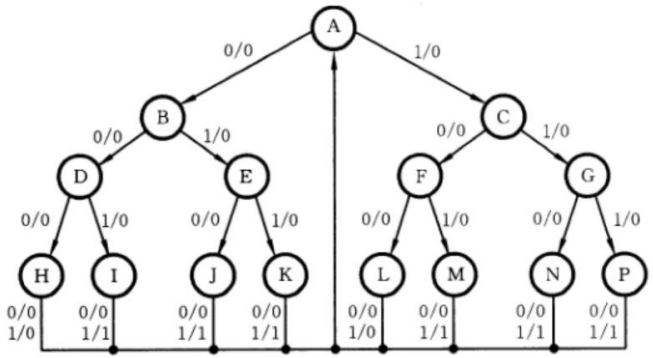


图 5.23　Mealy 型电路的原始状态图

表 5.12　Mealy 型电路的原始状态表

现　态	次态/输出		现　态	次态/输出	
	x=0	x=1		x=0	x=1
A	B/0	C/0	I	A/0	A/1
B	D/0	E/0	J	A/0	A/1
C	F/0	G/0	K	A/0	A/1
D	H/0	I/0	L	A/0	A/0
E	J/0	K/0	M	A/0	A/1
F	L/0	M/0	N	A/0	A/1
G	N/0	P/0	P	A/0	A/1
H	A/0	A/0			

状态图的起点为现态（15个圆圈），终点为次态（15个圆圈），连线上的信号为输入/输出（x/Z）

状态表的输入为现态（A、B、C、D、E、F、G、H、I、J、K、L、M、N、P）和电路的输入（x），输出为次态（A、B、C、D、E、F、G、H、I、J、K、L、M、N、P）和电路的输出Z

– 如果采用Moore型电路实现该序列检测器，除了与Mealy型电路相同的15个状态外，再增加2个状态：

① 状态X表示4位代码中10个合法代码（0000、0001、0010、0011、0100、0101、0110、0111、1000、1001）。

② 状态Y表示4位代码中6个[illegible]码（1010、[illegible]1100、1101、1110、1111）。

– 则得到原始状态图（图5.24）

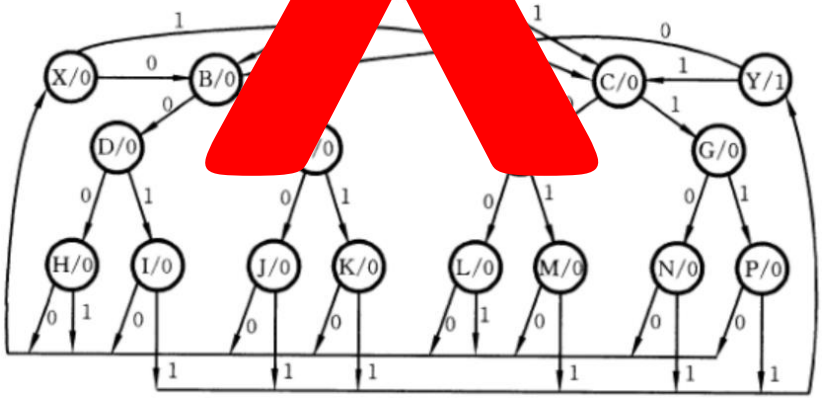


图 5.24 Moore 型电路的原始状态图

状态图的起点为现态（17个圆圈，圆圈中的值为状态和输出Z，状态/Z），终点为次态（17个圆圈），连线上的信号为输入（x）

- 例5.8：化简表5.13所示的原始状态表。

表 5.13　原始状态表

现　态	次态/输出	
	x=0	x=1
A	C/0	B/1
B	F/0	A/1
C	F/0	G/0
D	D/1	E/0
E	C/0	E/1
F	C/0	G/0
G	C/1	D/0

- 解：
  - （1）作隐含表框架（图5.26）
    - 隐含表的横坐标是（从左到右）：第1个状态至第n-1个状态
    - 隐含表的纵坐标是（从上到下）：第2个状态至第n个状态
  - （2）寻找等效对
    - 首先对原始状态表（表5.13）进行比较，得到图5.27(a)的隐含表。

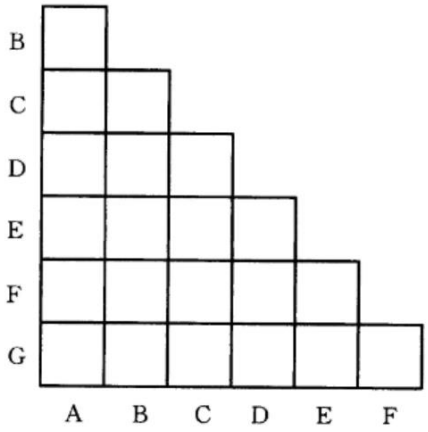


图 5.26　隐含表框架

“CF”表示状态A和状态E满足输出相同，但是它们的次态在x=1时分别为C和F，由于尚不能确定C和F是否等效，因此将CF填写到方格中

“BE”表示状态A和状态E满足输出相同，但是它们的次态在x=1时分别为B和E，由于尚不能确定B和E是否等效，因此将BE填写到方格中

“AE、CF”表示状态B和状态E满足输出相同，但是它们的次态在x=1时分别为A和E，在x=0时分别为F和C，由于尚不能确定A和E、C和F是否等效，因此将AE、CF填写到方格中

“X”表示相应的状态不满足状态等效条件

“√”表示状态C和状态F满足状态等效条件

“CD、DE”表示状态D和状态G满足输出相同，但是它们的次态在x=0时分别为D和C，在x=1时分别为E和D，由于尚不能确定C和D、D和E是否等效，因此将CD、DE填写到方格中

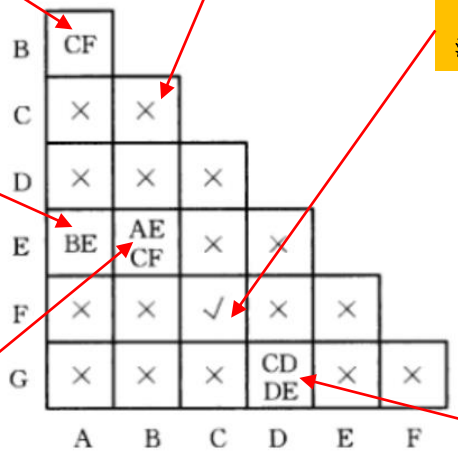


图5.27(a)　顺序比较后的隐含表

- 接下去进行关联比较，即对表5.27(a)中4个尚不能确定的方格进行关联比较，比较结果如图5.27(b)所示。
- 由图5.27(b)可知，7个状态中共有4个等效对：
  - (A,B)
  - (A,E)
  - (B,E)
  - (C,F)

表 5.13　原始状态表

现　态	次态/输出	
	x=0	x=1
A	C/0	B/1
B	F/0	A/1
C	F/0	G/0
D	D/1	E/0
E	C/0	E/1
F	C/0	G/0
G	C/1	D/0

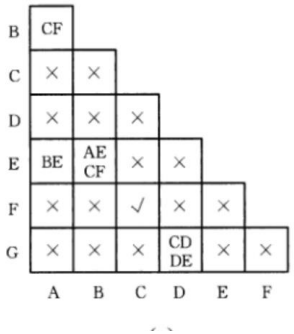
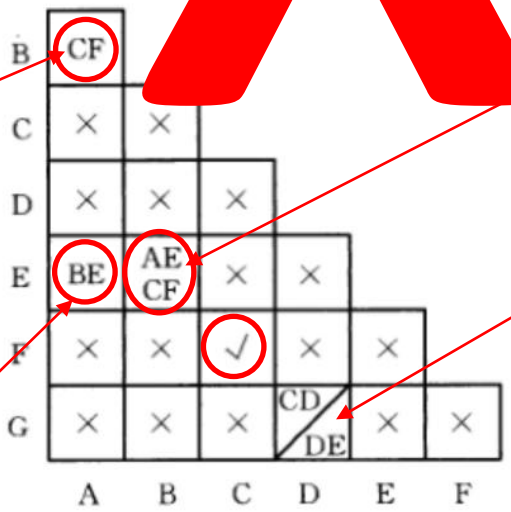


图5.27(a)　顺序比较后的隐含表



因为状态C和F是等效的，因此状态A和B也是等效的

因为状态A和E、C和F是等效的，因此状态B和E也是等效的

因为状态C和D、D和E是不等效的，因此状态D和G也是不等效的

检查状态A和E的次态对，出现如下的关系：

AE → BE → CF√

因为状态C和F是等效的，状态B和E、状态A和E构成循环，因此状态A和E是等效的

图5.27(b)　关联比较后的隐含表

– （3）求出最大等效类

- 4个等效对(A,B)、(A,E)、(B,E)、(C,F)中，(A,B)、(A,E)、(B,E)构成一个最大等效类：{A,B,E}； (C,F)不包含在其他等效类中，它也是一个最大等效类：{C,F}。
- 状态D和G不和其他任何[illegible]效，也各自[illegible]个最大等效类：{D}、{G}。
- 因此，共有4个最大等效类[illegible]E}，{[illegible]}，{G}。

– （4）作出最小化状态表

- 将最大等效类{A,B,E}，{C,F}[illegible]G}，[illegible]a、b、c、d表示，并对原始状态表（表5.13）中的状态作相应替代，即可得到化简[illegible]小化状[illegible]表5.14）。

表 5.13 原始状态表

现 态	次态/输出	
	x=0	x=1
A	C/0	B/1
B	F/0	A/1
C	F/0	G/0
D	D/1	E/0
E	C/0	E/1
F	C/0	G/0
G	C/1	D/0

表 5.14 最小化状态表

现 态		次态/输出	
		x=0	x=1
{A,B,E}	a	{C,F} b/0	{A,B,E} a/1
{C,F}	b	{C,F} b/0	{G} d/0
{D}	c	{D} c/1	{A,B,E} a/0
{G}	d	{C,F} b/1	{D} c/0

- 例5.9：对表5.15所示的状态表进行状态编码。

表 5.15　状态表

现　态	次态/输出	
	x=0	x=1
A	C/0	A/0
B	A/0	A/0
C	A/0	D/0
D	C/0	A/1

- 解：
  - 由表5.15可知，有4个状态，n=4；状态编码的长度m=2。
  - 根据相邻法的编码原则，表[illegible]中4个状[illegible]相邻关系如下：
    - 由原则①得到：状态B和C、[illegible]A和[illegible]配相邻的二进制代码；
    - 由原则②得到：状态A和C、[illegible]分[illegible]的二进制代码；
    - 由原则③得到：状态A和B、A[illegible]分配相邻的二进制代码。
  - 可见，如果要同时满足上述3[illegible]，[illegible]满足A和B、A和C、A和D、B和C都相邻，显然这是不能全部满足的。
  - 可以用卡诺图作为状态分[illegible]工具，假设[illegible]码为00，一种尽可能满足3条原则的状态编码方案如图5.28所示。该方案满足A和C、A和D、B和C相邻，不满足A和B相邻。
  - 该状态分配方案对应的二进制状态表如表5.16所示：A=00，B=11，C=01，D=10。

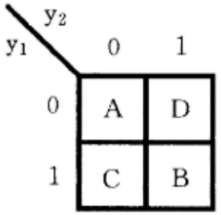


图 5.28　状态分配方案

表 5.16　二进制状态表

现　态		次态 $y_2^{n+1}y_1^{n+1}$/输出	
y_2	y_1	x=0	x=1
A 0	0	C 01/0	A 00/0
C 0	1	A 00/0	D 10/0
B 1	1	A 00/0	A 00/0
D 1	0	C 01/0	A 00/1

- 例5.10：用J-K触发器和适当的逻辑门，实现表5.21所示二进制状态表的功能（实现同步时序逻辑电路）。

- 解：
  - 首先，根据二进制状态表（表5.21）和J-K触发器激励表（表5.19），列出激励函数和输出函数真值表（表5.22）。前3列[illegible]写出，中间[illegible]由二进制状态表（表5.22）和J-K触发器的激励表（表5.19）得到，最后1列由[illegible]制状态[illegible]表5.22）得到。

表 5.21　二进制状态表

现态 y_2	现态 y_1	次态 $y_2^{n+1}y_1^{n+1}$/输出 Z x=0	次态 $y_2^{n+1}y_1^{n+1}$/输出 Z x=1
0	0	11/0	01/0
0	1	00/0	00/1
1	1	00/1	10/1
1	0	01/0	11/0

表 5.19　J-K 触发器激励表

Q	→	Q^{n+1}	J	K
0	→	0	0	d
0	→	1	1	d
1	→	0	d	1
1	→	1	d	0

$y_2 = 0 \to y_2^{n+1} = 1$，要求：JK=1 d
$y_1 = 0 \to y_1^{n+1} = 1$，要求：JK=1 d

$y_2 = 1 \to y_2^{n+1} = 1$，要求：JK=d 0
$y_1 = 1 \to y_1^{n+1} = 0$，要求：JK=d 1

表 5.22　激励函数和输出函数真值表

输入和现态 x	y_2	y_1	激励函数 J_2	K_2	J_1	K_1	输出函数 Z
0	0	0	1	d	1	d	0
0	0	1	0	d	d	1	0
0	1	0	d	1	1	d	0
0	1	1	d	1	d	1	1
1	0	0	0	d	1	d	0
1	0	1	0	d	d	1	1
1	1	0	d	0	1	d	0
1	1	1	d	0	d	1	1

– 其次，由真值表（表5.22），作出激励函数和输出函数的卡诺图（图5.29）。

表 5.22　激励函数和输出函数真值表

输入和现态			激励函数				输出函数
x	y_2	y_1	J_2	K_2	J_1	K_1	Z
0	0	0	1	d	1	d	0
0	0	1	0	d	d	1	0
0	1	0	d	1	1	d	0
0	1	1	d	1	d	1	1
1	0	0	0	d	1	d	0
1	0	1	0	d	d	1	1
1	1	0	d	0	1	d	0
1	1	1	d	0	d	1	1

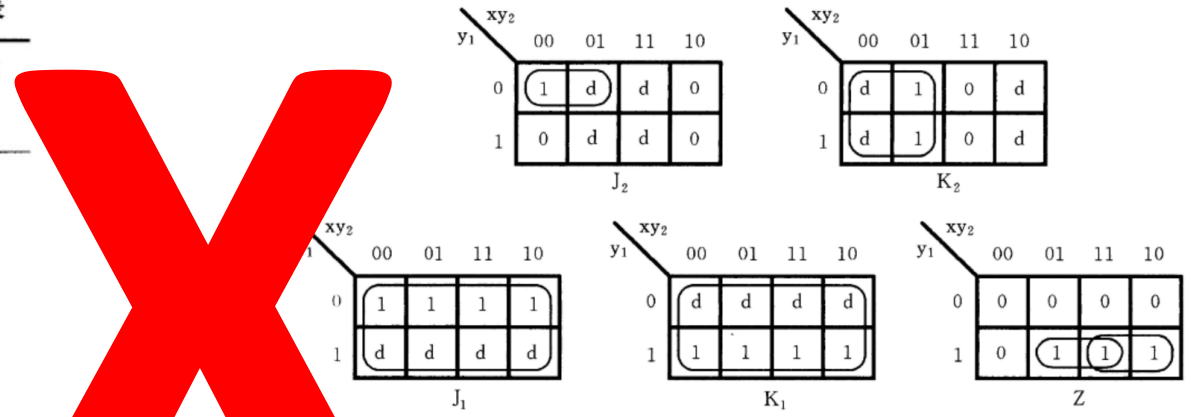


图 5.29　激励函数和输出函数卡诺图

– 根据卡诺图，得到化简后的逻辑公式：
  - $J_2 = /x \cdot /y_1$　　$K_2 = /x$
  - $J_1 = 1$　　$K_1 = 1$
  - $Z = y_2 \cdot y_1 + x \cdot y_1 = (y_2 + x) \cdot y_1$

– 由逻辑公式，画出逻辑电路（图5.30）。

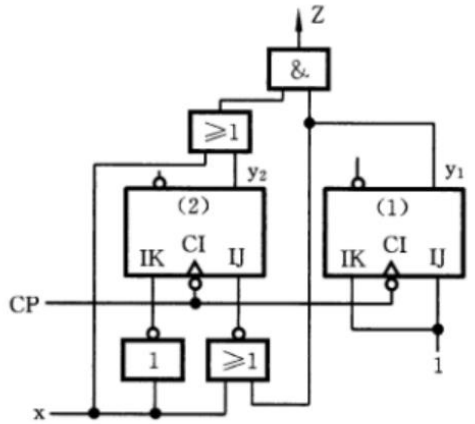


图 5.30　逻辑电路

– 现要求采用D触发器作为存储元件，实现表5.21所示二进制状态表的功能。

– 首先，根据二进制状态表（表5.21）和D触发器激励表（表5.18），可以直接作出激励函数的卡诺图（图5.31）。

表 5.21　二进制状态表

现态 y_2	现态 y_1	次态 $y_2^{n+1}y_1^{n+1}$/输出 Z	
		x=0	x=1
0	0	11/0	01/0
0	1	00/0	00/1
1	1	00/1	10/1
1	0	01/0	11/0

D 触 励表

Q →	D
0	0
0	1
1	0
1	1

因为D触发器的次态方程：$Q^{n+1}=D$，即次态=激励，因此D2卡诺图的值即为y2的次态

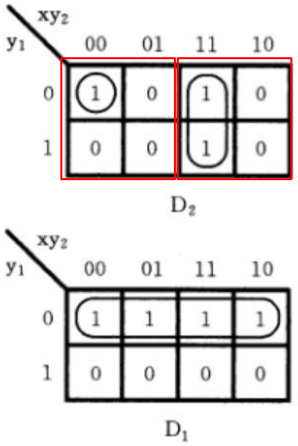


图 5.31　采用 D 触发器的激励函数卡诺图

– 根据卡诺图，得到化简后的逻辑公式：
  - $D_2 = /x \cdot /y_2 \cdot /y_1 + x \cdot y_2 = (x + y_2 + y_1) + x \cdot y_2$
  - $D_1 = /y_1$
  - $Z = y_2 \cdot y_1 + x \cdot y_1 = (y_2 + x) \cdot y_1$

– 根据逻辑公式，画出逻辑电路（图5.32）。

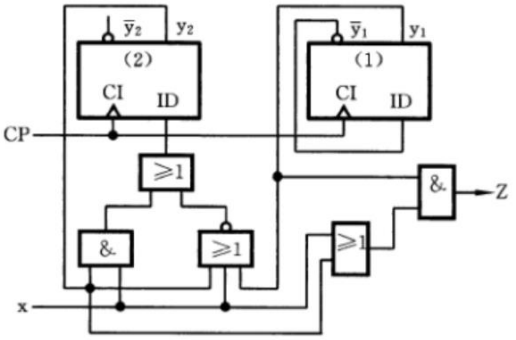


图 5.32　采用 D 触发器的逻辑电路

– 此外，激励函数也可以根据二进制状态表和触发器的次态方程确定。具体步骤如下：

– J-K触发器的次态方程为：$Q^{n+1}=J\cdot/Q+/K\cdot Q$。

– 首先，根据二进制状态表（表5.21）作出次态卡诺图（右下角）；根据次态卡诺图，得到次态函数表达式：

- $y_2^{n+1} = /x\cdot/y_2\cdot/y_1 + x\cdot y_2 = /x\cdot/$ $/x\cdot y_2$
- $y_1^{n+1} = /y_1 = 1\cdot/y_1 + /1\cdot y_1$

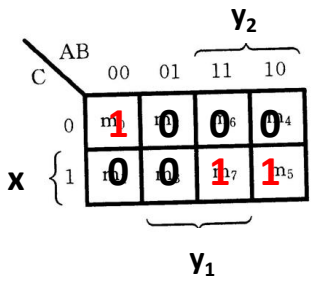
– 然后，将次态函数表达式与触 方程相比较，得到激励函数表达式：

- $J_2 = /x\cdot/y_1$ $K_2 = /x$
- $J_1 = 1$ $K_1 = 1$

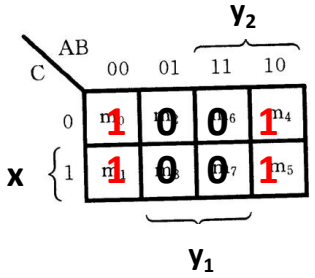
– 可见，这种方法与采用触发器激励表的方法，得到的激励函数表达式完全一样。

表 5.21 二进制状态表

现态 y_2	现态 y_1	次态 $y_2^{n+1}y_1^{n+1}$/输出 Z x=0	次态 $y_2^{n+1}y_1^{n+1}$/输出 Z x=1
0	0	11/0	01/0
0	1	00/0	00/1
1	1	00/1	10/1
1	0	01/0	11/0



次态卡诺图



- 例5.11：设计一个用于引爆控制的同步时序逻辑电路。该电路有一个输入端x和一个输出端Z。平时输入x始终为0，一旦需要引爆，则从x端连续输入4个“1”（不被“0”间断）。电路收到第4个“1”后，在输出端产生一个“1”信号点火引爆，该电路连同引爆装置一起被炸毁。试建立该电路的Mealy型原始状态图和原始状态表。

- 解：
  - 该电路实际上是一个“[illegible]序列[illegible]器。该电路有两个约束条件：（1）一旦输入出现1，则一定是连续[illegible]“[illegible]（2）输出端产生“1”后，电路自毁，此时不再存在次态问题。
  - 因此，可以得到该电路的[illegible]（也称不完全确定原始状态图）和原始状态表（也称不完全确定原始[illegible]表）

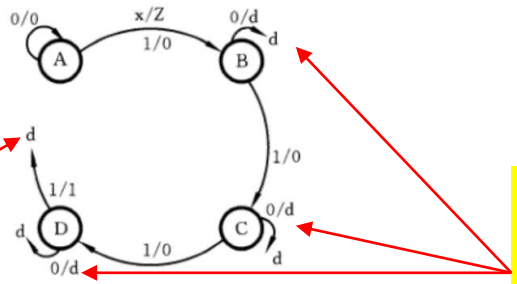


图 5.33 原始状态图

D状态时，输入x=1的情况，此时输出Z=1，并且电路将自毁，因此用d表示次态可以是任意

B、C、D状态时，因为不可能出现输入x=0的情况，因此用d表示次态可以是任意，0/d表示输出可以是任意

表 5.23 原始状态表

现 态	次态/输出	
	x=0	x=1
A	A/0	B/0
B	d/d	C/0
C	d/d	D/0
D	d/d	d/1

- 例5.12：化简表5.24所示的原始状态表。

表 5.24　原始状态表

现态	次态/输出	
	x=0	x=1
A	d/d	A/d
B	B/0	C/1
C	d/1	D/0
D	B/d	d/d
E	C/1	A/0

- 解：
  - （1）作隐含表（表5.36），寻找相容状态对。
    - 由表5.36可知有7个相容状态对：(A,B)、(A,D)、(A,E)、(B,D)、(C,D)、(C,E)。
  - （2）作状态合并图（图5.37），找出最大相容类。
    - 图5.37中的7条线对应7个相容状态对，由图5.37得到3个最大相容类：{A,B,D}、{A,C,D}、{A,C,E}（对应3个三角形）。
  - （3）作闭覆盖表（表5.25），找最小闭覆盖集。
    - 由表5.25可知，最小闭覆盖集由3个最大相容类{A,B,D}、{A,C,E}组成。
  - （4）状态合并，作出最小化状态表（最简状态表）。
    - 假设分别用a、b表示2个最大相容类{A,B,D}、{A,C,E}，则得到最小化状态表（最简状态表，表5.26）。

B	AC			
C	AD	×		
D	√	√	√	
E	√	×	AD	BC
	A	B	C	D

图 5.36　隐含表

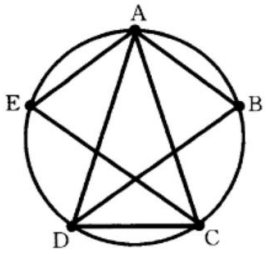


图 5.37　状态合并图

最大相容类{A,C,D}可以去掉，因为最大相容类{A,B,D}、{A,C,E}已经包含了5个状态（覆盖），并且{A,C,D}的闭合（B、AD）包含在{A,B,D}、{A,C,E}中

表 5.25　闭覆盖表

最大相容类	覆盖					闭合	
	A	B	C	D	E	x=0	x=1
ABD	A	B		D		B	AC
ACD	A		C	D		B	AD
ACE	A		C		E	C	AD

表 5.26　最小化状态表

现态	次态/输出	
	x=0	x=1
{A,B,D} a	{A,B,D} a/0	{A,C,E} b/1
{A,C,E} b	{A,C,E} b/1	{A,B,D} a/0

- 例5.13：化简表5.27所示的原始状态表。

表 5.27　原始状态表

现态	次态/输出	
	x=0	x=1
A	D/d	A/0
B	E/0	A/d
C	D/0	B/0
D	C/d	C/d
E	d/1	B /d

- 解：
  - （1）作隐含表（表5.38）
    - 由表5.38可知有7个相容对：(A,B)、(A,D)、(A,E)、(B,C)、(C,D)、(D,E)。
  - （2）作状态合并图（图5.39），求最大相容类。
    - 图5.39中的7条线对应7个相容对，由图5.39得到3个最大相容类：{A,B,C}、{A,C,D}、{A,D,E}（对应3个三角形）。
  - （3）作闭覆盖表（表5.28），求最小闭覆盖。
    - 由表5.28可知，最小闭覆盖由3个最大相容类{A,B,C}、{A,C,D}、{A,D,E}组成。
  - （4）状态合并，作出最小化状态表。
    - 假设分别用a、b、c表示3个最大相容类{A,B,C}、{A,C,D}、{A,D,E}，则得到最小化状态表（表5.29）。

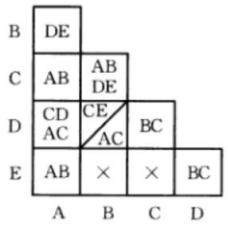


图 5.38　隐含表

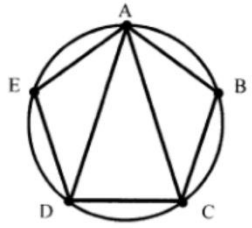


图 5.39　状态合并图

表 5.28　闭覆盖表

最大相容类	覆盖					闭合	
	A	B	C	D	E	x=0	x=1
ABC	A	B	C			DE	AB
ACD	A		C	D		CD	ABC
ADE	A			D	E	CD	ABC

表 5.29　最小化状态表

现态	次态/输出	
	x=0	x=1
{A,B,C} a	c/0	a/0
{A,C,D} b	b/0	a/0
{A,D,E} c	b/1	a/0

– 表5.29所示的最小化状态表并不是最简状态表。

– 可以选择{A,B,C}、{D,E}作为最小闭覆盖，得到表5.30的闭覆盖表。

– 假设分别用a、b表示相容类{A,B,C}、{D,E}，可以得到表5.31的最小化状态表（最简状态表）。

– 显然，表5.31比表5.29简单一些。

表 5.30　闭覆盖表

最大相容类	覆盖					闭合	
	A	B	C	D	E	x=0	x=1
ABC	A	B	C			DE	AB
DE				D	E	C	BC

表 5.31　最小化状态表

现态	次态/输出	
	x=0	x=1
{A,B,C} a	b/0	a/0
{D,E} b	a/1	a/d

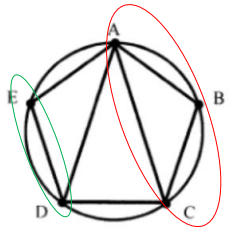


图 5.39　状态合并图

表 5.27　原始状态表

现态	次态/输出	
	x=0	x=1
A	D/d	A/0
B	E/0	A/d
C	D/0	B/0
D	C/d	C/d
E	d/1	B /d

表 5.28　闭覆盖表

最大相容类	覆盖					闭合	
	A	B	C	D	E	x=0	x=1
ABC	A	B	C			DE	AB
ACD	A		C	D		CD	ABC
ADE	A			D	E	CD	ABC

表 5.29　最小化状态表

现态	次态/输出	
	x=0	x=1
a	c/0	a/0
b	b/0	a/0
c	b/1	a/0

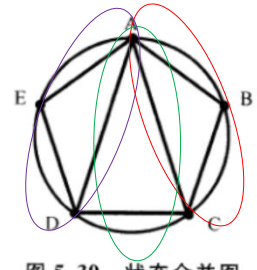


图 5.39　状态合并图

- 例5.14：用T触发器作为存储单元，设计一个2位二进制减1计数器。电路的工作状态受输入信号x控制，当x=0时，电路状态不变；当x=1时，在时钟脉冲作用下进行减1计数。计数器有一个输出端Z，当产生借位时，Z=1；否则，Z=0。

- 解：
  - （1）作出状态图和状[illegible]

    - 因为是2位二进制减1计[illegible]因此用y_2、y_1表示状态变量。状态图如图5.40所示，状态表如表5.32所

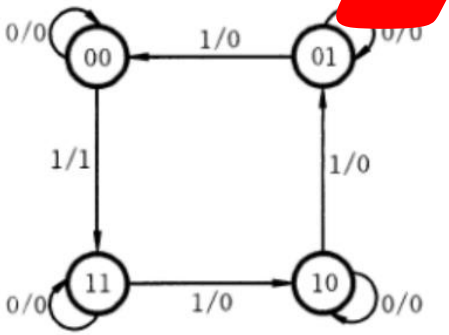


图 5.40　状态图

表 5.32　状态表

现态 y_2	现态 y_1	次态 $y_2^{n+1}y_1^{n+1}$/输出 Z	
		x=0	x=1
0	0	00/0	11/1
0	1	01/0	00/0
1	1	11/0	10/0
1	0	10/0	01/0

状态图的起点为现态（4个圆圈），终点为次态（4个圆圈），连线上的信号为输入/输出（x/Z）

状态表的输入为现态（y_2、y_1）和电路的输入（x），输出为次态（y_2^{n+1}、y_1^{n+1}）和电路的输出Z

– （2）确定激励函数和输出函数，并化简。

- 根据状态表（表5.32），以及T触发器的激励表（表5.20），可以画出激励函数和输出函数的卡诺图（图5.41）。
- 根据卡诺图，得到化简后的激励函数和输出函数公式：$T_2=y_1$　　$T_1=x$　　$Z=x\cdot\overline{y_2}\cdot\overline{y_1}$

x=0时：
次态=现态，因此，T_1=0、T_2=0

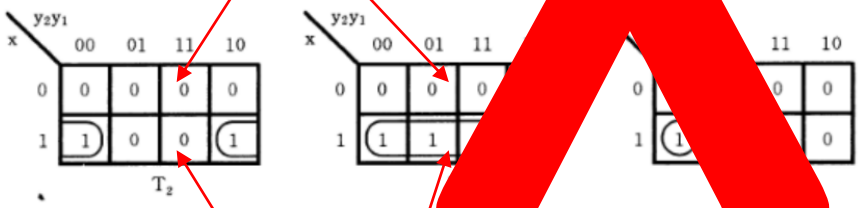


图 5.41　激励函数和输出函数卡诺图

x=1时：
现态=00，次态=11，因此，T_1=1、T_2=1
现态=01，次态=00，因此，T_1=0、T_2=1
现态=11，次态=10，因此，T_1=0、T_2=1
现态=10，次态=01，因此，T_1=1、T_2=1

x=0时：输出Z分别为0、0、0、0
x=1时：输出Z分别为1、0、0、0

表 5.32　状态表

现态 y_2	现态 y_1	次态 $y_2^{n+1}y_1^{n+1}$/输出 Z x=0	次态 $y_2^{n+1}y_1^{n+1}$/输出 Z x=1
0	0	00/0	11/1
0	1	01/0	00/0
1	1	11/0	10/0
1	0	10/0	01/0

表 5.20　T 触发器激励表

Q	→	Q^{n+1}	T
0		0	0
0		1	1
1		0	1
1		1	0

- （3）画出逻辑电路图（图5.42）。

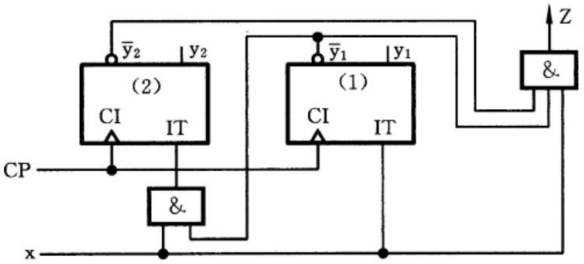
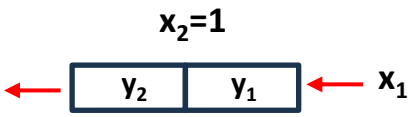


图 5.42　逻辑电路

- 例5.15：设计一个2位串行输入、并行输出双向移位寄存器。该寄存器有x_1和x_2两个输入端，其中x_2为控制端，用于控制移位方向，x_1为数据输入端。当x_2=0时，x_1往寄存器高位串行送数，寄存器中的数据从高位移向低位；当x_2=1时，x_1往寄存器低位串行送数，寄存器中的数据从低位移向高位。寄存器的输出为触发器状态本身。

- 解：
  - （1）作出状态图和状态表。
    - 因为是2位移位寄存器，因此用 个状态变量。状态图如图5.43所示，二进制状态表如表5.33所示。





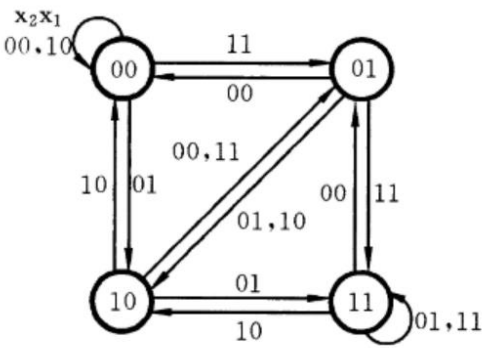


图 5.43　状态图

表 5.33　二进制状态表

现态 y_2	现态 y_1	次态 $y_2^{n+1}y_1^{n+1}$ $x_2x_1=00$	$x_2x_1=01$	$x_2x_1=11$	$x_2x_1=10$
0	0	0 0	1 0	0 1	0 0
0	1	0 0	1 0	1 1	1 0
1	1	0 1	1 1	1 1	1 0
1	0	0 1	1 1	0 1	0 0

x_2x_1=00：x_2=0，左移，移入的数据x_1=0，因此：
现态=00，次态=00
现态=01，次态=00
现态=11，次态=01
现态=10，次态=01

x_2x_1=01：x_2=0，左移，移入的数据x_1=1，因此：
现态=00，次态=10
现态=01，次态=10
现态=11，次态=11
现态=10，次态=11

x_2x_1=10：x_2=0，右移，移入的数据x_1=0，因此：
现态=00，次态=00
现态=01，次态=10
现态=11，次态=10
现态=10，次态=00

x_2x_1=11：x_2=1，右移，移入的数据x_1=1，因此：
现态=00，次态=01
现态=01，次态=11
现态=11，次态=11
现态=10，次态=01

状态图的起点为现态（4个圆圈），终点为次态（4个圆圈），连线上的信号为输入（x_2、x_1）

状态表的输入为现态（y_2、y_1）和电路的输入（x_2、x_1），输出为次态（y_2^{n+1}、y_1^{n+1}）

– （2）确定激励函数和输出函数，并化简。

- 采用D触发器作为存储元件，根据二进制状态表（表5.33），以及D触发器的激励表（表5.18），可以画出激励函数的卡诺图（图5.44）。

- 根据卡诺图，得到化简后的方程式： $D_2=/x_2\cdot x$ $D_1=/x_2\cdot y_2+x_2\cdot x_1$

因为D触发器的次态=激励，因此卡诺图中的1、2、3、4列，对应表5.33中的第3、5、7、9列

因为D 因此卡诺图中的1、2、3、 的第4、6、8、10列

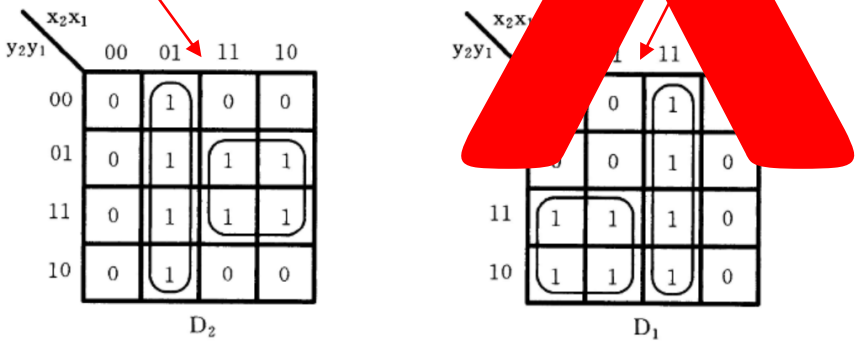


图 5.44 卡诺图

表 5.33 二进制状态表

现态 y_2	现态 y_1	次态 $y_2^{n+1}y_1^{n+1}$ $x_2x_1=00$	$x_2x_1=01$	$x_2x_1=11$	$x_2x_1=10$
0	0	0 0	1 0	0 1	0 0
0	1	0 0	1 0	1 1	1 0
1	1	0 1	1 1	1 1	1 0
1	0	0 1	1 1	0 1	0 0

表 5.18 D 触发器激励表

Q → Q^{n+1}	D
0 0	0
0 1	1
1 0	0
1 1	1

– （3）画出逻辑电路图（图5.45）。

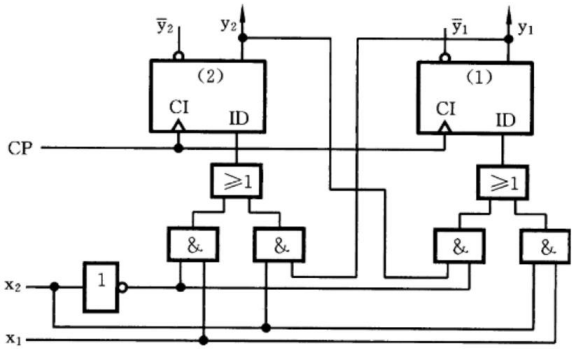


图 5.45 逻辑电路图

- 例5.16：用J-K触发器作为存储单元，设计一个“101”序列检测器。假设该电路有一个输入x和一个输出Z，当随机输入信号中出现“101”序列时，输出一个“1”信号。典型输入/输出序列如下：

输入 x	0	0	1	0	1	0	1	1	0	1	0	0
输出 Z	0	0	0	0	1	0	1	0	0	1	0	0

- 解：
  - 假设用Moore型电路实现给[illegible]。
  - （1）作出状态图和状态表。设[illegible]为A，根据题目要求，可以得到状态图如图5.46所示，状态表如表5.34所示。

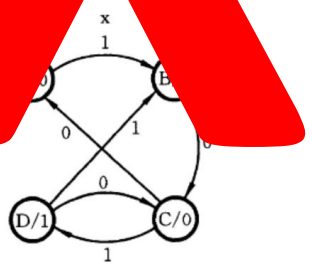


图 5.46 状态图

表 5.34 状态表

现态	次态 x=0	次态 x=1	输出 Z
A	A	B	0
B	C	B	0
C	A	D	0
D	C	B	1

状态图的起点为现态（4个圆圈，圆圈中的值为状态和输出Z，状态/Z），终点为次态（4个圆圈），连线上的信号为输入（x）

状态表的输入为现态（A、B、C、D）和电路的输入（x），输出为次态（A、B、C、D）和电路的输出Z

  - （2）状态化简。表5.34已经是最小化状态表（最简状态表）。

– （3）状态编码。因为有4个状态，采用2位二进制数进行编码（y_2、y_1），需要2个触发器。编码方案如图5.47所示。相应的二进制状态表如表5.35所示。

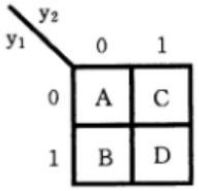


图 5.47　状态编码方案

表 5.35　二进制状态表

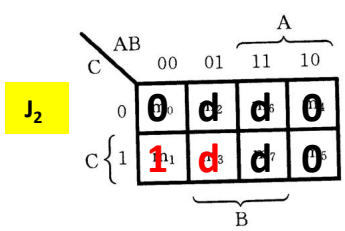
现态	次态 $y_2^{n+1}y_1^{n+1}$		输出
	x=0	x=1	
A	0 0	0	
B	1 0	0	0
C	0		0
D	1	0	1

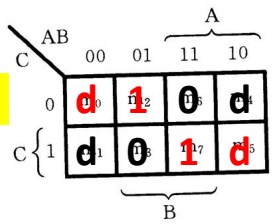
表 5.19　J-K 触发器激励表

Q → Q^{n+1}	J	K
0 0	0	d
0 1	1	d
1 0	d	1
1 1	d	0

– （4）确定激励函数和输出函数[illegible]进制状态表（表5.35），以及J-K触发器的激励表（表5.19），可以得到激励函数和[illegible]函[illegible]值表（表5.36）。根据真值表，画出卡诺图，然后得到化简后的激励函数和输[illegible]数：

- $J_2=/x\cdot y_1$　　$K_2=/x\cdot/y_1+/x\cdot y_1$
- $J_1=x$　　$K_1=/x$
- $Z=y_2\cdot y_1$





– （5）画出逻辑电路图（图5.48）。

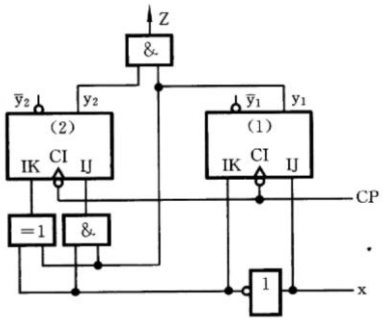
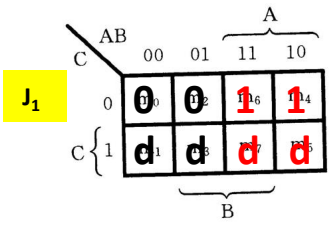
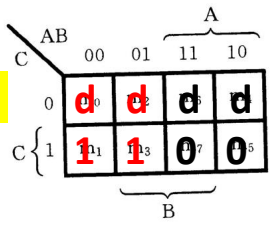


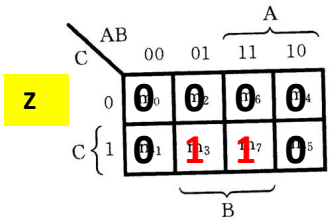
图 5.48　逻辑电路

表 5.36　激励函数和输出函数真值表

输入	现态		次态		激励函数				输出
x	y_2	y_1	y_2^{n+1}	y_1^{n+1}	J_2	K_2	J_1	K_1	Z
0	0	0	0	0	0	d	0	d	0
0	0	1	1	0	1	d	d	1	0
0	1	0	0	0	d	1	0	d	0
0	1	1	1	0	d	0	d	1	1
1	0	0	0	1	0	d	1	d	0
1	0	1	0	1	0	d	d	0	0
1	1	0	1	1	d	0	1	d	0
1	1	1	0	1	d	1	d	0	1







- 例5.17：设计一个串行输入3位二进制码的奇偶检测器。该电路从输入端x串行输入二进制代码，每3位为一组，当3位代码中含1的个数为偶数时，输出Z=1，平时Z=0。

- 解：
  - 假设采用Mealy型电路实现给定的功能。
  - （1）建立原始状态图和原始状态表。设初始状态为A，根据题目要求，可以得到原始状态图如图5.49所示（注：000也表示一个状态），原始状态表如表5.37所示，共有7个不同状态。
  - （2）状态化简。用隐含表法化简表5.38的最小化状态表（最简状态表），由7个状态简化为5个状态。

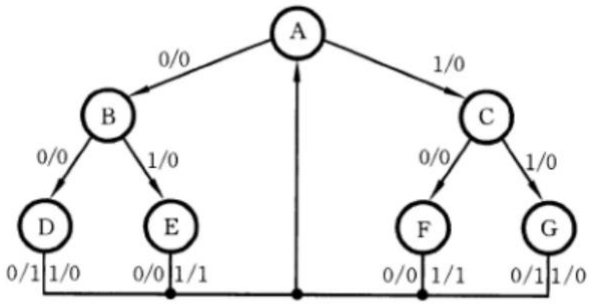


图 5.49　原始状态图

原始状态表

现　态	次态/输出	
	x=0	x=1
A	B/0	C/0
B	D/0	E/0
C	F/0	G/0
D	A/1	A/0
E	A/0	A/1
F	A/0	A/1
G	A/1	A/0

表 5.38　最小化状态表

现　态	次态/输出	
	x=0	x=1
A	B/0	C/0
B	D/0	E/0
C	E/0	D/0
D	A/1	A/0
E	A/0	A/1

状态图的起点为现态（7个圆圈），终点为次态（7个圆圈），连线上的信号为输入/输出（x/Z）

状态表的输入为现态（A、B、C、D、E、F、G）和电路的输入（x），输出为次态（A、B、C、D、E、F、G）和电路的输出Z

– （3）状态编码。因为有5个状态，采用3位二进制数进行编码（y_3、y_2、y_1），需要3个触发器。编码方案如图5.50所示。相应的二进制状态表如表5.39所示。

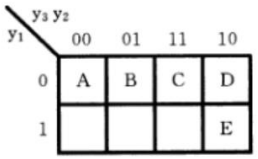


图 5.50　状态分配方案

表 5.39　二进制状态表

y_3 y_2 y_1	$y_3^{n+1}y_2^{n+1}y_1^{n+1}/Z$	
	x=0	x=1
A 0	010/0	110/0
B	100/0	101/0
C	101/0	100/0
D	000/1	000/0
1	000/0	000/1

– （4）确定激励函数和输出函数。选用D触发器作为存储元件，根据状态表（表5.39），以及D触发器的激励表（表5.18），可以得到激励函数和输出函数的卡诺图（图5.51）。根据卡诺图，得到化简后的激励函数和输出函数：

- $D_3=x\cdot/y_3+y_2$
- $D_2=/y_3\cdot/y_2=/(y_3+y_2)$
- $D_1=/x\cdot y_3\cdot y_2+x\cdot/y_3\cdot y_2=(x\oplus y_3)\cdot y_2$
- $Z=x\cdot y_1+/x\cdot y_3\cdot/y_2\cdot/y_1$

表 5.18　D 触发器激励表

Q → Q^{n+1}		D
0	0	0
0	1	1
1	0	0
1	1	1

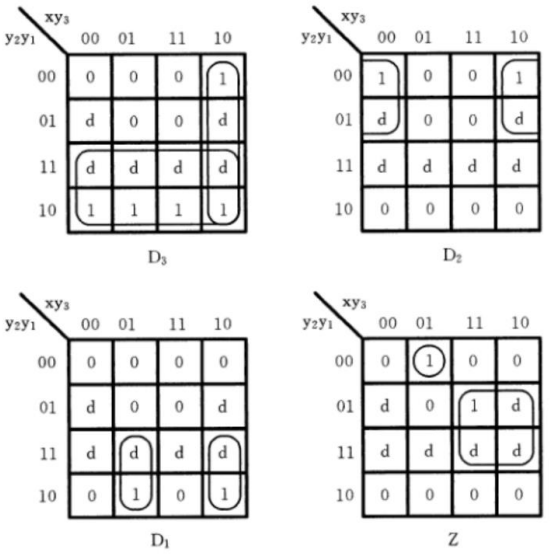


图 5.51　卡诺图

– （5）讨论。因为采用3位二进制数进行编码，共有8个状态，实际使用了5个状态，存在3个无效状态（001、011、111）。需要考虑两个问题：

- ①电路偶然进入无效状态时，能否在输入信号和时钟脉冲作用下，自动进入有效状态？如果能够进入有效状态，则称为具有自恢复功能；否则称为“挂起”。

- ②电路进入无效状态时会输出错误，针对本例，电路进入无效状态时，如果输出Z=1，则表示产生了错误的结果，不允许的。

- 解决方法：
  - 建立无效状态检查表（这里有3个无效状态，每个状态又有x=0、x=1两种情况，共8种情况，见表5.40中的第1
  - 根据前面得到的公式·/y_3 =/y_3·/y_2=/(y_3+y_2)；D_1=/x·y_3·y_2+x·/y_3·y_2 =(x⊕y_3)·y_2），计算8种情况的激励函数、D_2、见表5.40中第5至7列。
  - 根据D触发器的，得到8种情态值（y_3^{n+1}、y_2^{n+1}、y_1^{n+1}），见表5.40中第8至10列。
  - 根据前面得到的公式（Z=x·y_1+/x·y_3·/y_2·/y_1），计算8种情况的输出值Z，见表5.40最后1列。

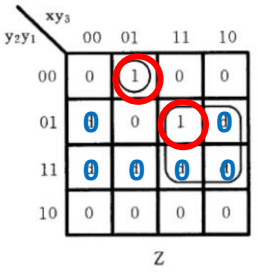
表 5.40　无效状态检查表

输入	现态			激励函数			次态			输出
x	y_3	y_2	y_1	D_3	D_2	D_1	y_3^{n+1}	y_2^{n+1}	y_1^{n+1}	Z
0	0	0	1	0	1	0	0	1	0	0
0	0	1	1	1	0	0	1	0	0	0
0	1	1	1	1	0	1	1	0	1	0
1	0	0	1	1	1	0	1	1	0	1
1	0	1	1	1	0	1	1	0	1	1
1	1	1	1	1	0	0	1	0	0	1

表 5.18　D 触发器激励表

Q → Q^{n+1}		D
0	0	0
0	1	1
1	0	0
1	1	1

- 根据表5.39和表5.40，得到包括无效状态的状态图（图5.52），可知无效状态不会产生“挂起”现象：
  - 001无效状态会转移到010或110有效状态。
  - 011无效状态会转移到100或101有效状态。
  - 111无效状态会转移到100或101有效状态。



- 但是，无效状态会导致输出Z=1（见表5.40最后3行）。
- 解决办法：将输出Z的卡诺图中的d置为0，得到右边的卡诺图。
- 根据右边的卡诺图，得到：Z=/(/y₃+y₂·y₁=/(/y₃+y₂+(x⊕y₁))。

– （6）画出逻辑电路（如图5.53所示）

表 5.39 二进制状态表

y_3 y_2 y_1	$y_3^{n+1}y_2^{n+1}y_1^{n+1}/Z$	
	x=0	x=1
0 0 0	010/0	110/0
0 1 0	100/0	101/0
1 1 0	101/0	100/0
1 0 0	000/1	000/0
1 0 1	000/0	000/1

表 5.40 无效状态检查表

输入	现态			激励函数			次态			输出
x	y_3	y_2	y_1	D_3	D_2	D_1	y_3^{n+1}	y_2^{n+1}	y_1^{n+1}	Z
0	0	0	1	0	1	0	0	1	0	0
0	0	1	1	1	0	0	1	0	0	0
0	1	1	1	1	0	1	1	0	1	0
1	0	0	1	1	1	0	1	1	0	1
1	0	1	1	1	0	1	1	0	1	1
1	1	1	1	1	0	0	1	0	0	1

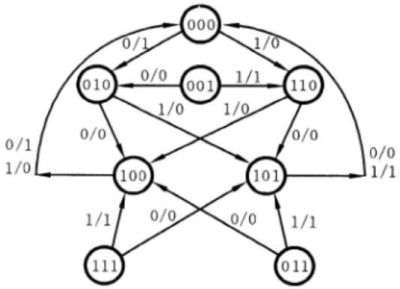


图 5.52 方案检查状态图

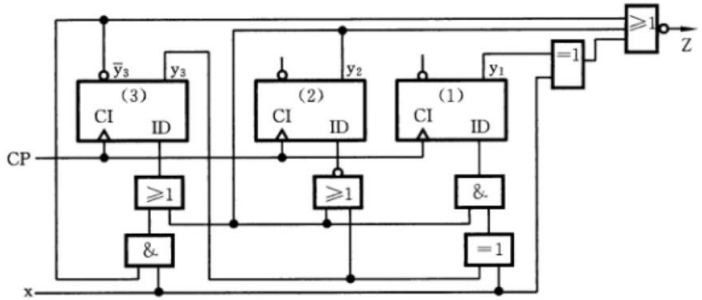


图 5.53 逻辑电路

第6章　异步时序逻辑电路

- 6.1　异步时序逻辑电路的特点与分类
- 6.2　脉冲异步时序逻辑电路
- 6.3　电平异步时序逻辑电路

第6章主要内容

- 异步时序逻辑电路的特点：
  ① 电路中没有统一的时钟脉冲信号同步，电路状态的改变是外部输入信号变化直接作用的结果。
  ② 在状态转移过程中，各存储元件的状态变化不一定发生在同一时刻，不同状态的维持时间不一定相同，并且可能出现非稳定状态。
  ③ 在研究异步时序逻辑电路时，对输入信号的形式有所区分，无论输入信号是脉冲信号还是电平信号，对其变化过程均有一定约束。

- 根据输入信号形式，异步时序逻辑电路分为：
  ① 脉冲异步时序逻辑电路：其存储电路由触发器组成（可以是钟控触发器，也可以是非钟控触发器），电路输入信号为脉冲信号。
  ② 电平异步时序逻辑电路：其存储电路由延迟元件组成（可以是专用的延迟元件，或者利用电路本身固有的延迟），通过延迟加反馈实现记忆功能，电路输入信号为电平信号。

- 脉冲异步时序逻辑电路的输入脉冲信号（即输入信号x）必须满足如下的约束条件：
  ① 输入脉冲的宽度，必须保证触发器可靠翻转。
  ② 输入脉冲的间隔，必须保证前一个脉冲引起的电路响应完全结束后，后一个脉冲才能到来。
  ③ 不允许在两个或两个以上输入端同时出现脉冲（这可能导致电路产生错误的状态转移）。

- Mealy型脉冲异步时序逻辑电路的输出通常是脉冲信号（因为输出是输入和状态的函数，而输入是脉冲信号，因此输出也会是脉冲信号）。

- Moore型脉冲异步时序逻辑电路的输出是电平信号（因为输出仅仅是状态的函数，因此输出是电平信号）。

- 脉冲异步时序逻辑电路的分析过程与同步时序逻辑电路（第5章）基本相同，分析步骤如下：
  ① 根据电路，写出输出函数和激励函数表达式。
  ② 根据激励函数以及触发器的功能表，列出电路次态真值表（或次态方程组）。
  ③ 根据次态真值表（或次态方程组）以及输出函数，作出状态表和状态图。
  ④ 拟定一典型输入序列，画出时间图，并用文字描述电路的逻辑功能。

- 脉冲异步时序逻辑电路分析与同步时序逻辑电路分析的区别：
  ① 当存储元件采用钟控触发器时，应将触发器的时钟控制端作为激励函数处理。
  ② 由于不允许两个或两个以上输入端同时出现脉冲，加之输入端无脉冲出现时电路状态不会发生变化。因此，分析时可以排除这些情况，从而使分析过程中使用的图、表简化。

- 脉冲异步时序逻辑电路的设计过程与同步时序逻辑电路（第5章）基本相同，设计步骤如下：
  ① 根据要求形成原始状态图和原始状态表。
  ② 对原始状态表进行化简，得到最简状态表（最小状态表）。
  ③ 对最简状态表的状态编码，得到二进制状态表。
  ④ 确定触发器的类型和数目。
  ⑤ 根据二进制状态表和触发器的激励表，确定激励函数和输出函数。
  ⑥ 根据激励函数和输出函数，画出电路图。

- 脉冲异步时序逻辑电路设计与同步时序逻辑电路设计不同的地方：
  ① 由于不允许两个或两个以上输入端同时为1，所以在形成原始状态图和原始状态表时，若有多个输入信号，则只需考虑多个输入信号中仅一个为1的情况，从而使问题的描述得以简化。此外，在确定激励函数和输出函数时，可将两个或两个以上输入同时为1的情况，作为无关条件处理，这将有利于函数的简化。
  ② 由于电路中没有统一的时钟脉冲，因此当存储电路采用钟控触发器时，触发器的时钟端是作为激励函数处理的，因此需要对触发器的激励表做适当的修改（表6.5至6.8）。在要求触发器状态保持不变时（0->0，或1->1），通常采用CP=0、输入端=d的方式进行处理。

- 电平异步时序逻辑电路的结构模型：
  - 包括组合电路和存储电路，存储电路由反馈回路中的延迟元件构成，延迟元件一般不用专门插入延迟线，而是利用组合电路本身固有的分布延迟在反馈回路中的“集总”。

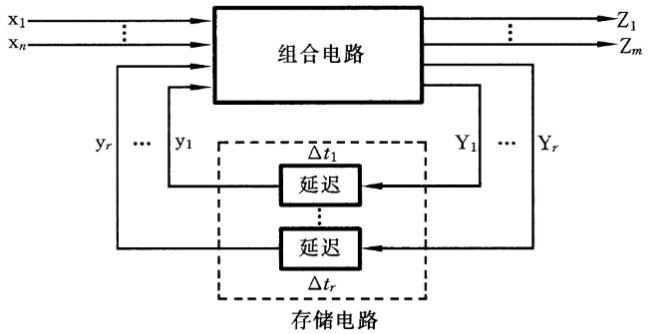


图 6.14　电平异步时序逻辑电路的结构模型

- 电平异步时序逻辑电路的分析步骤：
  ① 根据电路写出输出函数和激励函数表达式。
  ② 作出流程表。
  ③ 作出总态图或时间图。
  ④ 说明电路逻辑功能。

- 电平异步时序逻辑电路的竞争（类似组合逻辑电路中的险象）：
  - 实际电路中，各反馈回路之间的延迟是不相同的，这样可能会导致竞争。
  - 所谓竞争是指当输入信号变化引起电路中两个或两个以上状态变量发生变化时，由于各反馈回路延迟时间的不同，使状态变量的变化有先有后，而导致不同状态响应过程的现象。
  - 若竞争的结果最终能到达预定的稳态，则称为非临界竞争（状态转移可预测）。若竞争的结果使电路到达不同的稳态，即状态转移不可预测，则称为临界竞争（状态转移不可预测）。

- 电平异步时序逻辑电路的设计步骤：
  ① 根据设计要求，建立原始流程表。
  ② 化简原始流程表，得到最简流程表。
  ③ 状态编码，得到二进制流程表。
  ④ 根据二进制流程表，求出激励状态和输出函数表达式。
  ⑤ 画出逻辑电路图。

- 建立原始流程表。具体过程如下：
  - （1）画出典型输入/输出时间图，并设立相应状态，需要注意以下3点：
    ① 符合题意，即正确体现设计要求。
    ② 满足电平异步时序逻辑电路不允许两个或两个以上输入信号同时改变的约束条件。
    ③ 尽可能反映输入信号在各种取值下允许发生的变化。
  - （2）建立原始流程表，具体包括：
    ① 画出原始流程表，并填入稳定状态和相应输出。
    ② 填入非稳定状态并指定非稳定状态下的输出，完善流程表。
    ③ 填入无关状态和无关输出。

- 化简原始流程表。原始流程表的化简过程与不完全确定状态表（见第5章的5.3.3小节）的化简过程类似，其一般步骤如下：
  ① 作隐含表，找出相容行。
  ② 作合并图，求出最大相容行类。
  ③ 从相容行类中选择一个最小闭覆盖。
  ④ 作出最简流程表。

- 状态编码。为了避免反馈回路之间的临界竞争，保证电路可靠地实现预定功能。通常有3种状态编码方法。
  ① 相邻状态，相邻分配。
  ② 增加过渡状态，实现相邻分配。
  ③ 允许非临界竞争，避免临界竞争。

- 例6.1：分析图6.2所示脉冲异步时序逻辑电路，指出该电路的功能。

- 解：
  - 该电路属于Mealy型脉冲异步时序逻辑电路。

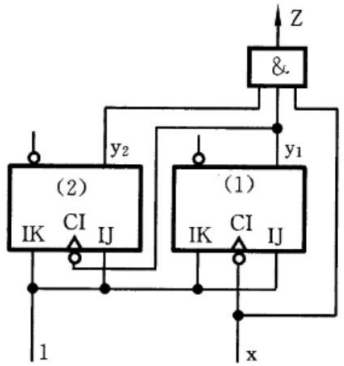


图 6.2　逻辑电路

  - （1）根据电路图，写出输出函数和激励函数表达式：
    - $Z=x\cdot y_2\cdot y_1$
    - $J_2=K_2=1$　　$C_2=y_1$
    - $J_1=K_1=1$　　$C_1=x$

  - （2）根据激励函数和J-K触发器的功能表，列出次态真值表（表6.1）：
    - 因为电路中的J-K触发器为下降沿触发，x=0时输入端无脉冲出现，因此表6.1中不需要给出x=0的情况（只有4种情况）。
    - 表6.1中 ↓ 表示触发器的时钟端出现下降沿。

表 3.23　钟控 J-K 触发器功能表

J	K	Q^{n+1}	功能说明
0	0	Q	不变
0	1	0	置 0
1	0	1	置 1
1	1	$\overline{Q}$	翻转

表 6.1　次态真值表

输入 x	现态 y_2	现态 y_1	J_2	K_2	C_2	J_1	K_1	C_1	次态 y_2^{n+1}	次态 y_1^{n+1}
1	0	0	1	1		1	1	↓	0	1
1	0	1	1	1	↓	1	1	↓	1	0
1	1	0	1	1		1	1	↓	1	1
1	1	1	1	1	↓	1	1	↓	0	0

前3列直接给出　　中间6列由激励函数得到　　最后2列由触发器的功能表得到

– （3）根据次态真值表和输出函数，作出状态表（表6.2），根据状态表画出状态图（图6.3）。

表 6.1　次态真值表

输入	现态		激励函数						次态	
x	y_2	y_1	J_2	K_2	C_2	J_1	K_1	C_1	y_2^{n+1}	y_1^{n+1}
1	0	0	1	1		1	1	↓	0	1
1	0	1	1	1	↓	1	1	↓	1	0
1	1	0	1	1		1	1	↓	1	1
1	1	1	1	1	↓	1	1	↓	0	0

表 6.2　状态表

现态		次态 $y_2^{n+1}y_1^{n+1}$/输出 Z
y_2	y_1	x=1
0	0	0 1 / 0
0	1	1 0 / 0
1	0	1 1 / 0
1	1	0 0 / 1

状态表是次态真值表的简化版

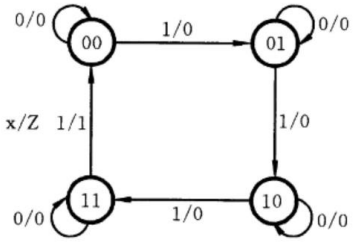


图 6.3　状态图

– （4）画出在某种典型输入信号下的时间图（图6.4）。根据时间图可知，当收到第4个输入脉冲时，电路产生一个进位输出脉冲。因此，该电路是一个“模4加1计数器”。

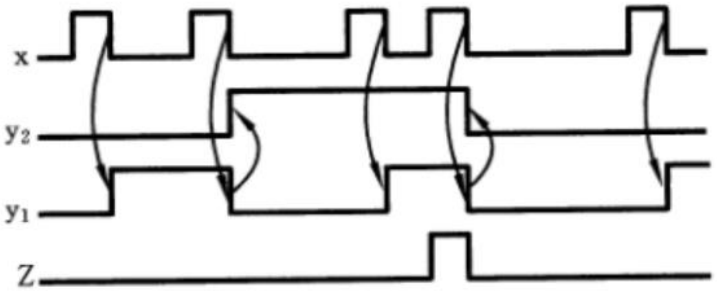


图 6.4　时间图

- 例6.2：分析图6.5所示的脉冲异步时序逻辑电路。

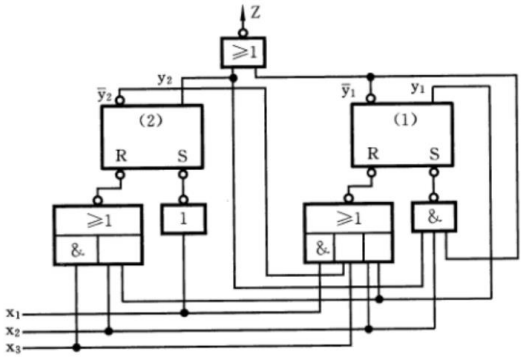


图 6.5 逻辑电路

- 解：
  - 该电路属于Moore型脉冲异步时序逻辑电路。
  - （1）根据电路图，写出输出函数和激励函数表达式：
    - $Z=/(y_2+/y_1)=/y_2 \cdot y_1$
    - $R_2=/(x_3+x_2 \cdot y_1)$　　$S_2=/x_1$
    - $R_1=/(x_1+x_3 \cdot /y_2+x_2 \cdot y_1)$　　$S_1=/(x_2 \cdot y_2 \cdot /y_1)$
  - （2）根据激励函数和与非门构成的基本R-S触发器功能表（表3.13），列出次态真值表（表6.3）：
    - 表6.3中只需要列出输入分别为1的情况（$x_1x_2x_3$=100、010、001），每种情况又有4种现态变化的情况（y_2y_1=00、01、10、11），因此共12种情况。

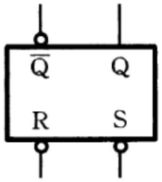


表 3.13 与非门构成的基本 R-S 触发器功能表

R	S	Q^{n+1}	功能说明
0	0	d	不定
0	1	0	置 0
1	0	1	置 1
1	1	Q	不变

表 6.3 次态真值表

输入			现态		激励函数				次态	
x_1	x_2	x_3	y_2	y_1	R_2	S_2	R_1	S_1	y_2^{n+1}	y_1^{n+1}
1	0	0	0	0	1	0	0	1	1	0
1	0	0	0	1	1	0	0	1	1	0
1	0	0	1	0	1	0	0	1	1	0
1	0	0	1	1	1	0	0	1	1	0
0	1	0	0	0	1	1	1	1	0	0
0	1	0	0	1	0	1	0	1	0	0
0	1	0	1	0	1	1	1	0	1	1
0	1	0	1	1	0	1	0	1	0	0
0	0	1	0	0	0	1	0	1	0	0
0	0	1	0	1	0	1	0	1	0	0
0	0	1	1	0	0	1	1	1	0	0
0	0	1	1	1	0	1	1	1	0	1

– （3）根据次态真值表和输出函数，作出状态表（表6.4），根据状态表画出状态图（图6.6）。

表 6.3　次态真值表

输入			现态		激励函数				次态	
x_1	x_2	x_3	y_2	y_1	R_2	S_2	R_1	S_1	y_2^{n+1}	y_1^{n+1}
1	0	0	0	0	1	0	0	1	1	0
1	0	0	0	1	1	0	0	1	1	0
1	0	0	1	0	1	0	0	1	1	0
1	0	0	1	1	1	0	0	1	1	0
0	1	0	0	0	1	1	1	1	0	0
0	1	0	0	1	0	1	0	1	0	0
0	1	0	1	0	1	1	1	0	1	1
0	1	0	1	1	0	1	0	1	0	0
0	0	1	0	0	0	1	0	1	0	0
0	0	1	0	1	0	1	0	1	0	0
0	0	1	1	0	0	1	1	1	0	0
0	0	1	1	1	0	1	1	1	0	1

表 6.4　状态表

现态		次态 $y_2^{n+1}y_1^{n+1}$			输出
y_2	y_1	x_1	x_2	x_3	Z
0	0	10	00	00	0
0	1	10	00	00	1
1	0	10	11	00	0
1	1	10	00	01	0

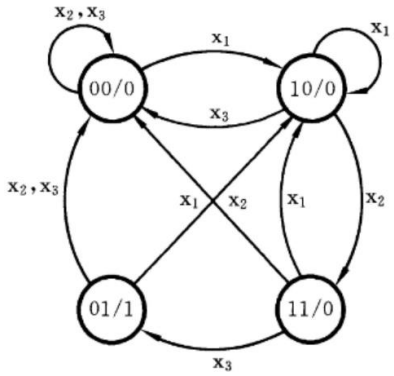


图 6.6　状态图

– （4）画出在某种典型输入信号下的时间图（图6.7）。根据时间图可知，当输入端按照x_1、x_2、x_3的顺序依次出现脉冲时，将产生一个“1”输出信号，其它情况输出为“0”。因此，该电路是一个“x_1 - x_2 - x_3”序列检测器。

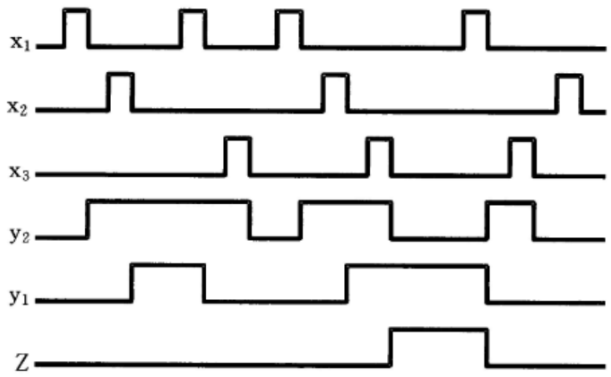


图 6.7　时间图

- **例6.3**：用D触发器作为存储元件，设计一个“$x_1 - x_2 - x_2$”序列检测器。该电路有两个输入（x_1、x_2），一个输出Z。仅当x_1输入一个脉冲后，x_2连续输入两个脉冲时，输出端Z由0变为1，该1信号将一直维持到输入端x_1或x_2再出现脉冲时，才由1变为0。其输入/输出时间图如图6.8所示。

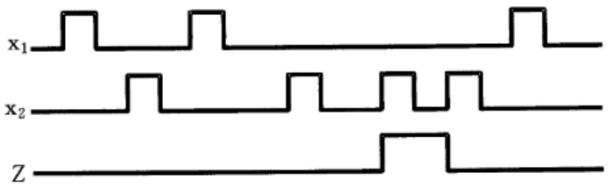


图 6.8　时间图

- 解：
  - 该电路属于Moore型脉冲[illegible]序逻辑电[illegible]
  - （1）作出状态图（图6.9）和[illegible]表（[illegible]）。

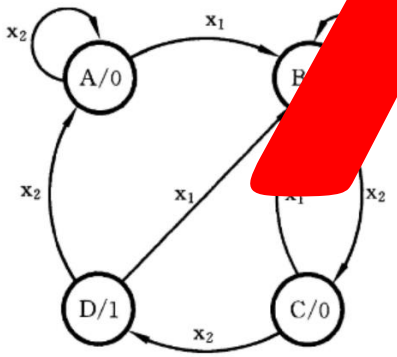


图 6.9　状态图

表 6.9　状态表

现　态	次　态		输　出
	x_1	x_2	Z
A	B	A	0
B	B	C	0
C	B	D	0
D	B	A	1

– （2）状态化简。用隐含表法检查表6.9，发现该状态表中的所有状态均不等效，即表6.9已经是最简状态表（最小状态表）。

– （3）状态编码。根据相邻编码法的原则，可以得到表6.10的编码方案，从而得到二进制状态表（表6.11）。

表 6.9　状态表

现　态	次　态		输　出
	x_1	x_2	Z
A	B	A	0
B	B	C	0
C	B	D	0
D	B	A	1

表 6.10　编码方案

状　态		
A		0
		0
	0	1
	1	1

表 6.11　二进制状态表

	现　态		次态 $y_2^{n+1}y_1^{n+1}$		输　出
	y_2	y_1	$x_1=1$	$x_2=1$	Z
A	0	0	10	00	0
C	0	1	10	11	0
B	1	0	10	01	0
D	1	1	10	00	1

– （4）确定输出函数和激励[illegible]采用2[illegible]发器作为存储单元，根据二进制状态表（表6.11）和D触发器的激励表（表6[illegible]态不变时[illegible]方法二：CP=0，D=d），得到激励函数和输出函数真值表（表6.12，输入只需要考虑两种情况：01、10）。

表 6.5　D 触发器激励表

Q	Q^{n+1}	CP	D
0	0	d	0
		0	d
0	1	1	1
1	0	1	0
1	1	d	1
		0	d

表 6.12　激励函数和输出函数真值表

输　入		现　态		激励函数				输出函数
x_2	x_1	y_2	y_1	C_2	D_2	C_1	D_1	Z
0	1	0	0	1	1	0	d	0
		0	1	1	1	1	0	0
		1	0	0	d	0	d	0
		1	1	0	d	1	0	1
1	0	0	0	0	d	0	d	0
		0	1	1	1	0	d	0
		1	0	1	0	1	1	0
		1	1	1	0	1	0	1

– 由真值表（表6.12）画出卡诺图（图6.10，当输入x_2x_1=00时，置C_2=d、C_1=d、D_2=d、D_1=d、Z=d；当输入x_2x_1=11时，置C_2=d、C_1=d、D_2=d、D_1=d、Z=d）。

表 6.12　激励函数和输出函数真值表

输入		现态		激励函数				输出函数
x_2	x_1	y_2	y_1	C_2	D_2	C_1	D_1	
0	1	0	0	1	1	0	d	
		0	1	1	1	1	0	
		1	0	0	d	0	d	
		1	1	0	d	1	0	1
1	0	0	0	0	d	0	d	0
		0	1	1	1	0	d	0
		1	0	1	0	1	1	0
		1	1	1	0	1	0	1

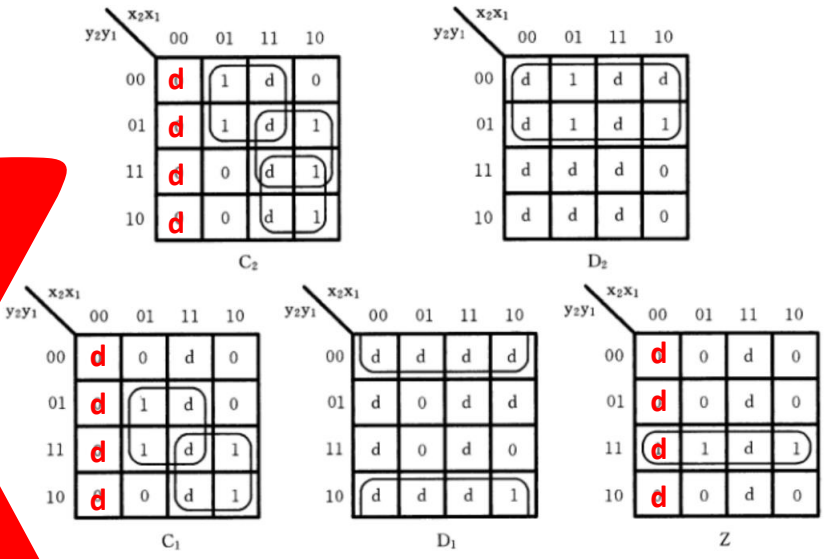


图 6.10　卡诺图

– 由卡诺图，得到激励函数和输出函数：

- $C_2=x_1\cdot\overline{y}_2+x_2\cdot y_1+x_2\cdot y_2$　　$D_2=\overline{y}_2$
- $C_1=x_1\cdot y_1+x_2\cdot y_2$　　$D_1=\overline{y}_1$
- $Z=y_2\cdot y_1$

– （5）由激励函数和输出函数，画出电路图（图6.11）。

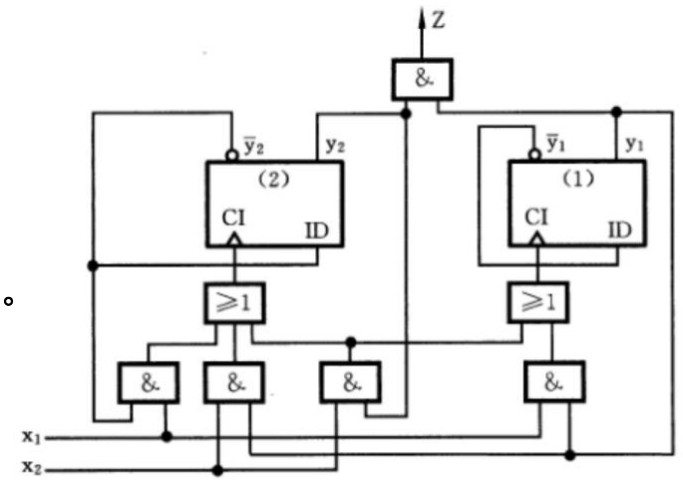


图 6.11　逻辑电路

- **例6.4**：用T触发器作为存储元件，设计一个异步模8加1计数器。该电路对输入端x出现的脉冲进行计数，当收到第8个脉冲时，输出端Z产生一个进位输出脉冲。

- 解：
  - 该电路属于Mealy型脉冲异步时序逻辑电路。
  - （1）作出状态图（图6.12）、二进制状态表（表6.13）。

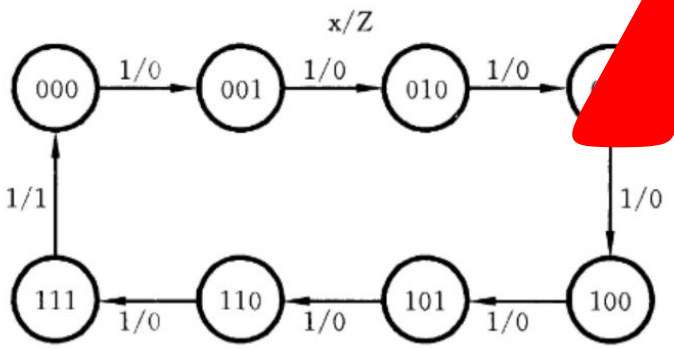


图 6.12　状态图

表 6.13　二进制状态表

现态 $y_3\ y_2\ y_1$	次态 $y_3^{n+1}\ y_2^{n+1}\ y_1^{n+1}$/输出 Z
	x=1
0 0 0	0 0 1 / 0
0 0 1	0 1 0 / 0
0 1 0	0 1 1 / 0
0 1 1	1 0 0 / 0
1 0 0	1 0 1 / 0
1 0 1	1 1 0 / 0
1 1 0	1 1 1 / 0
1 1 1	0 0 0 / 1

– （2）确定输出函数和激励函数。

- 采用3个T触发器（下降沿触发）作为存储单元，根据二进制状态表（表6.13）和T触发器的激励表（表6.7，状态不变时采用方法二：CP=0，T=d），得到状态转移关系及激励函数和输出函数真值表（表6.14，输入脉冲只需要考虑x=1的情况，激励函数中空白处表示CP=0，用下降沿符号 ↓ 表示CP=1）。

- 由表6.14可知，激励函数C_3可由y_2产生，激励函数C_2可由y_1产生，激励函数C_1可由x产生，并且T_3、T_2、T_1均可设为1。因此有：
  - $C_3=y_2$　　$C_1=x$　　$T_2=T_1=1$　　$Z=x\cdot y_3\cdot y_2\cdot y_1$

表 6.13　二进制状态表

现态 $y_3\ y_2\ y_1$	次态 $y_3^{n+1}\ y_2^{n+1}\ y_1^{n+1}$/输出 Z
	x=1
0 0 0	0 0 1 / 0
0 0 1	0 1 0 / 0
0 1 0	0 1 1 / 0
0 1 1	1 0 0 / 0
1 0 0	1 0 1 / 0
1 0 1	1 1 0 / 0
1 1 0	1 1 1 / 0
1 1 1	0 0 0 / 1

表 6.7　T 触发器激励表

Q	Q^{n+1}	CP	T
0	0	d	0
		0	d
0	1	1	1
1	0	1	1
1	1	d	0
		0	d

表 6.14　状态转移关系及激励函数、输出函数真值表

输入脉冲	现态	次态	状态跳变			激励函数						输出
x	$y_3\ y_2\ y_1$	$y_3^{n+1}\ y_2^{n+1}\ y_1^{n+1}$	y_3	y_2	y_1	C_3	T_3	C_2	T_2	C_1	T_1	Z
1(↓)	0 0 0	0 0 1			↑		d		d	↓	1	0
1(↓)	0 0 1	0 1 0		↑	↓		d	↓	1	↓	1	0
1(↓)	0 1 0	0 1 1			↑		d		d	↓	1	0
1(↓)	0 1 1	1 0 0	↑	↓	↓	↓	1	↓	1	↓	1	0
1(↓)	1 0 0	1 0 1			↑		d		d	↓	1	0
1(↓)	1 0 1	1 1 0		↑	↓		d	↓	1	↓	1	0
1(↓)	1 1 0	1 1 1			↑		d		d	↓	1	0
1(↓)	1 1 1	0 0 0	↓	↓	↓	↓	1	↓	1	↓	1	1

– （3）由激励函数和输出函数，画出电路图。

- $C_3=y_2$　　$C_2=y_1$　　$C_1=x$　　$T_3=T_2=T_1=1$　　$Z=x\cdot y_3\cdot y_2\cdot y_1$

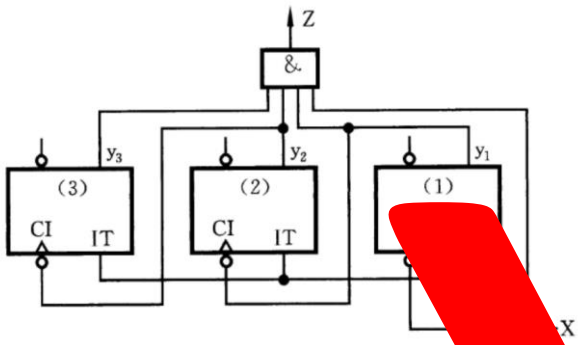


图 6.13　逻辑电路

表 6.14　状态转移关系及激励函数、输出函数真值表

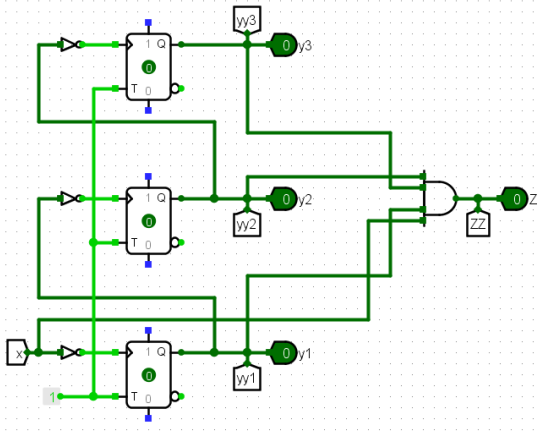
输入脉冲	现态	次态	状态跳变			激励函数						输出
x	$y_3\ y_2\ y_1$	$y_3^{n+1}\ y_2^{n+1}\ y_1^{n+1}$	y_3	y_2	y_1	C_3	T_3	C_2	T_2	C_1	T_1	Z
1(↑↓)	0 0 0	0 0 1			↑		d		d	↓	1	0
1(↑↓)	0 0 1	0 1 0		↑	↓		d	↓	1	↓	1	0
1(↑↓)	0 1 0	0 1 1			↑		d		d	↓	1	0
1(↑↓)	0 1 1	1 0 0	↑	↓	↓	↓	1	↓	1	↓	1	0
1(↑↓)	1 0 0	1 0 1			↑		d		d	↓	1	0
1(↑↓)	1 0 1	1 1 0		↑	↓		d	↓	1	↓	1	0
1(↑↓)	1 1 0	1 1 1			↑		d		d	↓	1	0
1(↑↓)	1 1 1	0 0 0	↓	↓	↓	↓	1	↓	1	↓	1	1

– 假如采用上升沿触发的T触发器[illegible]件，则表6.14的真值表修改为下表，激励函数修改为（输出函数不变）：

- $C_3=/y_2$　　$C_2=/y_1$　　$C_1=/$[illegible]　　$T_3=T_2=$[illegible]　　$Z=x\cdot y_3\cdot y_2\cdot y_1$

表 6.14　状态转移关系及激励函数、输出函数真值表

输入脉冲	现态	次态	状态跳变			激励函数						输出
x	$y_3\ y_2\ y_1$	$y_3^{n+1}\ y_2^{n+1}\ y_1^{n+1}$	y_3	y_2	y_1	C_3	T_3	C_2	T_2	C_1	T_1	Z
1(↑↓)	0 0 0	0 0 1			↑		d		d	↑	1	0
1(↑↓)	0 0 1	0 1 0		↑	↓		d	↑	1	↑	1	0
1(↑↓)	0 1 0	0 1 1			↑		d		d	↑	1	0
1(↑↓)	0 1 1	1 0 0	↑	↓	↓	↑	1	↑	1	↑	1	0
1(↑↓)	1 0 0	1 0 1			↑		d		d	↑	1	0
1(↑↓)	1 0 1	1 1 0		↑	↓		d	↑	1	↑	1	0
1(↑↓)	1 1 0	1 1 1			↑		d		d	↑	1	0
1(↑↓)	1 1 1	0 0 0	↓	↓	↓	↑	1	↑	1	↑	1	1



- 例6.5：分析图6.17所示电平异步时序逻辑电路。
- 解：
  - 该电路属于Moore型（输出只与状态有关）。

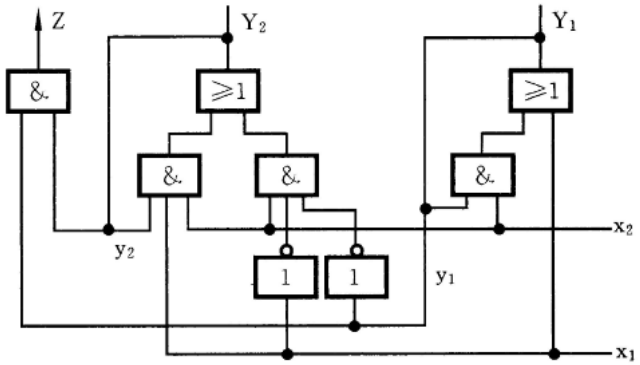


图 6.17　逻辑电路

– （1）根据电路写出输出函数和激励函数：
  - $Z=y_2 \cdot y_1$
  - $Y_2=x_2 \cdot x_1 \cdot y_2 + x_2 \cdot /x_1 \cdot /y_1$　　$Y_1=x_2 \cdot y_1 + x_1$

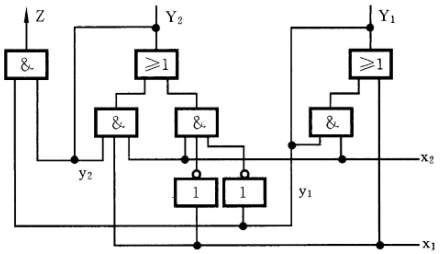


图 6.17　逻辑电路

– （2）根据激励函数和输出函数作出流程表（表6.18）：
  - 表6.18中画圆圈的为稳定总态，共6个。

– （3）根据流程表作总态图（图6.18）：
  - 图6.18中有6个稳定总态。

表 6.18　流程表

二次状态		激励状态 Y_2Y_1				输出
y_2	y_1	$x_2x_1=00$	$x_2x_1=01$	$x_2x_1=11$	$x_2x_1=10$	Z
0	0	(00)	01	01	10	0
0	1	00	(01)	(01)	(01)	0
1	1	00	01	(11)	01	1
1	0	00	01	11	(10)	0

这1列直接给出

这4列根据激励函数得到

这1列根据输出函数得到

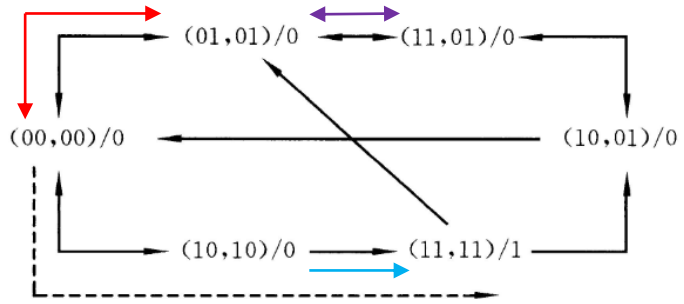


图 6.18　总态图

(00,00)/0：第1个00表示输入 x_2x_1，第2个00表示二次状态 y_2y_1，第3个0表示输出Z

- 假设电路的初始总态为(x_2x_1,y_2y_1)=(00,00)，输入的变化序列为00->10->11->01->00->01->11->10。

- 根据流程表作出总态和输出响应序列：
  - 其中加*的表示是非稳定总态。

表 6.18　流程表

二次状态		激励状态 Y_2Y_1				输出
y_2	y_1	$x_2x_1=00$	$x_2x_1=01$	$x_2x_1=11$	$x_2x_1=10$	Z
0	0	(00)	01	01	10	0
0	1	00	(01)	(01)	(01)	0
1	1	00	01	(11)	11	1
1	0	00	01	11	(10)	0

时刻 t_i	t_0	t_1	t_2	t_3	t_4	t_5	t_6	t_7
输入 x_2x_1	00	10	11	01	00	01	11	10
总态(x_2x_1,y_2y_1)	(00,00)	(10,00)* (10,10)	(11,10)* (11,11)	(01,11)* (01,11)	(00,01)* (00,00)	(01,00)* (01,01)	(11,01)	(10,01)
输出 Z	0	0	1	0	0	0	0	0

- 对应的时间图（图6.19）：

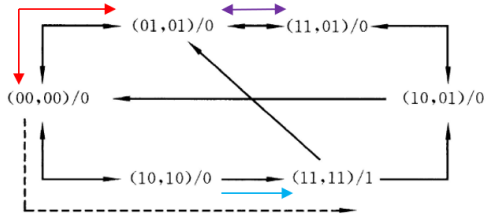


图 6.18　总态图

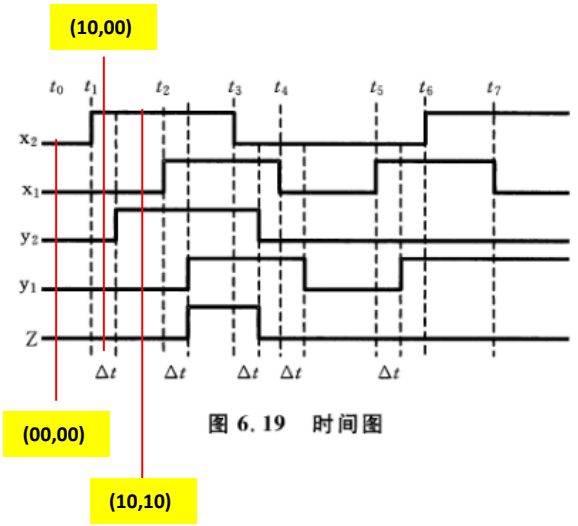


图 6.19　时间图

- （4）根据总态图和时间图可知，仅当电路收到输入序列“00->10->11”时，才产生一个高电平输出信号，其他情况均输出低电平。因此该电路是一个“00->10->11”序列检测器。

- 例6.6：某电平异步时序逻辑电路有两个输入端x_1和x_2，一个输出端Z。输出与输入之间的关系为：只要x_1x_2=00，则Z=0，在此之后当x_1x_2=01或10时，Z=1；只要x_1x_2=11，则Z=1，在此之后当x_1x_2=01或10时，Z=0。作出该电路的原始流程表。

- 解：
  - （1）画出典型输入/输[illegible]间图（图[illegible]），并设立相应状态，共有6个稳定状态：
    - 稳定状态①：x_1x_2=[illegible]0
    - 稳定状态②：x_1x_2=1[illegible]
    - 稳定状态③：x_1x_2=01，[illegible]
    - 稳定状态④：x_1x_2=11，[illegible]
    - 稳定状态⑤：x_1x_2=10，[illegible]
    - 稳定状态⑥：x_1x_2=01[illegible]

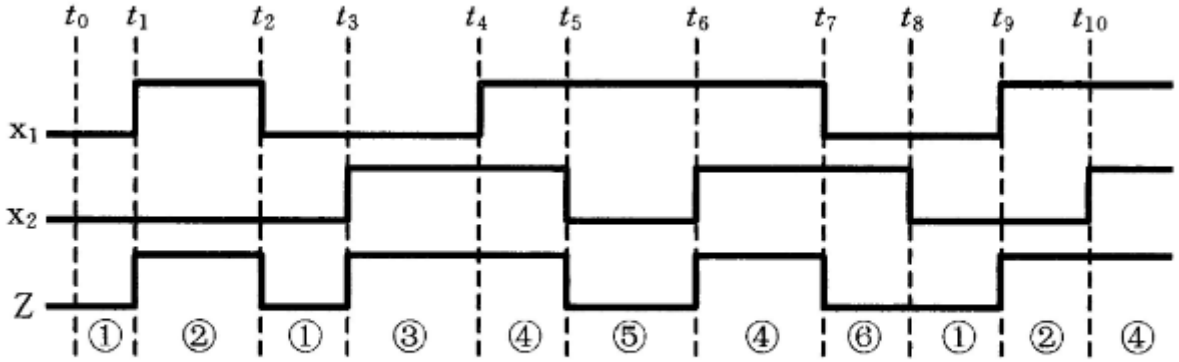


图 6.21　典型输入/输出时间图

– （2）建立原始流程表。

- 首先，画出含有6个稳定状态的部分流程表(I)（表6.20），6个稳定状态如下：
  - 稳定状态①：x_1x_2=00，Z=0
  - 稳定状态②：x_1x_2=10，Z=1
  - 稳定状态③：x_1x_2=01，Z=1
  - 稳定状态④：x_1[illegible]Z=1
  - 稳定状态⑤：x_1x_2[illegible]0
  - 稳定状态⑥：x_1x_2=0[illegible]

表[illegible]部[illegible]表(Ⅰ)

二次状态 y	激励状态[illegible]出 Z			
	$x_1x_2=00$	$x_1x_2=01$	$x_1x_2=11$	$x_1x_2=10$
1	①/0			
2				②/1
3		③/1		
4			④/1	
5				⑤/0
6		⑥/0		

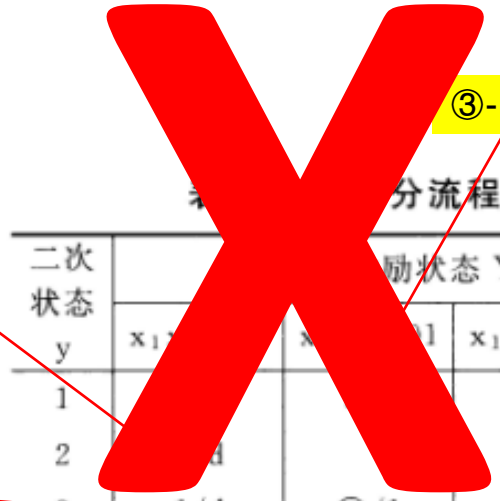
- 然后，填入非稳定状态，并指定非稳定状态下的输出，完善流程表，得到部分流程表(II)（表6.21）：

①->②

输出=d是因为状态①的输出为0、状态②的输出为1，发生了变化，状态转换过程中，输出是不定的

①->⑥

输出=0是因为状态①的输出是0、状态⑥的输出也是0，状态转换过程中，输出没有变化



分流程表(Ⅱ)

二次状态 y	激励状态 Y/输出 Z			
	$x_1x_2=01$	$x_1x_2=00$	$x_1x_2=11$	$x_1x_2=10$
1				2/d
2		d	4/1	②/1
3	1/d	③/1	4/1	
4		6/d	④/1	5/d
5	1/0		4/d	⑤/0
6	1/0	⑥/0	4/d	

③->①

②->①

④->②

①->③

④->③

①->⑤

⑤->④

④->⑤

⑥->④

④->⑥

- 最后，填入无关状态和无关输出（d/d），即可得到完整流程表（表6.22）。

22 完 表

二次状态 y	次态 Y/输出 Z			
	$x_1x_2=00$		$x_1x_2=11$	$x_1x_2=10$
1	①/0		→ d/d	2/d
2	1/d		4/1	②/1
3	1/d	③	4/1	→ d/d
4	→ d/d	6/d	④/1	5/d
5	1	→ d/d	4/d	⑤/0
6	1/0	⑥/0	4/d	→ d/d

- 例6.7：化简表6.22所示的原始流程表。

- 解：

① 根据完整流程表（表6.22）作隐含表（表6.22），从隐含表中找出相容行对：(1,2)、(1,3)、(2,3)、(2,6)、(3,5)、(4,5)、(4,6)、(5,6)。

② 根据相容行对作合并图（图6.23），从合并图中找出最大相容行类：(1,2,3)、(4,5,6)、(3,5)、(2,6)。

③ 从最大相容行类中选择最小[illegible]：(1[illegible](4,5,6)。

④ 将最小闭覆盖(1,2,3)、(4,5,6)分[illegible]表，即可得到最简流程表（表6.23）。

表 6.22　完整流程表

二次状态 y	激励状态 Y/输出 Z			
	$x_1x_2=00$	$x_1x_2=01$	$x_1x_2=11$	$x_1x_2=10$
1	①/0	3/d	d/d	2/d
2	1/d	d/d	4/1	②/1
3	1/d	③/1	4/1	d/d
4	d/d	6/d	④/1	5/d
5	1/0	d/d	4/d	⑤/0
6	1/0	⑥/0	4/d	d/d

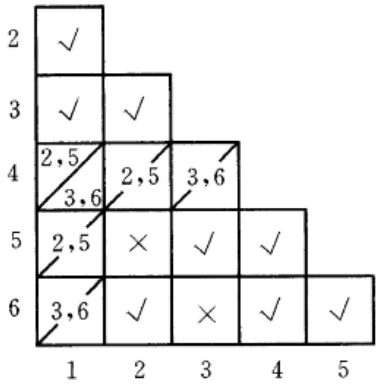


图 6.22　隐含表

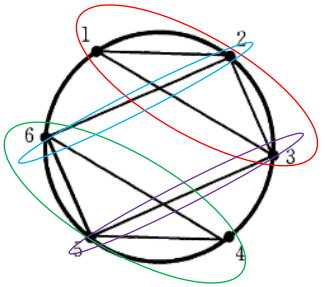


图 6.23　合并图

表 6.23　最简流程表

二次状态 y	激励状态 Y/输出 Z			
	$x_1x_2=00$	$x_1x_2=01$	$x_1x_2=11$	$x_1x_2=10$
A	Ⓐ/0	Ⓐ/1	B/1	Ⓐ/1
B	A/0	Ⓑ/0	Ⓑ/1	Ⓑ/0

- 例6.8：对表6.24所示流程表进行状态编码，作出二进制流程表。
- 解：
  - 首先，根据“相邻状态，相邻分配”的原则，作相邻状态图（图6.24）。
  - 根据状态相邻图，可以得到状态分配方案（图6.25），A、B、C、D分别用00、01、10、11表示。
  - 根据状态分配方案，得到[illegible]表（表6.25）。

表 6.24　流程表

二次状态 y	激励状态 Y			
	$x_2x_1=00$	$x_2x_1=01$	$x_2x_1=11$	$x_2x_1=10$
A	Ⓐ	Ⓐ	B	C
B	A	Ⓑ	Ⓑ	Ⓑ
C	Ⓒ	A	D	Ⓒ
D	C	Ⓓ	Ⓓ	Ⓓ

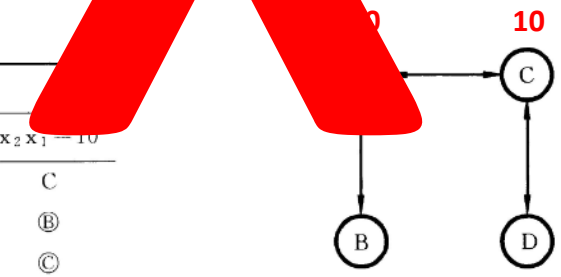


图 6.24　状态相邻图

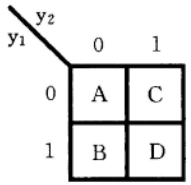


图 6.25　状态分配方案

表 6.25　二进制流程表

二次状态 y_2 y_1	激励状态 Y_2Y_1			
	$x_2x_1=00$	$x_2x_1=01$	$x_2x_1=11$	$x_2x_1=10$
A 0 0	⑳	⑳	01	10
B 0 1	00	㉑	㉑	㉑
D 1 1	10	㉛	㉛	㉛
C 1 0	⑩	00	11	⑩

- 例6.9：对表6.26所示流程表进行状态编码，得到二进制流程表。

- 解：
  - 根据流程表，画出相邻状态图（图6.26）。因为状态之间的相邻关系出现了由奇数（3）个状态构成的[illegible]此时需[illegible]状态A与状态C之间增加一个过渡状态D。增加过渡状态后的流程[illegible]表6.27[illegible]增加过渡状态后的状态相邻图如图6.27所示。用00、01、11、10分[illegible]示A[illegible]C、D状态，则可得到二进制流程表（表6.28）。

表 6.26　流程表

二次状态 y	激励状态 Y			
	$x_2x_1=00$	$x_2x_1=01$	$x_2x_1=11$	$x_2x_1=10$
A	Ⓐ	B	C	Ⓐ
B	A	Ⓑ	Ⓑ	C
C	A	B	Ⓒ	Ⓒ

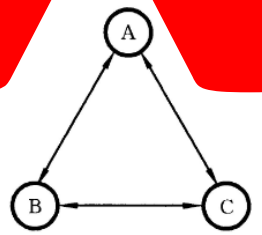


图 6.26　状态相邻图

表 6.27　增加过渡状态后的流程表

二次状态 y	激励状态 Y			
	$x_2x_1=00$	$x_2x_1=01$	$x_2x_1=11$	$x_2x_1=10$
A	Ⓐ	B	D	Ⓐ
B	A	Ⓑ	Ⓑ	C
C	D	B	Ⓒ	Ⓒ
D	A	d	C	d

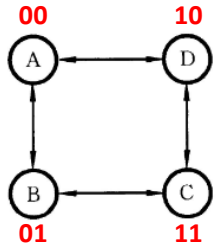
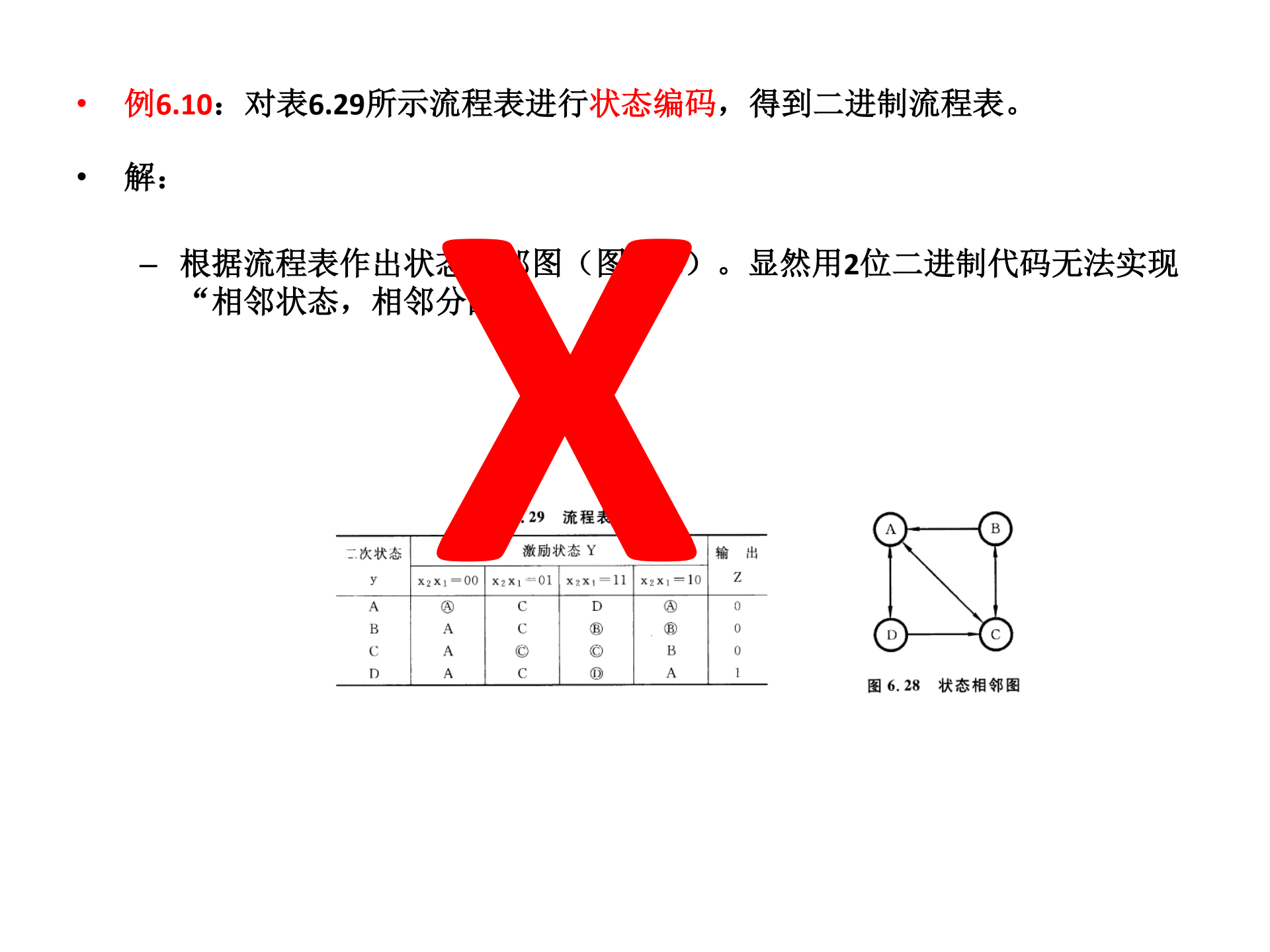


图 6.27　增加过渡状态后的状态相邻图

表 6.28　二进制流程表

二次状态 y_2 y_1	激励状态 Y_2Y_1			
	$x_2x_1=00$	$x_2x_1=01$	$x_2x_1=11$	$x_2x_1=10$
A 0 0	⓪⓪	01	10	⓪⓪
B 0 1	00	⓪①	⓪①	11
C 1 1	10	01	①①	①①
D 1 0	00	dd	11	dd

- 例6.10：对表6.29所示流程表进行状态编码，得到二进制流程表。
- 解：
  - 根据流程表作出状态相邻图（图6.28）。显然用2位二进制代码无法实现“相邻状态，相邻分配”。



. 29　流程表

二次状态	激励状态 Y				输　出
y	$x_2x_1=00$	$x_2x_1=01$	$x_2x_1=11$	$x_2x_1=10$	Z
A	Ⓐ	C	D	Ⓐ	0
B	A	C	Ⓑ	Ⓑ	0
C	A	Ⓒ	Ⓒ	B	0
D	A	C	Ⓓ	A	1

图 6.28　状态相邻图

- 可以通过增加过渡状态和增加代码位数实现相邻分配，但这样会导致增加电路的复杂性。
- 还可以采用“允许非临界竞争，避免临界竞争”的方法。
- 状态A和状态C之间的转移只发生在x_2x_1=00和01这两列，而这两列各只有一个稳定状态，这就意味着，A和C发生转移时，即使产生竞争，也属于非临界竞争。即在状态分配时，A和C可以相邻。
- 用00、01、11、10分别表示状态A、B、C、D，可以得到图6.30所示二进制流程表。

表 6.29　流程表

二次状态 y	激励状态 Y				输
	$x_2x_1=00$	$x_2x_1=01$	$x_2x_1=11$	$x_2x_1=10$	
A	Ⓐ	C	D	Ⓐ	
B	A	C	Ⓑ	Ⓑ	
C	A	Ⓒ	Ⓒ	B	0
D	A	C	Ⓓ	A	1

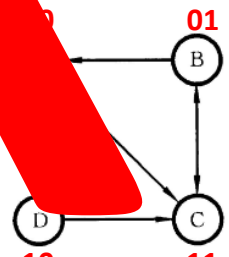


图 6.28　状态相邻图

表 6.30　二进制流程表

二次状态 y_2 y_1	激励状态 Y_2Y_1				输出 Z
	$x_2x_1=00$	$x_2x_1=01$	$x_2x_1=11$	$x_2x_1=10$	
A 0 0	⑳	11	10	⑳	0
B 0 1	00	11	⑪	⑪	0
C 1 1	00	⑪	⑪	01	0
D 1 0	00	11	⑩	00	1

- **例6.11**：用与非门设计一个单脉冲发生器，电路结构框图如图6.30所示。该电路有两个输入端x_2、x_1和一个输出端Z。x_2接时钟脉冲源，x_1接手动控制按钮。当不按下按钮时（x_1=0），x_2端的脉冲被封锁，输出Z为0，无脉冲输出；当按下按钮并释放（x_1由0->1，再由1->0）之后，输入端x_2出现的第一个完整脉冲被送至输出端Z，即用手启动一次，输出一个完整脉冲。电路规定每启动一次，必须在输出一个完整脉冲后才可再次启动。

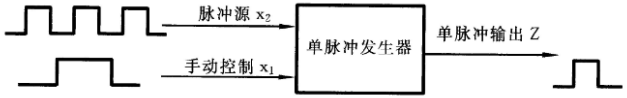


图 6.30　单脉冲发生器的结构框图

- 解：
  - （1）建立原始流程表。典型[illegible]输出[illegible]图如图6.31所示，根据图6.31可知有7个状态（稳态），根据时间图可以建立原[illegible]表6.31）。因为电路规定每启动一次，必须在输出一个完整脉冲后才可再次启动；[illegible]稳态⑤时输入取值不允许从10->11，处在稳态⑥时输入取值不允许从00->01，处在稳[illegible]入取值不允许从10->11。
    - 稳态①：x_2x_1=00（启[illegible]号均未出现），Z=0
    - 稳态②：x_2x_1=10（[illegible]出现[illegible]动信号），Z=0
    - 稳态③：x_2x_1=01（[illegible]信号但[illegible]信号），Z=0
    - 稳态④：x_2x_1=11（[illegible]信号和脉冲[illegible]时出现），Z=0
    - 稳态⑤：x_2x_1=10（x_2端有启动信号结束后的不完整脉冲），Z=0
    - 稳态⑥：x_2x_1=00（启动信号已结束，但第一个完整脉冲尚未出现），Z=0
    - 稳态⑦：x_2x_1=10（出现了启动信号结束后的第一个完整脉冲，产生一个完整输出脉冲），Z=1

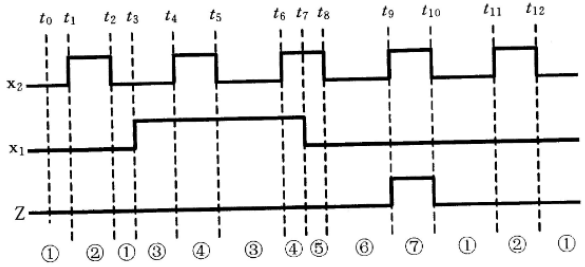


图 6.31　典型时间图

表 6.31　原始流程表

二次状态 y	激励状态 Y/输出 Z			
	x_2x_1=00	x_2x_1=01	x_2x_1=11	x_2x_1=10
1	①/0	3/0	d/d	2/0
2	1/0	d/d	4/0	②/0
3	6/0	③/0	4/0	d/d
4	d/d	3/0	④/0	5/0
5	6/0	d/d	d/d	⑤/0
6	⑥/0	d/d	d/d	7/d
7	1/d	d/d	d/d	⑦/1

– （2）化简原始流程表。

- 首先作出隐含表（图6.32(a)），由隐含表得到相容行对：(1,2)、(3,4)、(3,5)、(3,6)、(4,5)。
- 由相容行对，画状态合并图（图6.32(b)），得到最大相容行类：(1,2)、(3,4,5)、(3,6)、(7)。
- 从最大相容行类中选[illegible]小闭覆盖：[illegible]、(3,4,5)、(6)、(7)。
- 分别用A、B、C、D分别[illegible],2)[illegible],5)、(6)、(7)，即可得到最简流程表（表6.32）。

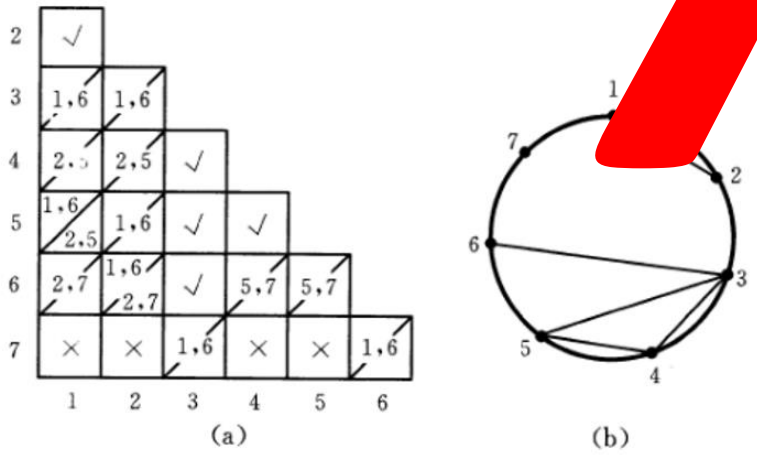


图 6.32　隐含表和状态合并图

表 6.32　最简流程表

次状态 y	激励状态 Y/输出 Z			
	$x_2x_1=00$	$x_2x_1=01$	$x_2x_1=11$	$x_2x_1=10$
A	Ⓐ/0	B/0	B/0	Ⓐ/0
B	C/0	Ⓑ/0	Ⓑ/0	Ⓑ/0
C	Ⓒ/0	d/d	d/d	D/d
D	A/d	d/d	d/d	Ⓓ/1

– （3）状态编码。根据最简流程表作出状态相邻图（图6.33），分别用00、01、11、10表示A、B、C、D，则得到二进制流程表（表6.33）。

– （4）确定激励状态和输出函数表达式。根据二进制流程表画出卡诺图（图6.34），根据卡诺图得到简化后的逻辑公式：

- $Y_2=x_2\cdot y_2+ /x_2\cdot/x_1\cdot y_1 =/(/(x_2\cdot y_2)\cdot/(/x_2\cdot/x_1\cdot y_1))$
- $Y_1=x_1+/x_2\cdot y_1 +/y_2\cdot y_1 =/(/x_1\cdot/(/(x_2\cdot y_2)\cdot y_1))$
- $Z=x_2\cdot y_2 =//(x_2\cdot y_2)$

– （5）根据逻辑公式，画出电[illegible]（图[illegible]。

表 6.32　最简流程表

二次状态 y	激励状态 Y/输出 Z			
	$x_2x_1=00$	$x_2x_1=01$	$x_2x_1=11$	$x_2x_1=10$
A	Ⓐ/0	B/0	B/0	Ⓐ/0
B	C/0	Ⓑ/0	Ⓑ/0	Ⓑ/0
C	Ⓒ/0	d/d	d/d	D/d
D	A/d	d/d	d/d	Ⓓ/1

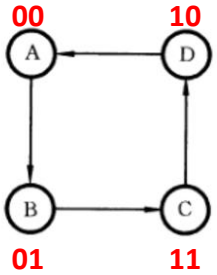


图 6.33　状态相邻图

表 6.33　二进制流程表

状态	激励状态 Y_2Y_1/输出 Z			
	$x_2x_1=00$	$x_2x_1=01$	$x_2x_1=11$	$x_2x_1=10$
A	(00)/0	01/0	01/0	(00)/0
B	11/0	(01)/0	(01)/0	(01)/0
C	(11)/0	d/d	d/d	10/d
D	00/d	d/d	d/d	(10)/1

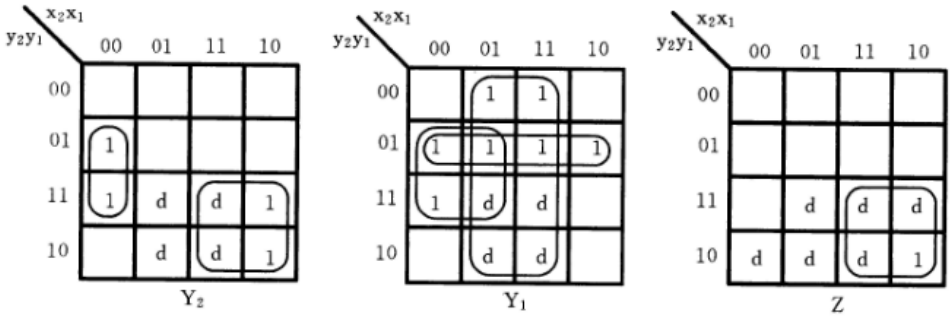


图 6.34　激励函数和输出函数卡诺图

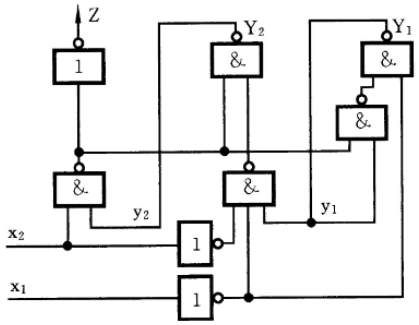


图 6.35　逻辑电路

第7章　中规模通用集成电路及其应用

- 7.1　常用中规模组合逻辑电路
- 7.2　常用中规模时序逻辑电路
- 7.3　常用中规模信号产生与变换电路

第7章主要内容

- 一、二进制并行加法器
  - 4位二进制串行加法器（串行进位）（图7.1）
  - 4位二进制并行加法器（超前进位）：74283芯片（图7.2）
    - 进位产生函数：Gi=Ai·Bi　　进位传递函数：Pi=Ai⊕Bi
    - C1=P1·C0+G1
    - C2=P2·C1+G2=P2·P1·C0+P2·G1+G2
    - C3=P3·C2+G3=P3·P2·P1·C0+P3·P2·G1+P3·G2+G3
    - C4=P4·C3+G4= P4·P3·P2·P1·C0+P4·P3·P2·G1+P4·P3·G2+P4·G3+G4
    - Fi=Pi⊕Ci-1　　Ci=Pi·Ci-1+Gi
    - 利用74283芯片可以实现代码转换、二进制减法运算、二进制乘法运算、十进制加法运算等功能。

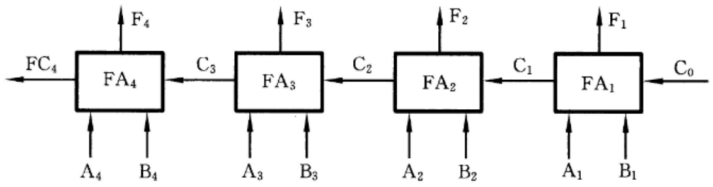


图 7.1　4 位串行进位二进制并行加法器的结构框图

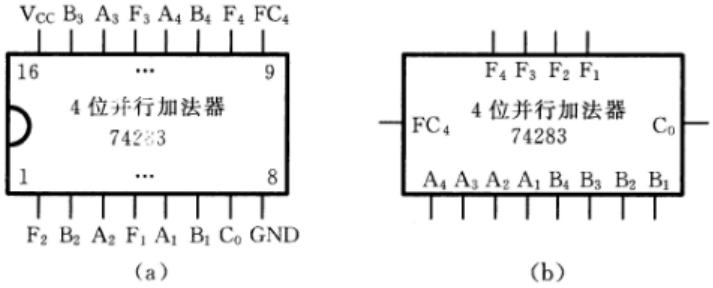
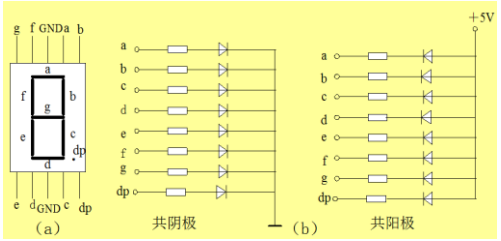


图 7.2　74283 的引脚排列图和逻辑符号

- 二、译码器
  - 二进制译码器：74138芯片（3-8译码器）（图7.7）
  - 二-十进制译码器：7442芯片（图7.10）
  - 七段显示译码器：7448芯片（图7.11）、7447芯片（Logisim）
    - 七段数码管（八段数码管）（共阴极、共阳极）
    - 七段码（共阴极七段码、共阳极七段码）



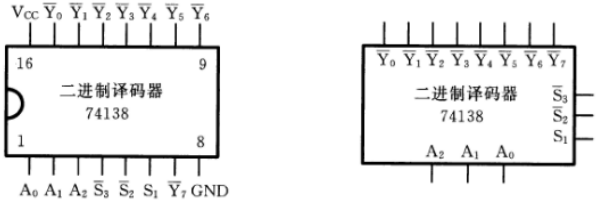


图 7.7　74138 译码器的引脚排列图和逻辑符号

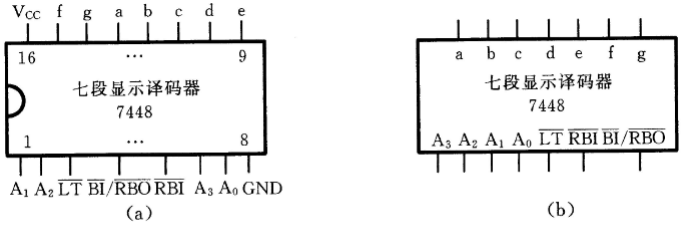
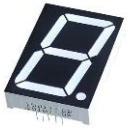


图 7.11　7448 的引脚排列图和逻辑符号



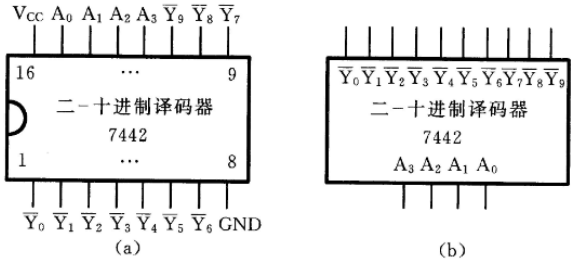
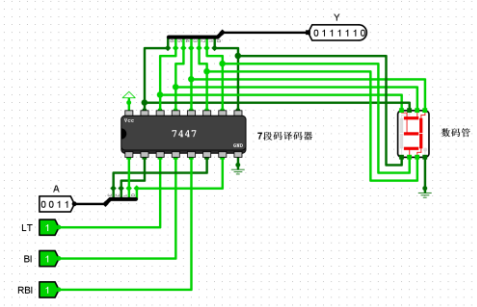


图 7.10　二-十进制译码器 7442 的引脚排列图和逻辑符号



显示字符	共阴极段选码	共阳极段选码	显示字符	共阴极段选码	共阳极段选码
0	3FH	C0H	C	39H	C6H
1	06H	F9H	d	5EH	A1H
2	5BH	A4H	E	79H	86H
3	4FH	B0H	F	71H	84H
4	66H	99H			
5	6DH	92H			
6	7DH	82H			
7	07H	F8H			
8	7FH	80H			
9	6FH	90H	“灭”	00H	FFH
A	77H	88H			⋮
b	7CH	83H			

七段码

- 三、编码器
  - 二-十进制编码器：按键式8421码编码器（图7.13）
  - 优先编码器：74148芯片（8-3线优先编码器）（图7.14、表7.6）

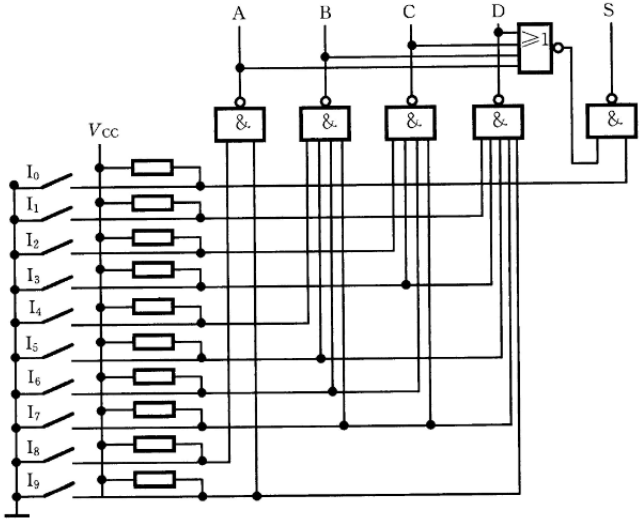


图 7.13　按键式 8421 码编码器

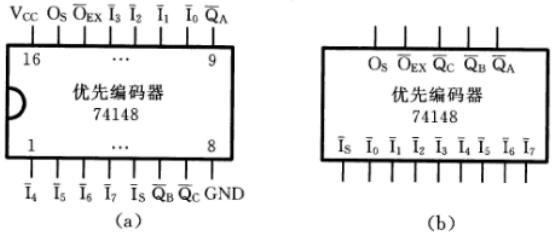
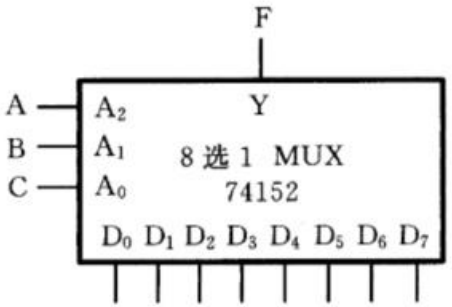


图 7.14　74148 优先编码器引脚排列和逻辑符号

表 7.6　74148 真值表

输入									输出				
$\overline{I}_S$	$\overline{I}_0$	$\overline{I}_1$	$\overline{I}_2$	$\overline{I}_3$	$\overline{I}_4$	$\overline{I}_5$	$\overline{I}_6$	$\overline{I}_7$	$\overline{Q}_C$	$\overline{Q}_B$	$\overline{Q}_A$	$\overline{Q}_{EX}$	O_S
1	d	d	d	d	d	d	d	d	1	1	1	1	1
0	1	1	1	1	1	1	1	1	1	1	1	1	0
0	d	d	d	d	d	d	d	0	0	0	0	0	1
0	d	d	d	d	d	d	0	1	0	0	1	0	1
0	d	d	d	d	d	0	1	1	0	1	0	0	1
0	d	d	d	d	0	1	1	1	0	1	1	0	1
0	d	d	d	0	1	1	1	1	1	0	0	0	1
0	d	d	0	1	1	1	1	1	1	0	1	0	1
0	d	0	1	1	1	1	1	1	1	1	0	0	1
0	0	1	1	1	1	1	1	1	1	1	1	0	1

- 四、多路选择器（MUX）
  - 74152芯片：8路MUX（左下角图）
  - 74153芯片：双4路MUX（图7.16）
  - 可以用具有n个选择控制变量的MUX（4路MUX的n=2，8路MUX的n=3，16路MUX的n=4……）实现m个变量的函数，有3种情况：
    ① m=n
    ② m=n+1
    ③ m>=n+2



74152芯片的逻辑符号

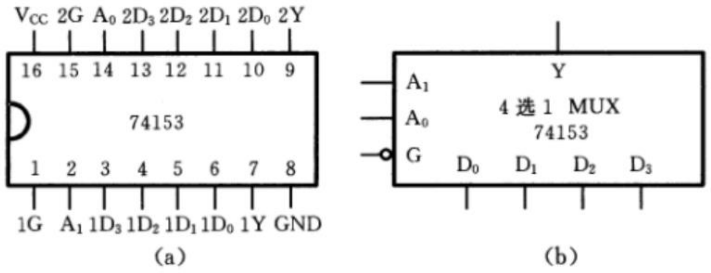


图 7.16 74153 引脚排列图和逻辑符号

- 五、多路分配器（DEMUX）
  - 4路DEMUX（图7.20）
  - 若将4路DEMUX的数据输入端D接1，则将实现2-4线译码器的功能
  - 可以将DEMUX（多路分配器）与MUX（多路选择器）联用，实现多通道数据分时传送（图7.21）

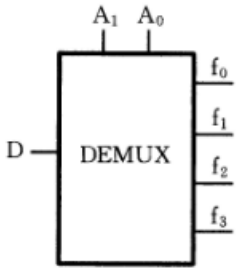


图 7.20　4 路 DEMUX 的逻辑符号

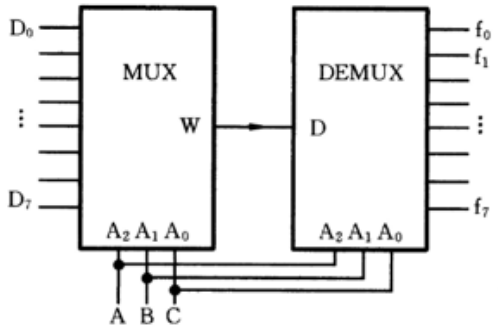


图 7.21　8 路数据传输示意图

- 六、同步计数器
  - **74193**芯片：双时钟4位二进制同步可逆计数器（具有清零、预置、加1、减1等功能）（图**7.23**，表**7.9**，表**7.10**）
  - 利用**74193**的清零、预置等功能，可以实现模小于**16**的计数器
  - 通过将多个**74193**串联，可以实现模大于**16**的计数器

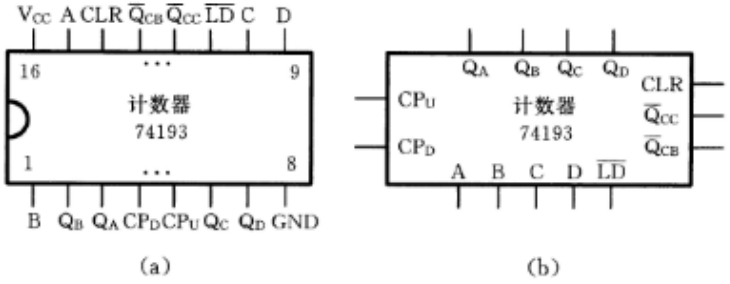


图 7.23　74193 的引脚排列图和逻辑符号

表 7.9　74193 输入/输出信号的说明

引线名称		说　明
输入端	CLR	清除
	$\overline{LD}$	预置控制
	D,C,B,A	预置初置
	CP_U ↑	累加计数脉冲
	CP_D ↑	累减计数脉冲
输出端	$Q_DQ_CQ_BQ_A$	记数值
	$\overline{Q}_{CC}$	进位输出负脉冲
	$\overline{Q}_{CB}$	借位输出负脉冲

表 7.10　74193 的功能表

输入								输出			
CLR	$\overline{LD}$	D	C	B	A	CP_U	CP_D	Q_D	Q_C	Q_B	Q_A
1	d	d	d	d	d	d	d	0	0	0	0
0	0	x_3	x_2	x_1	x_0	d	d	x_3	x_2	x_1	x_0
0	1	d	d	d	d	↑	1	累	加	计	数
0	1	d	d	d	d	1	↑	累	减	计	数

- 七、异步计数器
  - **74290芯片：二-五-十进制加法计数器（具有异步清零、异步置9、模2计数、模5计数、模10计数等功能）（图7.28，表7.11）**

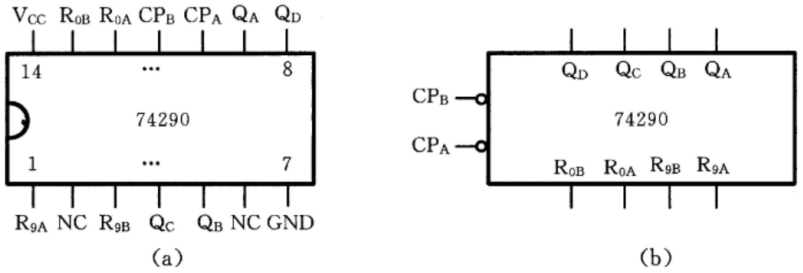
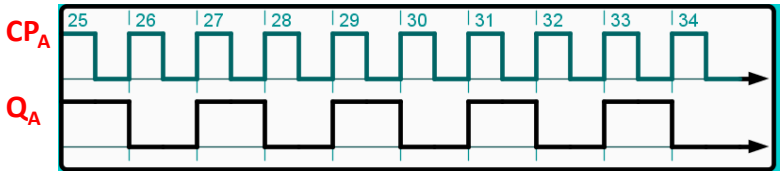


图 7.28　74290 的引脚排列图和逻辑符号



模2计数器

表 7.11　74290 的功能表

输入					输出			
R_{0A}	R_{0B}	R_{9A}	R_{9B}	CP	Q_D	Q_C	Q_B	Q_A
1	1	0	d	d	0	0	0	0
1	1	d	0	d	0	0	0	0
d	d	1	1	d	1	0	0	1
d	0	d	0	↓	计数			
0	d	0	d	↓	计数			
0	d	d	0	↓	计数			
d	0	0	d	↓	计数			

表 7.12　74290 模 5 计数器状态表

序号	Q_D	Q_C	Q_B
0	0	0	0
1	0	0	1
2	0	1	0
3	0	1	1
4	1	0	0

表 7.13　8421 码模 10 计数器状态表

序号	Q_D	Q_C	Q_B	Q_A
0	0	0	0	0
1	0	0	0	1
2	0	0	1	0
3	0	0	1	1
4	0	1	0	0
5	0	1	0	1
6	0	1	1	0
7	0	1	1	1
8	1	0	0	0
9	1	0	0	1

表 7.14　5421 码模 10 计数器状态表

序号	Q_A	Q_D	Q_C	Q_B
0	0	0	0	0
1	0	0	0	1
2	0	0	1	0
3	0	0	1	1
4	0	1	0	0
5	1	0	0	0
6	1	0	0	1
7	1	0	1	0
8	1	0	1	1
9	1	1	0	0

- 八、寄存器
  - **74194**芯片：**4**位双向移位寄存器（具有并行输入、右移串行输入、左移串行输入、保持、清零等功能）（图**7.32**，表**7.15**，表**7.16**，表**7.17**）
  - 寄存器还可以用来构成计数器、序列信号发生器等逻辑器件

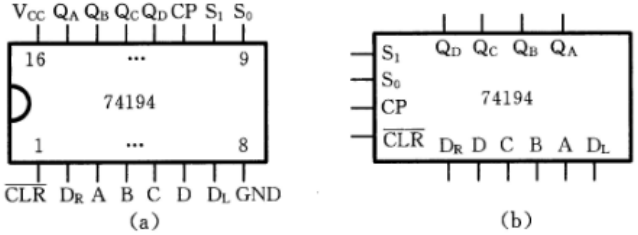


图 7.32　74194 的管脚排列图和逻辑符号

表 7.15　74194 引线说明

引线名称		说　明
输入端	$\overline{CLR}$	清除
	D,C,B,A	并行数据输入
	D_R	右移串行数据输入
	D_L	左移串行数据输入
	S_1,S_0	工作方式选择控制
	CP	工作脉冲
输出端	Q_D,Q_C,Q_B,Q_A	寄存器的状态

表 7.16　控制端的功能

控制信号 S_1	控制信号 S_0	功　能
0	0	保　持
0	1	右　移
1	0	左　移
1	1	并行输入

表 7.17　74194 的功能表

输入										输出			
$\overline{CLR}$	CP	S_1	S_0	D_R	D_L	D	C	B	A	Q_D	Q_C	Q_B	Q_A
0	d	d	d	d	d	d	d	d	d	0	0	0	0
1	0	d	d	d	d	d	d	d	d	Q_D^n	Q_C^n	Q_B^n	Q_A^n
1	↑	1	1	d	d	x_3	x_2	x_1	x_0	x_3	x_2	x_1	x_0
1	↑	0	1	1	d	d	d	d	d	1	Q_D^n	Q_C^n	Q_B^n
1	↑	0	1	0	d	d	d	d	d	0	Q_D^n	Q_C^n	Q_B^n
1	↑	1	0	d	1	d	d	d	d	Q_C^n	Q_B^n	Q_A^n	1
1	↑	1	0	d	0	d	d	d	d	Q_C^n	Q_B^n	Q_A^n	0
1	d	0	0	d	d	d	d	d	d	Q_D^n	Q_C^n	Q_B^n	Q_A^n

第1行：清零
第2、8行：保持
第3行：并行输入
第4行：右移串行输入1
第5行：右移串行输入0
第6行：左移串行输入1
第7行：左移串行输入0

- 九、定时器555
  - 5G555芯片：5个输入CO、TH、/TR、D、/RD，1个输出OUT（图7.37）
  - 定时器555可以实现多谐振荡器（图7.38，图7.39）、施密特触发器（图7.41）、单稳态触发器（图7.43）等电路，完成脉冲信号的产生、定时和整形等功能触发器

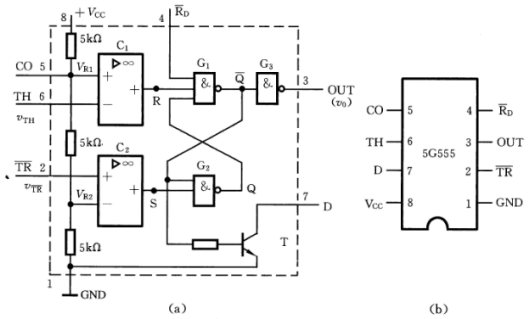


图 7.37　5G555 的电路结构和管脚功能

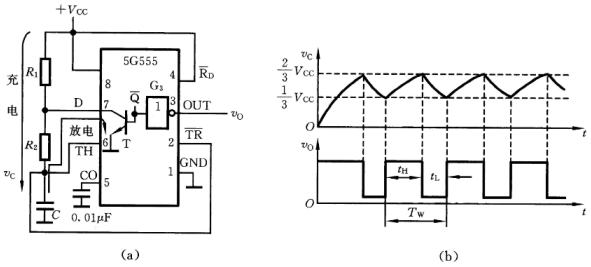


图 7.38　用 5G555 构成的多谐振荡器电路及其工作波形图

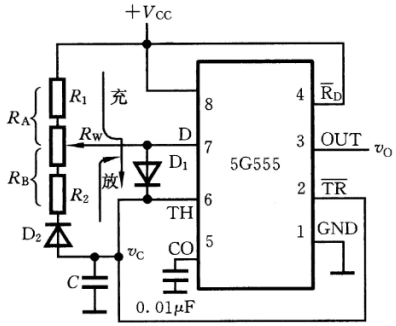


图 7.39　占空比可调的多谐振荡器

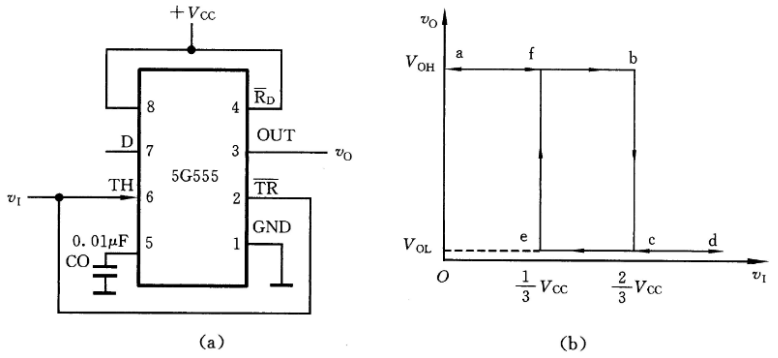


图 7.41　5G555 构成的施密特触发器

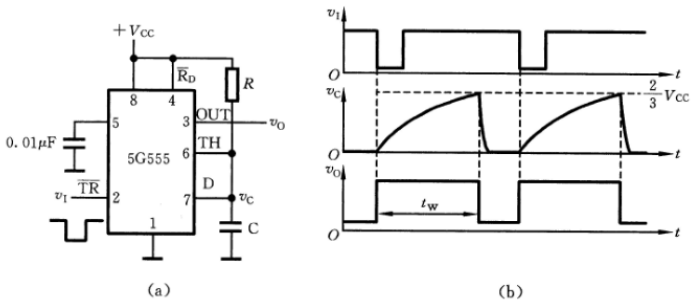


图 7.43　5G555 构成的单稳态触发器

- **十、D/A转换器**
  - **4位电流型R-2R倒T形电阻网络D/A转换器的原理图（图7.46）**
  - **DAC0832芯片：8位D/A转换器（图7.49）**
  - **DAC0832芯片的3种工作方式：**
    ① 双缓冲方式

    ② 单缓冲方式

    ③ 直通方式

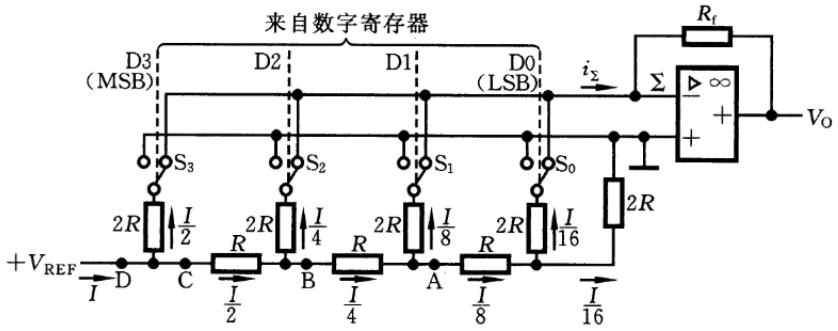


图 7.46　电流型倒 T 形电阻网络

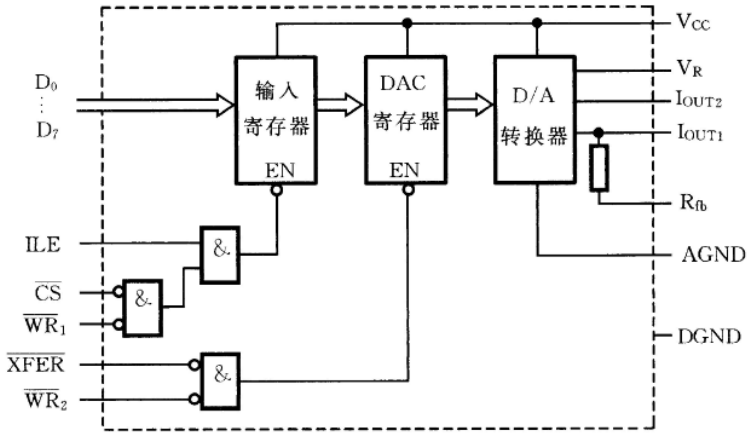


图 7.49　DAC0832 的内部结构图

- 十一、A/D转换器
  - 逐次比较型A/D转换器也称逐次逼近型A/D转换器（图7.51），其工作原理与太平称重的过程类似
  - ADC0809芯片：8位A/D转换器（图7.52）

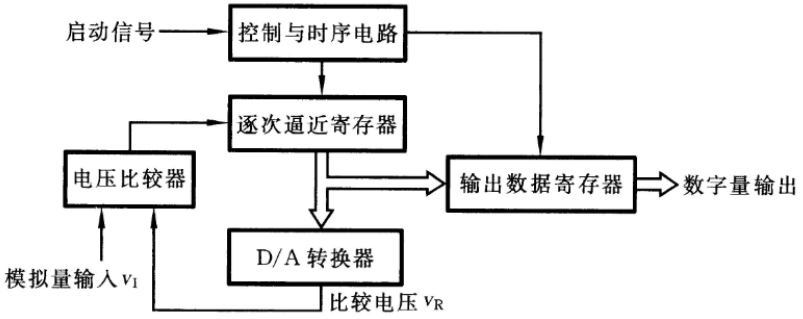


图 7.51　逐次比较型 A/D 转换器的结构框图

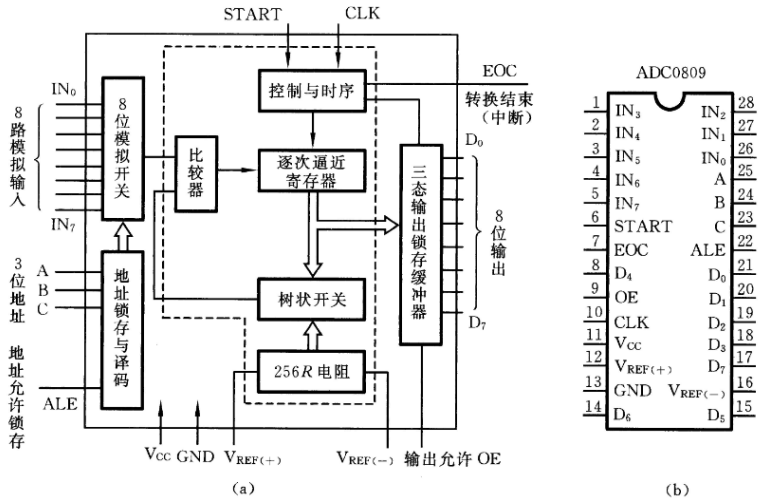


图 7.52　ADC0809 原理框图和管脚排列图

- 例7.1：用4位二进制并行加法器（74283）设计一个8421码转换成余3码的代码转换电路。
- 解：
  - 余3码=8421码+3。
  - 转换电路见图7.3。

表 1.3　常用的 3 种 BCD 码

十进制字符	8421 码	2421 码	余 3 码
0	0000	0000	0011
1	0001	0001	0100
2	0010	0010	0101
3	0011	0011	0110
4	0100	0100	0111
5	0101	1011	1000
6	0110	1100	1001
7	0111	1101	1010
8	1000	1110	1011
9	1001	1111	1100

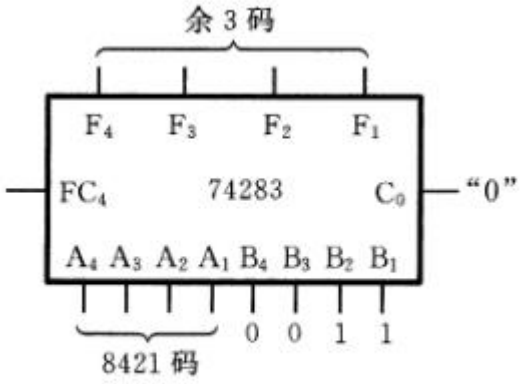


图 7.3　逻辑电路

- 例7.2：用4位二进制并行加法器（74283）设计一个4位二进制并行加法/减法器。

- 解：
  - A-B=A+/B+1，例如：
    - A=6=0,110　B=5=0,101　/B=1,010　A+/B+1=0,110+1,010+1=0,001=1=6-5。
    - A=5=0,101　B=6=0,110　/B=1,001　A+/B+1=0,101+1,001+1=1,111=-1=5-6。
  - 4位二进制并行加法/减法器电路如图7.4所示。
  - M=0，A=a，B=b⊕0=b，s=F=A+B+C_0=A+B+0=a+b。
  - M=1，A=a，B=b⊕1=/b，s=F=A+B+C_0=a+/b+1=a-b。

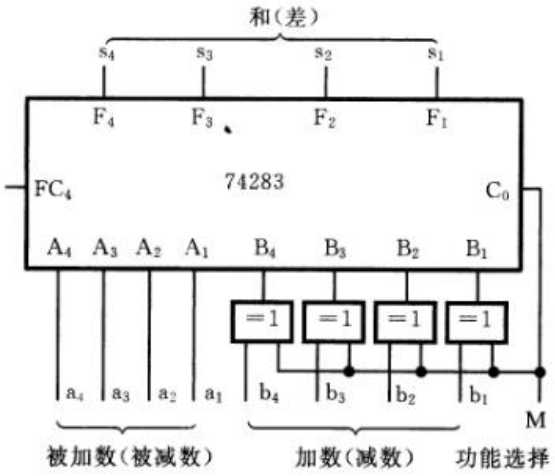


图 7.4　逻辑电路

- 例7.3：用4位二进制并行加法器（74283）设计一个用余3码表示的1位十进制数加法器。

- 解：
  - 余3码加法规则：如果相加结果无进位产生，则“和”需要减3；如果相加结果有进位产生，则“和”需要加3。例如：
    - 1+2=0100+0101=1001　　无进位　　“和”=1001-3=1001-11=0110=3
    - 5+6=1000+1001=10001　　有进位　　“和”=10001+3=10001+11=10100=11
  - 用余3码表示的1位十进制数加法器电路如图7.5所示：第I片74283完成余3码相加，第II片74283对相加结果进行修正。
    - 当第I片相加结果无进位时，进位输出=0，第II片的$A_4 \sim A_1$=1101=-3，即完成减3的操作。
    - 当第I片相加结果有进位时，进位输出=1，第II片的$A_4 \sim A_1$=0011=3，即完成加3的操作。

表 1.3　常用的 3 种 BCD 码

十进制字符	8421 码	2421 码	余 3 码
0	0000	0000	0011
1	0001	0001	0100
2	0010	0010	0101
3	0011	0011	0110
4	0100	0100	0111
5	0101	1011	1000
6	0110	1100	1001
7	0111	1101	1010
8	1000	1110	1011
9	1001	1111	1100

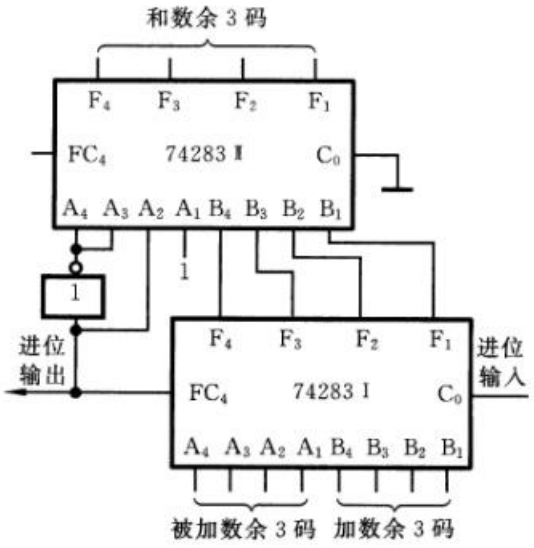


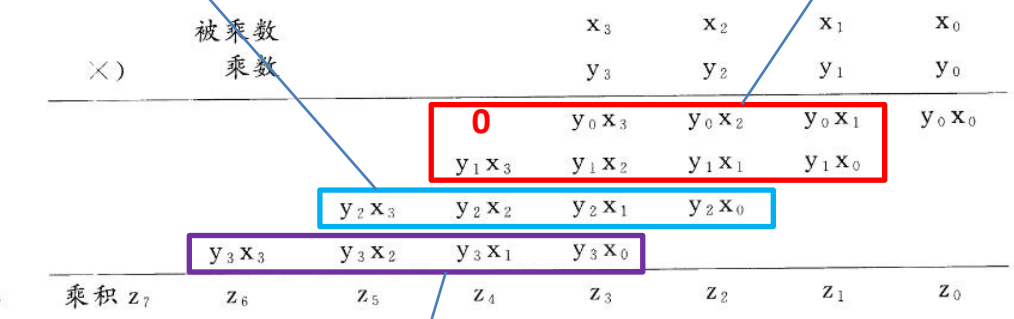
图 7.5　逻辑电路

- 例7.4：用4位二进制并行加法器（74283）实现4位二进制乘法器的逻辑功能。
- 解：
  - 设$X=x_3x_2x_1x_0$，$Y=y_3y_2y_1y_0$，$Z=X*Y=z_7z_6z_5z_4z_3z_2z_1z_0$，运算过程如下（左下图）。
  - 可以用16个与门、3个4位并行加法器（74283）实现上述运算过程，具体电路如图7.6所示。

第1片74283（右下角）完成红色框中2个4位数的相加

第2片74283（左下角）完成第1片74283相加后结果中的FC_4、F_4、F_3、F_2与蓝色框中4位数的相加

第3片74283（左上角）完成第2片74283相加后结果中的FC_4、F_4、F_3、F_2与紫色框中4位数的相加



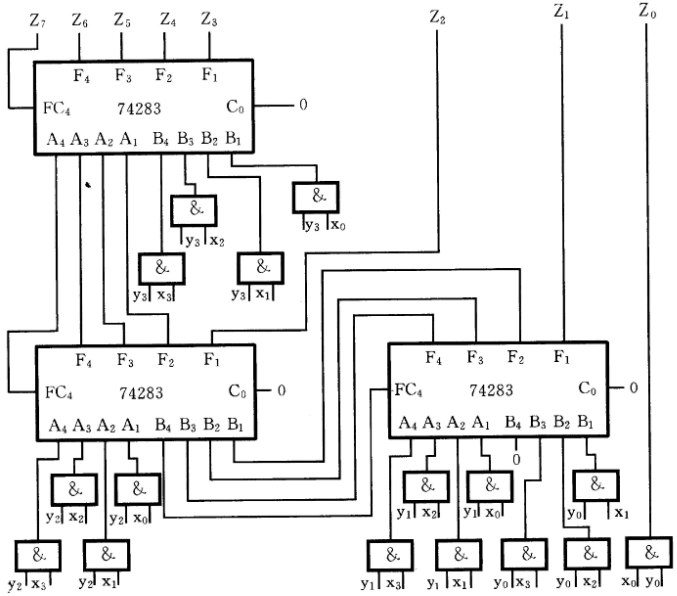


图 7.6　逻辑电路

- 例7.5：用3-8译码器74138和与非门实现全减器的功能。
- 解：
  - 全减器：$\{G_i, D_i\} = A_i - B_i - G_{i-1}$。其中$D_i$为差，$G_{i-1}$为低位借位，$G_i$为高位借位。全减器的真值表见表7.2。
  - 由真值表可以得到下面的逻辑公式：
    - $D_i=m_1+m_2+m_4+m_7=/(/m_1\cdot/m_2\cdot/m_4\cdot/m_7)$
    - $G_i=m_1+m_2+m_3+m_7=/(/m_1\cdot/m_2\cdot/m_3\cdot/m_7)$
  - 因为3-8译码器的输出就是8个最小项（$m_0 \sim m_7$），因此可以通过3-8译码器实现上述逻辑公式，具体见图7.8（译码器的输入接A_i、B_i、G_{i-1}，控制端：S_1=1、$/S_2$=0、$/S_3$=0）。

表 7.2　全减器真值表

输入			输出	
A_i	B_i	G_{i-1}	D_i	G_i
0	0	0	0	0
0	0	1	1	1
0	1	0	1	1
0	1	1	0	1
1	0	0	1	0
1	0	1	0	0
1	1	0	0	0
1	1	1	1	1

够减，0-0-0=0，因此D_i=0、G_i=0

不够减，借1位，相当于10+0-0-1=1，因此D_i=1、G_i=1

不够减，借1位，相当于10+0-1-0=1，因此D_i=1、G_i=1

不够减，借1位，相当于10+0-1-1=0，因此D_i=1、G_i=0

够减，1-0-0=1，因此D_i=0、G_i=1

够减，1-0-1=0，因此D_i=0、G_i=0

够减，1-1-0=0，因此D_i=0、G_i=0

不够减，借1位，相当于10+1-1-1=1，因此D_i=1、G_i=1

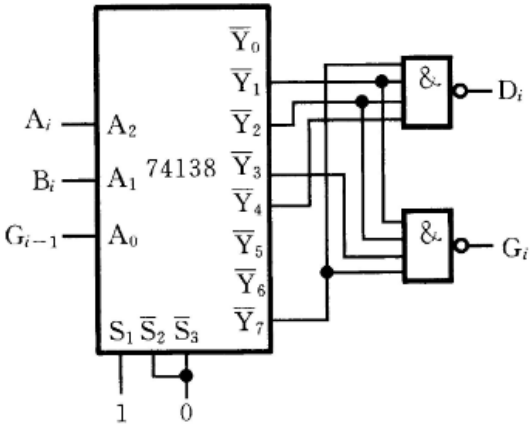
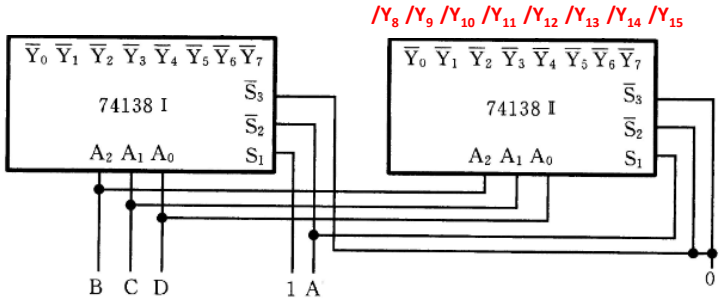


图 7.8　逻辑电路

- 例7.6：用译码器和与非门实现逻辑函数：F(A,B,C,D)= ∑ m(2,4,6,8,10,12,14)。
- 解：
  - 因为有4个变量，因此需要使用4-16译码器实现上述逻辑函数。
  - 也可以使用2个3-8译码器实现4-16译码的功能（见左下图）。
    - A=0时，第I片3-8译码器工作、第II片3-8译码器不工作，BCD=000～111，对应第I片3-8译码器的8个输出。
    - A=1时，第I片3-8译码器不工作、第II片3-8译码器工作，BCD=000～111，对应第II片3-8译码器的8个输出。
  - 因此可以使用2个3-8译码器实现上述逻辑函数（见图7.9）。



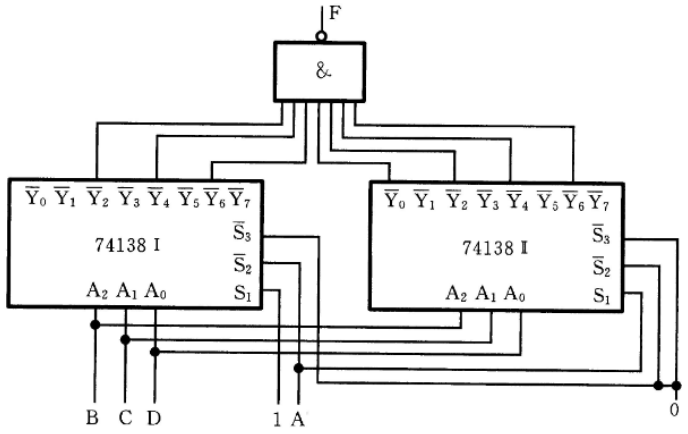


图 7.9　逻辑电路

- 例7.7：用优先编码器74148设计一个能裁决16级不同中断请求的中断优先编码器。
- 解：
  - 具体见图7.15：$/I_{Z0}$～$/I_{Z15}$为16个中断请求信号，$/I_{ZS}$为选通输入端。$/Q_{ZD}$～$/Q_{ZA}$为编码输出，O_{ZS}为选通输出端，$/Q_{ZEX}$为工作状态标志。
    - 若$/I_{ZS}$=1：第II片74148输出全部为1，导致第I片74148的$/I_S$=0，第I片74148输出也全部为1；即$/Q_{ZD}$～$/Q_{ZA}$=1111，O_{ZEX}=1，O_{ZS}=1。
    - 若$/I_{ZS}$=0，$/I_{Z15}$～$/I_{Z0}$无中断请求信号（全部为1）：第II片74148的O_S=0、其余4个输出全部为1，第I片74148的$/I_S$=0，则第I片74148的O_S=0 、其余4个输出全部为1。因此，$/Q_{ZD}$=$/Q_{EX}$(第II片)=1，$/Q_{ZC}$=1·1=1，$/Q_{ZB}$=1·1=1，$/Q_{ZA}$=1·1=1，O_{ZS}=O_S(第I片)=0，O_{ZEX}=1·1=1；即$/Q_{ZD}$～$/Q_{ZA}$=1111，O_{ZEX}=1，O_{ZS}=0。
    - 若$/I_{ZS}$=0，$/I_{Z15}$～$/I_{Z8}$有中断请求信号（有一个或多个0，例如$/I_{Z10}$=0）：第II片74148处于工作状态，其O_S=1，使第I片74148不工作，其5个输出全部为1。因此，$/Q_{ZD}$=$/O_{EX}$(第II片)=0，$/Q_{ZC}$=1·/Q_C(第II片)=1，$/Q_{ZB}$=1·/Q_B(第II片)=0，$/Q_{ZA}$=1·/Q_A(第II片)=1，O_{ZS}=O_S(第I片)=1，O_{ZEX}=1·/O_{EX}(第II片)=0；即$/Q_{ZD}$～$/Q_{ZA}$=0101，O_{ZEX}=0，O_{ZS}=1。
    - 若$/I_{ZS}$=0，$/I_{Z7}$～$/I_{Z0}$有中断请求信号（有一个或多个0，例如$/I_{Z3}$=0）：第II片74148处于工作状态，其O_S=0，其余4个输出全部为1，使第I片74148工作。因此，$/Q_{ZD}$=$/O_{EX}$(第II片)=1，$/Q_{ZC}$=1·/Q_C(第I片)=1，$/Q_{ZB}$=1·/Q_B(第I片)=0，$/Q_{ZA}$=1·/Q_A(第I片)=0，O_{ZS}=O_S(第I片)=1，O_{ZEX}=1·/O_{EX}(第I片)=0。即$/Q_{ZD}$～$/Q_{ZA}$=1100，O_{ZEX}=0，O_{ZS}=1。

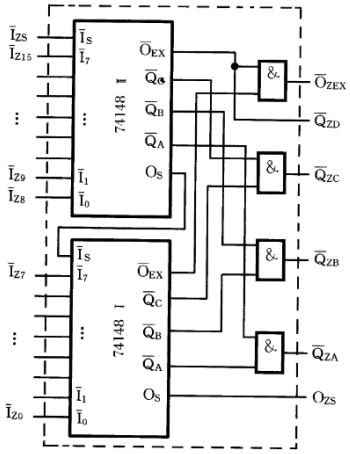


图 7.15　16 级中断优先编码器

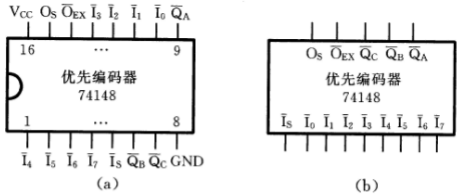
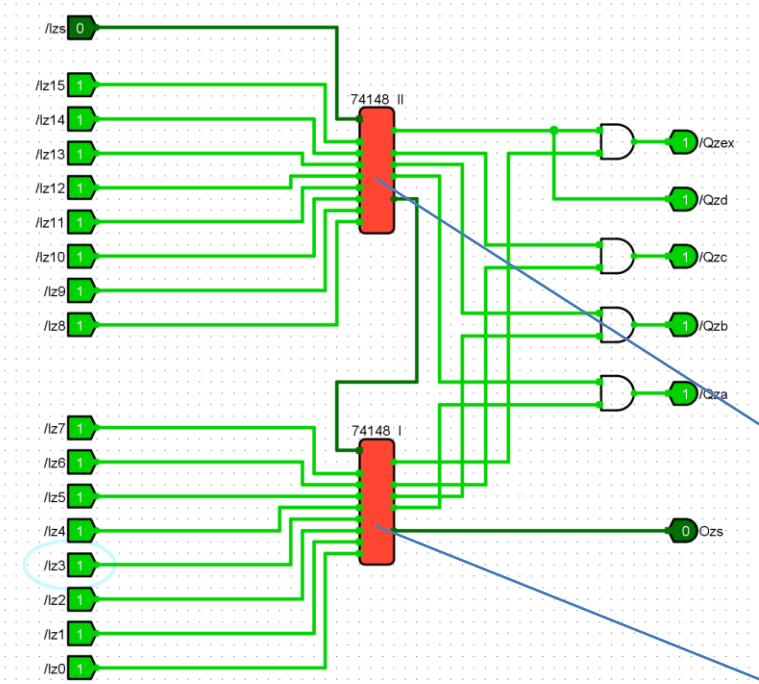


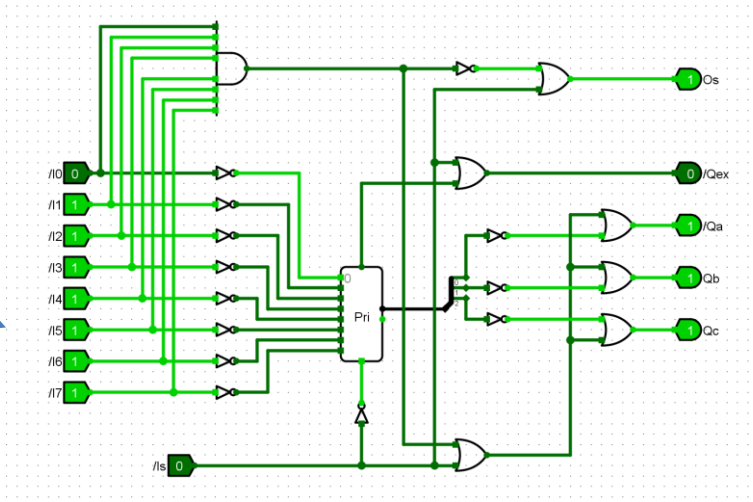
图 7.14　74148 优先编码器引脚排列和逻辑符号

表 7.6　74148 真值表

输入									输出				
$\bar{I}_S$	$\bar{I}_0$	$\bar{I}_1$	$\bar{I}_2$	$\bar{I}_3$	$\bar{I}_4$	$\bar{I}_5$	$\bar{I}_6$	$\bar{I}_7$	$\bar{Q}_C$	$\bar{Q}_B$	$\bar{Q}_A$	$\bar{Q}_{EX}$	O_S
1	d	d	d	d	d	d	d	d	1	1	1	1	1
0	1	1	1	1	1	1	1	1	1	1	1	1	0
0	d	d	d	d	d	d	d	0	0	0	0	0	1
0	d	d	d	d	d	d	0	1	0	0	1	0	1
0	d	d	d	d	d	0	1	1	0	1	0	0	1
0	d	d	d	d	0	1	1	1	0	1	1	0	1
0	d	d	d	0	1	1	1	1	1	0	0	0	1
0	d	d	0	1	1	1	1	1	1	0	1	0	1
0	d	0	1	1	1	1	1	1	1	1	0	0	1
0	0	1	1	1	1	1	1	1	1	1	1	0	1

– 例7.7在Logisim中的实现。





74148

- 例7.8：用MUX实现以下逻辑函数的功能：Y(A,B,C)= ∑ m(2,3,5,6)。
- 解：
  - 方案一：采用8路MXU（74152芯片）实现。
    - 属于上面的第①种情况，m=3，n=3，m=n。
    - 根据第①情况的方法，8路MUX的数据输入端为：D_0=0、D_1=0、D_2=1、D_3=1、D_4=0、D_5=1、D_6=1、D_7=0。具体电路见图7.17(a)。

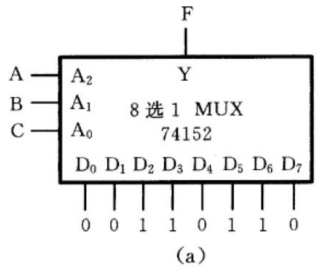


图7.17　例7.8实现的第一种方案

– 方案二：采用4路MUX（74153芯片）实现。

- 属于上面的第②种情况，m=3，n=2，m=n+1。

- 根据第②情况的方法，从A、B、C中选择A、B作为4路MUX的选择控制变量（A_1、A_0），将逻辑函数转换为：
  Y(A,B,C)= ∑ m(2,3,5,6)=/A·B·/C+/A·B·C+A·/B·C+A·B·/C=/A·/B·0+/A·B·(/C+C)+A·/B·C+A·B·/C
  =/A·/B·0+/A·B·1+A·/B·C+A·B·/C。

- 因此4路MUX的数据输入端为：D_0=0、D_1=1、D_2=C、D_3=/C。具体电路见图7.17(b)。

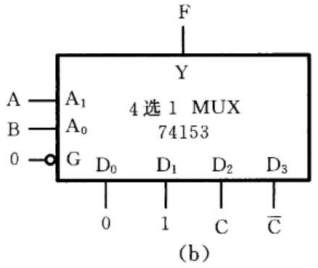


图7.17　例7.8实现的第二种方案

- 例7.9：用4路MUX实现4变量逻辑函数的功能：Y(A,B,C,D)= ∑ m(0,2,3,7,8,9,10,13)。
- 解：
  - 该例属于上面的第③种情况，m=4，n=2，m>=n+2。
  - 方案一：选用A、B作为4路MUX（74153芯片）的选择控制变量（A_1、A_0）。
    - 根据第③情况的方法，将逻辑函数转换为：
      Y(A,B,C,D)= ∑ m(0,2,3,7,8,9,10,13)=/A·/B·/C·/D+/A·/B·C·/D+/A·/B·C·D+/A·B·C·D+A·/B·/C·/D+A·/B·/C·D+/A·B·/C·D+/A·/B·C·/D=/A·/B·(C+/D)+/A·B·(C·D)+A·/B·(/C+/D)+A·B·(/C·D)。
    - 4路MUX的数据输入端为：D_0= C+/D、D_1= C·D 、D_2=/C+/D、D_3=/C·D。具体电路见图7.18(a)。

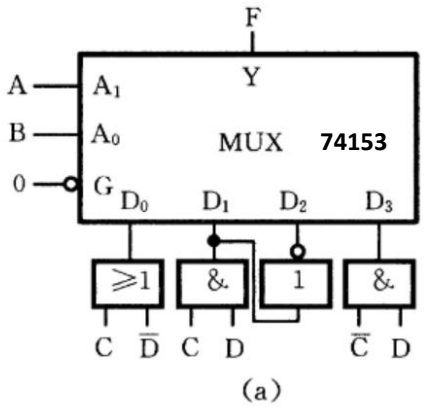


图7.18　例7.9实现的第一种方案

– 方案二：选用C、D作为4路MUX的选择控制变量（A_1、A_0）。

- 根据第③情况的方法，将逻辑函数转换为：
  Y(A,B,C,D)=∑m(0,2,3,7,8,9,10,13)=/A·/B·/C·/D+/A·/B·C·/D+/A·/B·C·D+/A·B·C·D+A·/B·/C·/D+A·/B·/C·D+/A·B·/C·D+/A·/B·C·/D=/C·/D·/B+/C·D·A+C·/D·/B+C·D·/A。

- 4路MUX的数据输入端为：D_0=/B、D_1=A 、D_2=/B、D_3=/A。具体电路见图7.18(b)。

- 可见方案二的电路比方案一的电路简单。

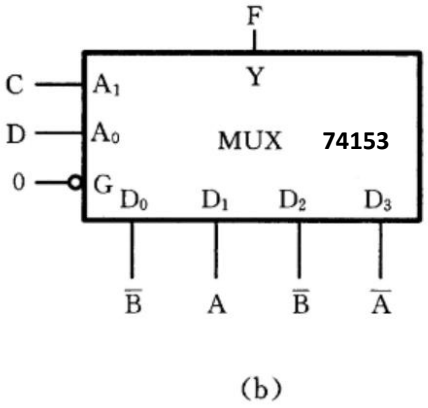


图7.18　例7.9实现的第二种方案

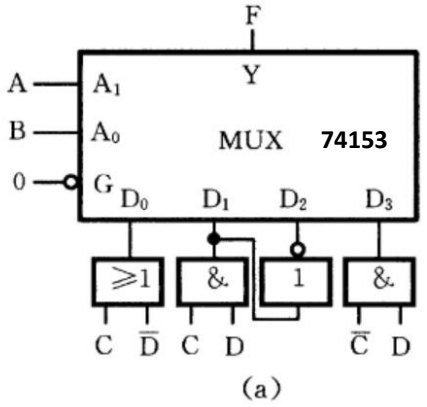


图7.18　例7.9实现的第一种方案

- 例7.10：用一片双4路MUX芯片74153实现4变量多输出函数：$F_1(A,B,C,D)=\sum m(0,1,5,7,10,13,15)$，$F_2(A,B,C,D)=\sum m(8,10,12,13,15)$。

- 解：
  - 该例属于上面的第③种情况，m=4，n=2，m>=n+2。
  - 选用A、B作为4路MUX的选择控制变量（A_1、A_0）。
  - 根据第③情况的方法，将逻辑函数转换为：
    - $F_1(A,B,C,D)=\sum m(0,1,5,7,10,13,15)=/A\cdot/B\cdot/C\cdot/D+/A\cdot/B\cdot/C\cdot D+/A\cdot B\cdot/C\cdot D+A\cdot B\cdot C\cdot D+A\cdot/B\cdot C\cdot/D+A\cdot B\cdot/C\cdot D+A\cdot B\cdot C\cdot D=/A\cdot/B\cdot/C+/A\cdot B\cdot D+A\cdot/B\cdot C\cdot/D+A\cdot B\cdot D$。
    - $F_2(A,B,C,D)=\sum m(8,10,12,13,15)=A\cdot/B\cdot/C\cdot/D+A\cdot/B\cdot C\cdot D+A\cdot B\cdot/C\cdot/D+A\cdot B\cdot/C\cdot D+A\cdot B\cdot C\cdot D=/A\cdot/B\cdot 0+/A\cdot B\cdot 0+A\cdot/B\cdot/D+A\cdot B\cdot(/C+D)$。
    - 双4路MUX的数据输入端为：$1D_0=/C$、$1D_1=D$ 、$1D_2=C\cdot/D$、$1D_3=D$；$2D_0=0$、$2D_1=0$、$2D_2=/D$、$2D_3=/C+D$。具体电路见图7.19。

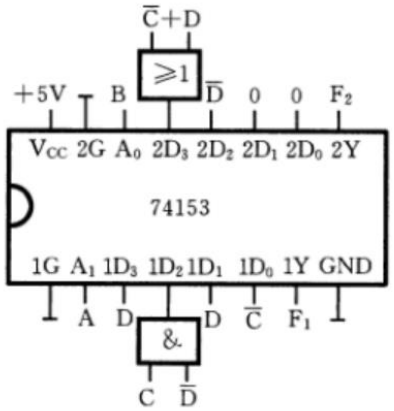


图 7.19　逻辑电路

- 例7.11：用8路MUX和3-8线译码器构造一个3位二进制数等值比较器。
- 解：
  - 比较器的电路如图7.22所示。两个3位二进制数（ABC，XYZ）分别接译码器的$A_2A_1A_0$和MUX的$A_2A_1A_0$。
  - 当ABC=XYZ时，输出F=0；当ABC≠XYZ时，输出F=1。
  - 例如，当ABC=XYZ=010时，译码器的/Y_2=0，MUX的D_2=/Y_2=0，故F=0。
  - 当ABC=010，XYZ≠010时，因为除/Y_2之外译码器的其它输出都是1，因此MUX的输入除D_2之外，其它都是1，故F=1。

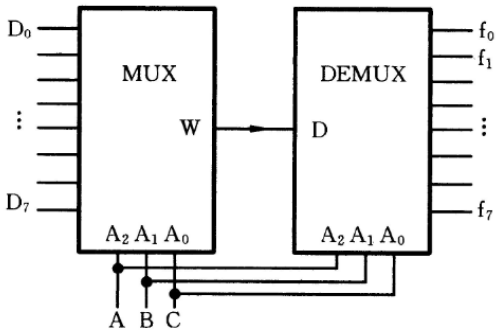


图 7.21　8 路数据传输示意图

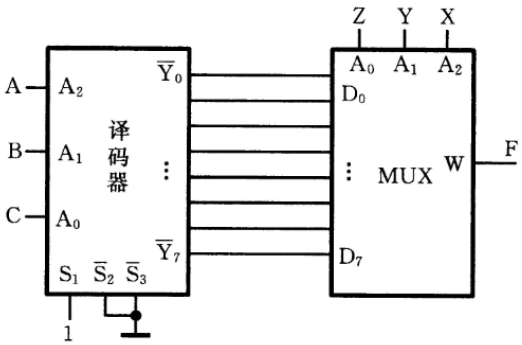


图 7.22　比较器的逻辑电路

- 例7.12：利用74193芯片（4位二进制同步可逆计数器）构成模10加法计数器。
- 解：
  - 模10加法计数器的状态为：0000->0001->0010->0011->0100->0101->0110->0111->1000->1001->0000……
  - 电路如图7.24所示：/LD=1，CP_D=1，CP_U接时钟脉冲信号CP，Q_3和Q_1通过与门接CLR。
  - 当计数器输出由1001变为1010时，Q_3=1、Q_1=1，与门输出=1，CLR=1，计数器清零，计数器输出=0000，即输出由1000变为0000，从而实现模10加法计数器的功能。

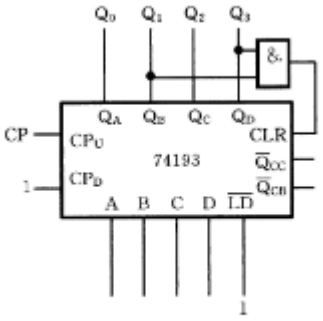


图 7.24　模 10 加法计数器逻辑电路

表 7.10　74193 的功能表

输入								输出			
CLR	$\overline{LD}$	D	C	B	A	CP_U	CP_D	Q_D	Q_C	Q_B	Q_A
1	d	d	d	d	d	d	d	0	0	0	0
0	0	x_3	x_2	x_1	x_0	d	d	x_3	x_2	x_1	x_0
0	1	d	d	d	d	↑	1	累	加	计	数
0	1	d	d	d	d	1	↑	累	减	计	数

- 例7.13：利用74193芯片（4位二进制同步可逆计数器）构成模12减法计数器。
- 解：
  - 假设模12减法计数器的状态为：1111->1110->1101->1100->1011->1010->1001->1000->0111->0110->0101->0100->1111……
  - 电路如图7.25所示：CLR=0，CP_U=1，CP_D接时钟脉冲信号CP，ABCD全部接1，Q_3和Q_2的“或”再和“初态设置”信号“与”后接/LD。
  - “初态设置”信号平时=1，需要进行初态设置时，该信号置为0，此时/LD=0，则将ABCD=1111初态送到计数器。
  - 当计数器的输出由0100变为0011时，Q_3=0、Q_2=0，或门输出=0，/LD=0，从而将ABCD=1111初态送到计数器，计数器输出=1111，即输出由0100变为1111，从而实现模12减法计数器的功能。

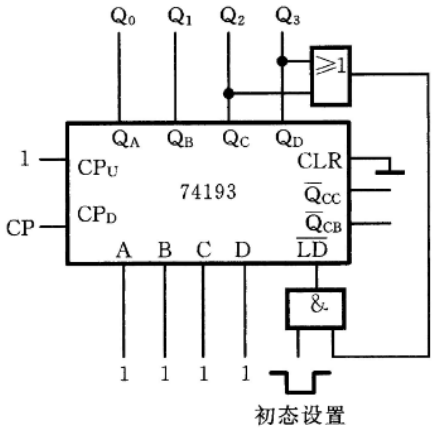


图 7.25　模 12 减法计数器逻辑电路

表 7.10　74193 的功能表

输入								输出			
CLR	$\overline{LD}$	D	C	B	A	CP_U	CP_D	Q_D	Q_C	Q_B	Q_A
1	d	d	d	d	d	d	d	0	0	0	0
0	0	x_3	x_2	x_1	x_0	d	d	x_3	x_2	x_1	x_0
0	1	d	d	d	d	↑	1	累	加	计	数
0	1	d	d	d	d	1	↑	累	减	计	数

- 例7.14：利用两片74193（4位二进制同步可逆计数器）构成模147（十进制数）加法计数器。
- 解：
  - 147=10010011，146=10010010。
  - 计数器的状态变化为：00000000（0）->00000001（1）->……->10010010（146）->00000000（0）。
  - 电路如图7.27所示：第1片74193：CP_D=1，CP_U接时钟脉冲信号CP，/LD=1，$/Q_{CC}$通过一个反相器接第2片CP_U。第2片74193：CP_D=1，/LD=1。第1片的Q_A、Q_B和第2片的Q_A、Q_D通过与门接到或门的一个输入端，或门的另一个输入端接“起始清零端”按钮。
  - 当计数器的输出由10010010（146）变为10010011（147）时，Q_0=1、Q_1=1、Q_4=1、Q_7=1，与门输出=1，或门输出=1，两片74193的CLR=1，两片74193的输出全部为0，从而实现10010010（146）->00000000（0）。

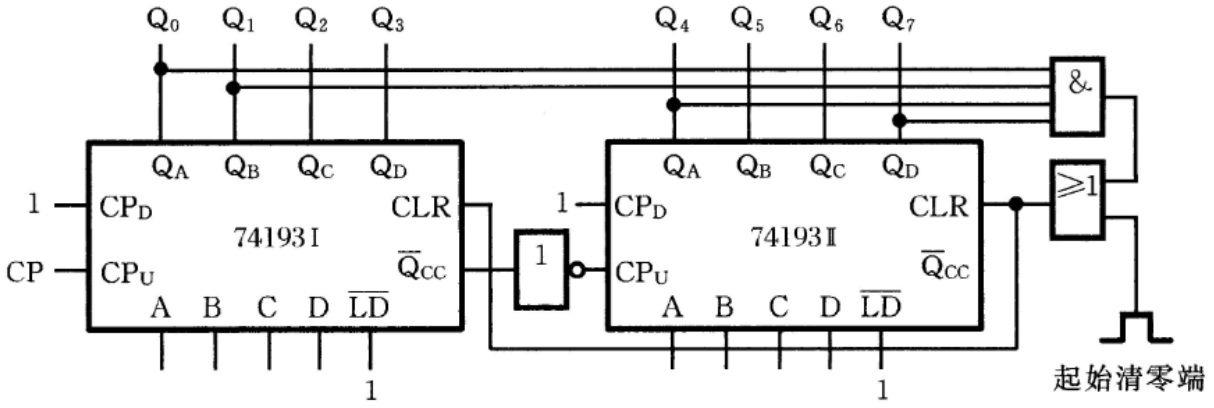


图 7.27　模 147 加法计数器逻辑电路

- 例7.15：利用74290（二-五-十进制加法计数器）设计一个模8加法计数器。
- 解：
  - 模8加法计数器的状态转移图见图7.30。
  - 逻辑电路见图7.31。
  - 该电路为74290工作在8421码十进制计数器方式（图7.29(a)），并且当输出由0111变为1000时，Q_D=1，使R_{0A}=R_{0B}=1，计数器清零，即0111变为0000，从而实现模8加法计数器的功能。

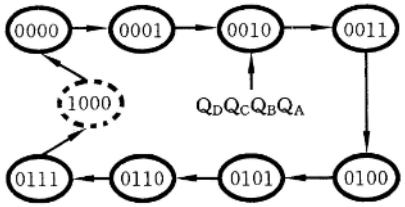


图 7.30　状态转移图

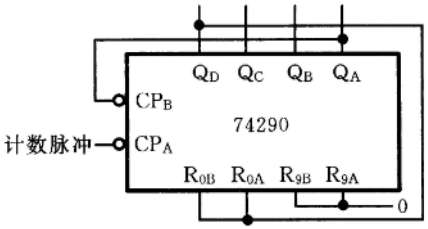


图 7.31　逻辑电路

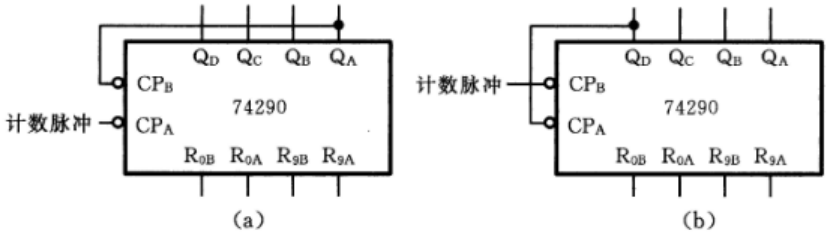


图 7.29　74290 构成的两种模 10 计数器连接示意图

表 7.11　74290 的功能表

输入					输出			
R_{0A}	R_{0B}	R_{9A}	R_{9B}	CP	Q_D	Q_C	Q_B	Q_A
1	1	0	d	d	0	0	0	0
1	1	d	0	d	0	0	0	0
d	d	1	1	d	1	0	0	1
d	0	d	0	↓	计数			
0	d	0	d	↓	计数			
0	d	d	0	↓	计数			
d	0	0	d	↓	计数			

- 例7.16：用74194（4位双向移位寄存器）构成一个模8计数器。该计数器的状态转移关系如表7.18所示。

- 解：

  – 要实现表7.18的功能，需要让74194工作在左移串行输入方式（S_1=1、S_0=0，表7.17中的第6、7行），且Q_D经反相器接D_L。

  – 一开始输出全部为0，Q_D=0，则D_L=1，74194实现左移串行输入1的功能。当Q_D=1，则D_L=0，74194实现左移串行输入0的功能。

表 7.18　状态转移表

现态				次态			
Q_D	Q_C	Q_B	Q_A	Q_D^{n+1}	Q_C^{n+1}	Q_B^{n+1}	Q_A^{n+1}
0	0	0	0	0	0	0	1
0	0	0	1	0	0	1	1
0	0	1	1	0	1	1	1
0	1	1	1	1	1	1	1
1	1	1	1	1	1	1	0
1	1	1	0	1	1	0	0
1	1	0	0	1	0	0	0
1	0	0	0	0	0	0	0

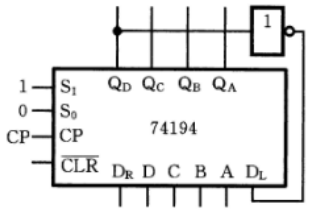


图 7.33　逻辑电路图

表 7.17　74194 的功能表

输入										输出			
$\overline{CLR}$	CP	S_1	S_0	D_R	D_L	D	C	B	A	Q_D	Q_C	Q_B	Q_A
0	d	d	d	d	d	d	d	d	d	0	0	0	0
1	0	d	d	d	d	d	d	d	d	Q_D^n	Q_C^n	Q_B^n	Q_A^n
1	↑	1	1	d	d	x_3	x_2	x_1	x_0	x_3	x_2	x_1	x_0
1	↑	0	1	1	d	d	d	d	d	1	Q_D^n	Q_C^n	Q_B^n
1	↑	0	1	0	d	d	d	d	d	0	Q_D^n	Q_C^n	Q_B^n
1	↑	1	0	d	1	d	d	d	d	Q_C^n	Q_B^n	Q_A^n	1
1	↑	1	0	d	0	d	d	d	d	Q_C^n	Q_B^n	Q_A^n	0
1	d	0	0	d	d	d	d	d	d	Q_D^n	Q_C^n	Q_B^n	Q_A^n

- 例7.17：用74194（4位双向移位寄存器）和适当的逻辑门构成产生序列为00101110的序列发生器。
- 解：
  - 序列发生器可由移位寄存器和反馈逻辑电路构成，如图7.34所示。假设序列发生器产生的序列周期（序列长度）为T_p（本例中T_p=8），移位寄存器的位数为n（74194芯片n=4），则需要满足$2^n>=T_p$（本例满足该要求，$2^4>=8$）。
  - 图7.35为序列与移位寄存器状态的关系。根据图7.35，假设移位寄存器的初始状态$Q_DQ_CQ_B$=100，则状态变化如表7.19所示（74194工作在右移串行输入方式）。
    - 当前状态$Q_DQ_CQ_B$=100，$F(D_R)$=0，则下一个状态$Q_DQ_CQ_B$=010
    - 当前状态$Q_DQ_CQ_B$=010，$F(D_R)$=1，则下一个状态$Q_DQ_CQ_B$=101
    - 当前状态$Q_DQ_CQ_B$=101，$F(D_R)$=1，则下一个状态$Q_DQ_CQ_B$=110
    - 当前状态$Q_DQ_CQ_B$=110，$F(D_R)$=1，则下一个状态$Q_DQ_CQ_B$=111
    - 当前状态$Q_DQ_CQ_B$=111，$F(D_R)$=0，则下一个状态$Q_DQ_CQ_B$=011
    - 当前状态$Q_DQ_CQ_B$=011，$F(D_R)$=0，则下一个状态$Q_DQ_CQ_B$=001
    - 当前状态$Q_DQ_CQ_B$=001，$F(D_R)$=0，则下一个状态$Q_DQ_CQ_B$=000
    - 当前状态$Q_DQ_CQ_B$=000，$F(D_R)$=1，则下一个状态$Q_DQ_CQ_B$=100

表 7.17　74194 的功能表

输入										输出			
$\overline{CLR}$	CP	S_1	S_0	D_R	D_L	D	C	B	A	Q_D	Q_C	Q_B	Q_A
0	d	d	d	d	d	d	d	d	d	0	0	0	0
1	0	d	d	d	d	d	d	d	d	Q_D^n	Q_C^n	Q_B^n	Q_A^n
1	↑	1	1	d	d	x_3	x_2	x_1	x_0	x_3	x_2	x_1	x_0
1	↑	0	1	1	d	d	d	d	d	1	Q_D^n	Q_C^n	Q_B^n
1	↑	0	1	0	d	d	d	d	d	0	Q_D^n	Q_C^n	Q_B^n
1	↑	1	0	d	1	d	d	d	d	Q_C^n	Q_B^n	Q_A^n	1
1	↑	1	0	d	0	d	d	d	d	Q_C^n	Q_B^n	Q_A^n	0
1	d	0	0	d	d	d	d	d	d	Q_D^n	Q_C^n	Q_B^n	Q_A^n

  - 根据表7.19，可以得到$F=(\overline{Q_D}+Q_C)\oplus Q_B$。由F的逻辑公式得到逻辑电路（图7.36）。一开始置S_1S_0=11，将DCB=100送寄存器，输出$Q_DQ_CQ_B$=100。然后，置S_1S_0=01（右移串行输入方式），则在时钟脉冲作用下，输出Z（Q_B）将产生00101110……序列信号。

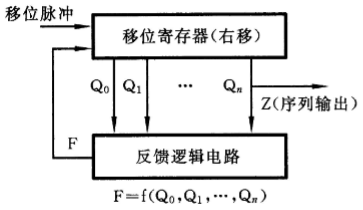


图 7.34　序列发生器结构框图

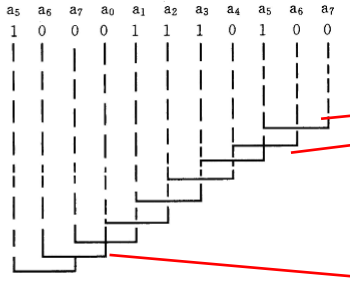


图 7.35　序列与移位寄存器状态的关系

表 7.19　电路状态变化表

CP	$F(D_R)$	Q_D	Q_C	Q_B
0	0	1	0	0
1	1	0	1	0
2	1	1	0	1
3	1	1	1	0
4	0	1	1	1
5	0	0	1	1
6	0	0	0	1
7	1	0	0	0

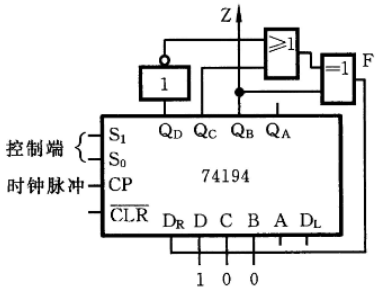


图 7.36　逻辑电路

- 例7.18：采用DAC0832芯片作为波形发生器，产生n个三角波信号。
- 解：
  - 图7.50：DAC0832与PC总线的连接。采用单缓冲方式（DAC寄存器处于直通状态）。
  - 因此， /XFER=0，/WR_2=0 ，1，/CS接 码输出信号，/WR_1接/IOW控制信号。
  - 产生n个三角波的程序：

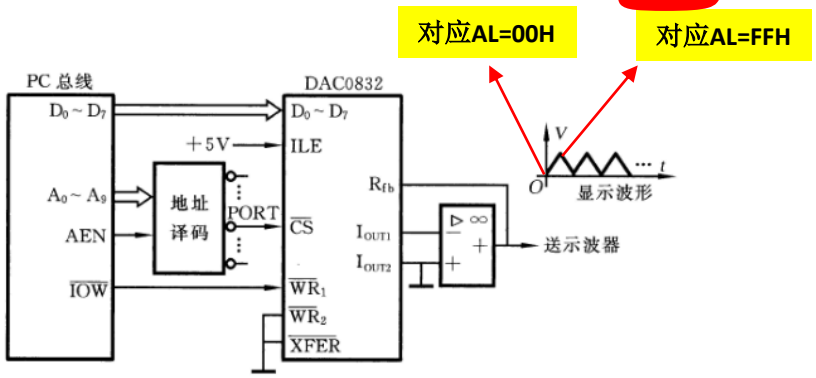


图 7.50　三角波信号发生器硬件连接图

```
TRIANGULARWAVE  PROC  FAR
        MOV   AX, DATA
        MOV   DS, AX          ;初始化 DS
        MOV   DX, PORT        ; 置端口地址为 PORT
        MOV   CX, n           ; 置循环次数 CX=n
        MOV   AL, 00H         ; 置数字量初值 AL=00H
L1:     OUT   DX, AL          ; 输出 AL 到 DAC0832
        INC   AL              ; AL 加 1
        JNZ   L1              ; 不为 0，继续输出
        MOV   AL, FEH         ; 为 0，重新置 AL=FEH
L2:     OUT   DX, AL          ; 输出 AL 到 DAC0832
        DEC   AL              ; AL 减 1
        JNZ   L2              ; 不为 0，继续输出
        DEC   CX              ; 为 0，将循环次数减 1
        JNZ   L1              ; 循环次数不为 0，继续生成下一个三角波
TRIANGULARWAVE  ENDP
```

关于期末考试

- 试卷题型
  1. 填空题：20分（每1空1分）
  2. 简答题：20分
  3. 综合题：60分

- 考试范围和试卷难度不超过每章的例题和习题（不要求的除外）

祝同学们期末考试取得好成绩！

Thanks